

LeetCode Intern Prep

230+ Medium & Hard Problems (Hard $\geq$ Medium)

Arrays · Trees · Graphs · DP · and more

Company tags: [\[G\]](#) Google [\[MS\]](#) Microsoft [\[A\]](#) Amazon [\[Meta\]](#) Meta [\[Ap\]](#) Apple

Sahil Raj (After Dark)

A curated set of 230+ LeetCode Medium and Hard problems for SWE intern season.

Each problem has a brief statement and a clean C++ solution.

Company tags: [\[G\]](#) Google, [\[MS\]](#) Microsoft, [\[A\]](#) Amazon, [\[Meta\]](#) Meta, [\[Ap\]](#) Apple.

Contents

1	Arrays & Two Pointers	11
□ 1.1	LC#15. 3Sum [Medium] [G] [MS] [A]	11
□ 1.2	LC#11. Container With Most Water [Medium] [G] [A] [Meta]	11
□ 1.3	LC#238. Product of Array Except Self [Medium] [G] [A] [MS] [Meta]	12
□ 1.4	LC#53. Maximum Subarray [Medium] [A] [MS]	12
□ 1.5	LC#189. Rotate Array [Medium] [MS] [A]	12
□ 1.6	LC#54. Spiral Matrix [Medium] [G] [MS] [A] [Ap]	12
□ 1.7	LC#73. Set Matrix Zeroes [Medium] [A] [MS] [Ap]	13
□ 1.8	LC#560. Subarray Sum Equals K [Medium] [G] [A] [Meta]	13
□ 1.9	LC#128. Longest Consecutive Sequence [Medium] [G] [A] [Meta]	14
□ 1.10	LC#18. 4Sum [Medium] [A]	14
□ 1.11	LC#45. Jump Game II [Medium] [A] [G]	14
□ 1.12	LC#287. Find the Duplicate Number [Medium] [A] [MS]	15
□ 1.13	LC#31. Next Permutation [Medium] [G] [MS]	15
□ 1.14	LC#75. Sort Colors [Medium] [A] [MS] [G]	15
□ 1.15	LC#169. Majority Element [Medium] [A] [MS] [G] [Meta]	16
2	Binary Search	17
□ 2.1	LC#33. Search in Rotated Sorted Array [Medium] [G] [A] [MS]	17
□ 2.2	LC#153. Find Minimum in Rotated Sorted Array [Medium] [G] [A]	17
□ 2.3	LC#162. Find Peak Element [Medium] [G] [MS] [A]	17
□ 2.4	LC#74. Search a 2D Matrix [Medium] [G] [MS]	18
□ 2.5	LC#875. Koko Eating Bananas [Medium] [A] [G]	18
□ 2.6	LC#4. Median of Two Sorted Arrays [Hard] [G] [A] [MS] [Ap]	18
□ 2.7	LC#1011. Capacity To Ship Packages [Medium] [A] [G]	19
□ 2.8	LC#81. Search in Rotated Sorted Array II [Medium] [A] [MS]	19
□ 2.9	LC#378. Kth Smallest in Sorted Matrix [Medium] [G]	20
□ 2.10	LC#540. Single Element in a Sorted Array [Medium] [A] [G] [MS]	20
3	Sliding Window & Strings	21
□ 3.1	LC#3. Longest Substring Without Repeating Characters [Medium] [G] [A] [MS] [Ap]	21
□ 3.2	LC#424. Longest Repeating Character Replacement [Medium] [G] [MS]	21
□ 3.3	LC#567. Permutation in String [Medium] [G] [MS] [A]	21
□ 3.4	LC#76. Minimum Window Substring [Hard] [G] [A] [MS] [Meta]	22
□ 3.5	LC#5. Longest Palindromic Substring [Medium] [A] [MS] [G]	22
□ 3.6	LC#49. Group Anagrams [Medium] [G] [A] [MS] [Meta]	23
□ 3.7	LC#242. Valid Anagram [Medium] [A] [MS]	23
□ 3.8	LC#271. Encode and Decode Strings [Medium] [G]	23
□ 3.9	LC#239. Sliding Window Maximum [Hard] [G] [A] [MS]	23
□ 3.10	LC#438. Find All Anagrams in a String [Medium] [A] [MS] [G]	24
4	Linked Lists	25
□ 4.1	LC#206. Reverse Linked List [Medium] [A] [MS] [G]	25
□ 4.2	LC#19. Remove Nth Node From End of List [Medium] [A] [MS] [G]	25
□ 4.3	LC#92. Reverse Linked List II [Medium] [A] [MS] [G]	26
□ 4.4	LC#142. Linked List Cycle II [Medium] [A] [MS]	26
□ 4.5	LC#143. Reorder List [Medium] [A] [MS] [G]	26
□ 4.6	LC#138. Copy List with Random Pointer [Medium] [A] [MS] [Meta]	27

□ 4.7	LC#146. LRU Cache	[Medium]	[A]	[MS]	[G]	[Meta]	27
□ 4.8	LC#23. Merge K Sorted Lists	[Hard]	[G]	[A]	[MS]	[Ap]	28
□ 4.9	LC#430. Flatten a Multilevel Doubly Linked List	[Medium]		[MS]			28
5	Trees & Binary Search Trees						29
□ 5.1	LC#226. Invert Binary Tree	[Medium]	[A]	[G]	[MS]		29
□ 5.2	LC#102. Binary Tree Level Order Traversal	[Medium]	[A]	[MS]	[G]		29
□ 5.3	LC#103. Binary Tree Zigzag Level Order	[Medium]	[A]	[MS]	[Meta]		29
□ 5.4	LC#105. Construct Binary Tree from Preorder and Inorder	[Medium]	[A]	[MS]	[G]		30
□ 5.5	LC#235. Lowest Common Ancestor of BST	[Medium]	[A]	[MS]	[G]	[Meta]	30
□ 5.6	LC#572. Subtree of Another Tree	[Medium]	[A]	[MS]	[G]		31
□ 5.7	LC#98. Validate Binary Search Tree	[Medium]	[A]	[MS]	[G]		31
□ 5.8	LC#230. Kth Smallest Element in a BST	[Medium]	[A]	[MS]			31
□ 5.9	LC#297. Serialize and Deserialize Binary Tree	[Hard]	[A]	[G]	[MS]	[Meta]	31
□ 5.10	LC#437. Path Sum III	[Medium]	[G]	[A]			32
□ 5.11	LC#114. Flatten Binary Tree to Linked List	[Medium]	[A]	[MS]			32
□ 5.12	LC#199. Binary Tree Right Side View	[Medium]	[A]	[MS]	[G]		33
□ 5.13	LC#543. Diameter of Binary Tree	[Medium]	[A]	[G]	[Ap]		33
6	Graphs						35
□ 6.1	LC#200. Number of Islands	[Medium]	[A]	[MS]	[G]	[Meta]	35
□ 6.2	LC#695. Max Area of Island	[Medium]	[A]	[MS]	[G]		35
□ 6.3	LC#133. Clone Graph	[Medium]	[A]	[MS]	[G]		36
□ 6.4	LC#207. Course Schedule (Cycle Detection)	[Medium]	[A]	[MS]	[G]		36
□ 6.5	LC#210. Course Schedule II	[Medium]	[A]	[MS]	[G]		36
□ 6.6	LC#417. Pacific Atlantic Water Flow	[Medium]	[A]	[G]			37
□ 6.7	LC#994. Rotting Oranges	[Medium]	[A]	[MS]	[G]		37
□ 6.8	LC#1091. Shortest Path in Binary Matrix	[Medium]	[A]	[G]	[MS]		38
□ 6.9	LC#269. Alien Dictionary	[Hard]	[G]	[Meta]			38
□ 6.10	LC#743. Network Delay Time	[Medium]	[A]	[G]			39
□ 6.11	LC#787. Cheapest Flights Within K Stops	[Medium]	[A]	[G]			39
□ 6.12	LC#323. Number of Connected Components	[Medium]	[G]	[MS]	[A]		40
□ 6.13	LC#684. Redundant Connection	[Medium]	[A]	[MS]			40
7	Dynamic Programming						41
□ 7.1	LC#198. House Robber	[Medium]	[A]	[MS]	[G]		41
□ 7.2	LC#213. House Robber II	[Medium]	[A]	[MS]	[G]		41
□ 7.3	LC#300. Longest Increasing Subsequence	[Medium]	[A]	[MS]	[G]		41
□ 7.4	LC#72. Edit Distance	[Hard]	[A]	[G]	[MS]		42
□ 7.5	LC#322. Coin Change	[Medium]	[A]	[G]	[MS]		42
□ 7.6	LC#518. Coin Change II	[Medium]	[A]	[G]			42
□ 7.7	LC#139. Word Break	[Medium]	[A]	[G]	[MS]	[Meta]	43
□ 7.8	LC#62. Unique Paths	[Medium]	[A]	[G]	[MS]		43
□ 7.9	LC#1143. Longest Common Subsequence	[Medium]	[A]	[G]	[MS]		43
□ 7.10	LC#337. House Robber III	[Medium]	[A]	[G]			44
□ 7.11	LC#416. Partition Equal Subset Sum	[Medium]	[A]	[G]	[MS]		44
□ 7.12	LC#91. Decode Ways	[Medium]	[A]	[MS]	[G]		44
□ 7.13	LC#221. Maximal Square	[Medium]	[A]	[MS]	[G]		45
□ 7.14	LC#55. Jump Game	[Medium]	[A]	[G]			45
□ 7.15	LC#188. Best Time to Buy and Sell Stock IV	[Hard]	[A]	[G]			45
8	Backtracking						47
□ 8.1	LC#78. Subsets	[Medium]	[A]	[MS]	[G]		47
□ 8.2	LC#90. Subsets II	[Medium]	[A]	[G]			47
□ 8.3	LC#46. Permutations	[Medium]	[A]	[MS]	[G]		48
□ 8.4	LC#77. Combinations	[Medium]	[G]	[A]	[MS]		48
□ 8.5	LC#39. Combination Sum	[Medium]	[A]	[G]	[MS]		48
□ 8.6	LC#40. Combination Sum II	[Medium]	[A]	[MS]			49
□ 8.7	LC#51. N-Queens	[Hard]	[A]	[G]	[MS]		49
□ 8.8	LC#79. Word Search	[Medium]	[A]	[MS]	[G]		49
□ 8.9	LC#131. Palindrome Partitioning	[Medium]	[A]	[G]	[MS]		50
□ 8.10	LC#17. Letter Combinations of a Phone Number	[Medium]	[A]	[MS]	[G]		50

9	Stacks, Queues & Heaps	53
9.1	LC#20. Valid Parentheses [Medium] [A] [G] [MS]	53
9.2	LC#155. Min Stack [Medium] [A] [MS] [G]	53
9.3	LC#496. Next Greater Element I [Medium] [A] [MS] [G]	53
9.4	LC#84. Largest Rectangle in Histogram [Hard] [A] [G] [MS]	54
9.5	LC#739. Daily Temperatures [Medium] [A] [MS] [G]	54
9.6	LC#853. Car Fleet [Medium] [A] [G]	54
9.7	LC#215. Kth Largest Element in an Array [Medium] [A] [MS] [G] [Meta]	55
9.8	LC#295. Find Median from Data Stream [Hard] [A] [MS] [G]	55
9.9	LC#347. Top K Frequent Elements [Medium] [A] [MS] [G] [Meta]	55
9.10	LC#355. Design Twitter [Medium] [A] [MS] [G]	56
9.11	LC#621. Task Scheduler [Medium] [A] [MS] [G]	56
10	Greedy, Intervals & Miscellaneous	57
10.1	LC#253. Meeting Rooms II [Medium] [G] [A] [MS]	57
10.2	LC#435. Non-overlapping Intervals [Medium] [G] [A] [MS]	57
10.3	LC#56. Merge Intervals [Medium] [A] [MS] [G]	57
10.4	LC#57. Insert Interval [Medium] [A] [G] [MS]	58
10.5	LC#134. Gas Station [Medium] [A] [MS] [G]	58
10.6	LC#406. Queue Reconstruction by Height [Medium] [A] [G] [MS]	58
10.7	LC#846. Hand of Straights [Medium] [G]	59
10.8	LC#452. Minimum Number of Arrows to Burst Balloons [Medium] [A] [G]	59
10.9	LC#1027. Longest Arithmetic Subsequence [Medium] [A] [G]	59
10.10	LC#179. Largest Number [Medium] [A] [MS]	60
10.11	LC#1306. Jump Game III [Medium] [A] [G]	60
10.12	LC#1229. Meeting Scheduler [Medium] [MS] [G]	60
10.13	LC#208. Implement Trie [Medium] [A] [MS] [G]	61
10.14	LC#212. Word Search II [Hard] [A] [G] [MS]	61
10.15	LC#973. K Closest Points to Origin [Medium] [A] [G] [MS]	62
10.16	LC#1167. Minimum Cost to Connect Sticks [Medium] [A]	62
10.17	LC#1851. Minimum Interval to Include Each Query [Hard] [G] [A]	63
10.18	LC#528. Random Pick with Weight [Medium] [A] [G]	63
10.19	LC#332. Reconstruct Itinerary [Hard] [G] [A]	63
10.20	LC#907. Sum of Subarray Minimums [Medium] [A] [MS]	64
10.21	LC#32. Longest Valid Parentheses [Hard] [A] [G] [MS]	64
10.22	LC#264. Ugly Number II [Medium] [A] [G]	64
10.23	LC#151. Reverse Words in a String [Medium] [A] [MS]	65
10.24	LC#443. String Compression [Medium] [A] [MS] [G]	65
10.25	LC#48. Rotate Image [Medium] [A] [MS] [G] [Ap]	65
10.26	LC#978. Longest Turbulent Subarray [Medium] [A] [G]	66
10.27	LC#2265. Count Nodes Equal to Average of Subtree [Medium] [A]	66
10.28	LC#652. Find Duplicate Subtrees [Medium] [A] [G]	66
10.29	LC#662. Maximum Width of Binary Tree [Medium] [A] [G]	67
10.30	LC#211. Design Add and Search Words Data Structure [Medium] [A] [MS]	67
10.31	LC#1235. Maximum Profit in Job Scheduling [Hard] [A] [G]	67
10.32	LC#721. Accounts Merge [Medium] [A] [G] [MS]	68
10.33	LC#227. Basic Calculator II [Medium] [A] [MS] [G]	68
10.34	LC#150. Minimum Number of Platforms [Medium] [A] [MS] [G]	69
11	Additional Hard Problems	71
11.1	Dynamic Programming	71
11.1.1	LC#44. Wildcard Matching [Hard] [G] [MS] [A]	71
11.1.2	LC#87. Scramble String [Hard] [A]	71
11.1.3	LC#123. Best Time to Buy and Sell Stock III [Hard] [A] [MS]	71
11.1.4	LC#140. Word Break II [Hard] [A] [G] [Meta]	72
11.1.5	LC#174. Dungeon Game [Hard] [A] [G]	72
11.1.6	LC#329. Longest Increasing Path in a Matrix [Hard] [G] [A]	72
11.1.7	LC#354. Russian Doll Envelopes [Hard] [G] [A]	73
11.1.8	LC#363. Max Sum of Rectangle No Larger Than K [Hard] [G]	73
11.1.9	LC#403. Frog Jump [Hard] [A]	73
11.1.10	LC#446. Arithmetic Slices II – Subsequence [Hard] [G]	73

11.1.11	LC#514. Freedom Trail	[Hard]	[G]	74
11.1.12	LC#546. Remove Boxes	[Hard]	[G]	74
11.1.13	LC#552. Student Attendance Record II	[Hard]	[G]	74
11.1.14	LC#629. K Inverse Pairs Array	[Hard]	[G]	75
11.1.15	LC#639. Decode Ways II	[Hard]	[G] [A]	75
11.1.16	LC#664. Strange Printer	[Hard]	[G]	75
11.1.17	LC#689. Maximum Sum of 3 Non-Overlapping Subarrays	[Hard]	[A]	76
11.1.18	LC#730. Count Different Palindromic Subsequences	[Hard]	[G]	76
11.1.19	LC#887. Super Egg Drop	[Hard]	[A] [G] [MS]	76
11.1.20	LC#940. Distinct Subsequences II	[Hard]	[G]	77
11.1.21	LC#956. Tallest Billboard	[Hard]	[A] [G]	77
11.1.22	LC#975. Odd Even Jump	[Hard]	[G]	77
11.1.23	LC#980. Unique Paths III	[Hard]	[G]	77
11.1.24	LC#1000. Minimum Cost to Merge Stones	[Hard]	[G]	78
11.1.25	LC#1125. Smallest Sufficient Team	[Hard]	[A]	78
11.1.26	LC#1220. Count Vowels Permutation	[Hard]	[G]	78
11.1.27	LC#1312. Minimum Insertion Steps to Make a String Palindrome	[Hard]	[A]	79
11.1.28	LC#1335. Minimum Difficulty of a Job Schedule	[Hard]	[A]	79
11.1.29	LC#1425. Constrained Subsequence Sum	[Hard]	[A]	79
11.1.30	LC#1444. Number of Ways of Cutting a Pizza	[Hard]	[G]	80
11.1.31	LC#1458. Max Dot Product of Two Subsequences	[Hard]	[G]	80
11.1.32	LC#1463. Cherry Pickup II	[Hard]	[A]	80
11.1.33	LC#1531. String Compression II	[Hard]	[G]	81
11.1.34	LC#1671. Minimum Number of Removals to Make Mountain Array	[Hard]	[G]	81
11.1.35	LC#1691. Maximum Height by Stacking Cuboids	[Hard]	[A]	81
11.1.36	LC#1751. Maximum Number of Events That Can Be Attended II	[Hard]	[A]	82
11.1.37	LC#1359. Count All Valid Pickup and Delivery Options	[Hard]	[A]	82
11.1.38	LC#1411. Number of Ways to Paint N x 3 Grid	[Hard]	[A]	82
□ 11.2	Binary Search			82
11.2.1	LC#154. Find Min in Rotated Sorted Array II	[Hard]	[A]	82
11.2.2	LC#164. Maximum Gap	[Hard]	[A]	83
11.2.3	LC#410. Split Array Largest Sum	[Hard]	[G] [A] [MS]	83
11.2.4	LC#668. Kth Smallest Number in Multiplication Table	[Hard]	[G]	83
11.2.5	LC#1095. Find in Mountain Array	[Hard]	[A] [G]	83
□ 11.3	Strings & Parsing			84
11.3.1	LC#68. Text Justification	[Hard]	[G] [MS]	84
11.3.2	LC#214. Shortest Palindrome	[Hard]	[G]	84
11.3.3	LC#224. Basic Calculator	[Hard]	[A] [G] [MS]	84
11.3.4	LC#273. Integer to English Words	[Hard]	[A] [MS] [G]	85
11.3.5	LC#472. Concatenated Words	[Hard]	[A]	85
11.3.6	LC#726. Number of Atoms	[Hard]	[G]	85
11.3.7	LC#1032. Stream of Characters	[Hard]	[A]	86
11.3.8	LC#1044. Longest Duplicate Substring	[Hard]	[A]	86
□ 11.4	Divide & Conquer / Merge Sort			86
11.4.1	LC#315. Count of Smaller Numbers After Self	[Hard]	[G] [A]	86
11.4.2	LC#493. Reverse Pairs	[Hard]	[A] [G]	87
□ 11.5	Graphs			87
11.5.1	LC#847. Shortest Path Visiting All Nodes	[Hard]	[G] [A]	87
11.5.2	LC#1203. Sort Items by Groups Respecting Dependencies	[Hard]	[G]	87
11.5.3	LC#1857. Largest Color Value in a Directed Graph	[Hard]	[G]	88
11.5.4	LC#2050. Parallel Courses III	[Hard]	[A]	88
□ 11.6	Heaps & Priority Queues			88
11.6.1	LC#480. Sliding Window Median	[Hard]	[A] [G]	89
11.6.2	LC#632. Smallest Range Covering K Lists	[Hard]	[A] [G]	89
11.6.3	LC#871. Minimum Number of Refueling Stops	[Hard]	[A] [G]	89
11.6.4	LC#895. Maximum Frequency Stack	[Hard]	[A] [G] [MS]	89
□ 11.7	Greedy			90
11.7.1	LC#330. Patching Array	[Hard]	[G]	90
11.7.2	LC#517. Super Washing Machines	[Hard]	[G] [A]	90
11.7.3	LC#992. Subarrays with K Different Integers	[Hard]	[G] [A]	90
11.7.4	LC#995. Minimum Number of K Consecutive Bit Flips	[Hard]	[G]	91

□ 11.8 Math & Geometry	91
11.8.1 LC#149. Max Points on a Line [Hard] [G] [A]	91
11.8.2 LC#233. Number of Digit One [Hard] [MS] [A]	91
11.8.3 LC#321. Create Maximum Number [Hard] [G]	91
11.8.4 LC#391. Perfect Rectangle [Hard] [G]	92
11.8.5 LC#679. 24 Game [Hard] [G]	92
□ 11.9 Backtracking	92
11.9.1 LC#282. Expression Add Operators [Hard] [A] [G] [Meta]	92
11.9.2 LC#301. Remove Invalid Parentheses [Hard] [A] [Meta]	93
11.9.3 LC#336. Palindrome Pairs [Hard] [A] [G]	93
11.9.4 LC#753. Cracking the Safe [Hard] [G]	94
□ 11.10 Design & Data Structures	94
11.10.1 LC#381. Insert Delete GetRandom O(1) – Duplicates Allowed [Hard] [A]	94
11.10.2 LC#432. All O'one Data Structure [Hard] [G]	94
11.10.3 LC#732. My Calendar III [Hard] [G]	94
□ 11.11 Intervals & Scheduling	95
11.11.1 LC#759. Employee Free Time [Hard] [G] [A]	95
11.11.2 LC#765. Couples Holding Hands [Hard] [G]	95
11.11.3 LC#1499. Max Value of Equation [Hard] [A]	95
12 Medium Problems — Extended Set	97
□ 12.1 Arrays & Two Pointers	97
12.1.1 LC#80. Remove Duplicates from Sorted Array II [Medium] [G] [MS]	97
12.1.2 LC#209. Minimum Size Subarray Sum [Medium] [A] [MS] [G]	97
12.1.3 LC#334. Increasing Triplet Subsequence [Medium] [A] [MS]	97
12.1.4 LC#523. Continuous Subarray Sum [Medium] [A] [G]	98
12.1.5 LC#525. Contiguous Array [Medium] [A] [MS]	98
12.1.6 LC#904. Fruit Into Baskets [Medium] [A] [G]	98
12.1.7 LC#974. Subarray Sums Divisible by K [Medium] [A] [G]	99
12.1.8 LC#1004. Max Consecutive Ones III [Medium] [A] [MS]	99
12.1.9 LC#1248. Count Number of Nice Subarrays [Medium] [A]	99
12.1.10 LC#1493. Longest Subarray of 1s After Deleting One Element [Medium] [A]	100
12.1.11 LC#1658. Minimum Operations to Reduce X to Zero [Medium] [A] [MS]	100
12.1.12 LC#152. Maximum Product Subarray [Medium] [A] [MS] [G]	100
12.1.13 LC#845. Longest Mountain in Array [Medium] [A] [G]	100
12.1.14 LC#930. Binary Subarrays With Sum [Medium] [A] [G]	101
12.1.15 LC#1574. Shortest Subarray to be Removed to Make Array Sorted [Medium] [MS]	101
12.1.16 LC#2401. Longest Nice Subarray [Medium] [A]	101
12.1.17 LC#498. Diagonal Traverse [Medium] [A] [MS]	102
12.1.18 LC#667. Beautiful Arrangement II [Medium] [A]	102
□ 12.2 Binary Search	102
12.2.1 LC#34. Find First and Last Position of Element [Medium] [A] [MS] [G]	102
12.2.2 LC#658. Find K Closest Elements [Medium] [A] [MS] [G]	103
12.2.3 LC#1283. Find the Smallest Divisor Given a Threshold [Medium] [A] [G]	103
12.2.4 LC#1482. Minimum Number of Days to Make m Bouquets [Medium] [A] [G]	103
12.2.5 LC#1552. Magnetic Force Between Two Balls [Medium] [A] [G]	104
12.2.6 LC#1760. Minimum Limit of Balls in a Bag [Medium] [A]	104
12.2.7 LC#275. H-Index II [Medium] [A] [G]	104
12.2.8 LC#2300. Successful Pairs of Spells and Potions [Medium] [A]	105
12.2.9 LC#2226. Maximum Candies Allocated to K Children [Medium] [A]	105
12.2.10 LC#2064. Minimized Maximum of Products Distributed [Medium] [G]	105
□ 12.3 Strings	105
12.3.1 LC#6. Zigzag Conversion [Medium] [A] [MS]	105
12.3.2 LC#12. Integer to Roman [Medium] [A] [MS]	106
12.3.3 LC#38. Count and Say [Medium] [A]	106
12.3.4 LC#43. Multiply Strings [Medium] [A] [MS]	106
12.3.5 LC#150. Evaluate Reverse Polish Notation [Medium] [A] [MS]	107
12.3.6 LC#165. Compare Version Numbers [Medium] [A] [MS]	107
12.3.7 LC#394. Decode String [Medium] [A] [MS] [G]	107
12.3.8 LC#539. Minimum Time Difference [Medium] [A]	108
12.3.9 LC#647. Palindromic Substrings [Medium] [A] [MS] [G]	108

12.3.10	LC#763. Partition Labels	[Medium]	[A]	[MS]	[G]	109
12.3.11	LC#1047. Remove All Adjacent Duplicates in String	[Medium]	[A]			109
12.3.12	LC#1209. Remove All Adjacent Duplicates in String II	[Medium]	[A]	[MS]		109
□ 12.4	Linked List					109
12.4.1	LC#2. Add Two Numbers	[Medium]	[A]	[MS]	[G]	110
12.4.2	LC#24. Swap Nodes in Pairs	[Medium]	[A]	[MS]		110
12.4.3	LC#61. Rotate List	[Medium]	[A]	[MS]		110
12.4.4	LC#82. Remove Duplicates from Sorted List II	[Medium]	[A]	[MS]		110
12.4.5	LC#86. Partition List	[Medium]	[A]	[MS]		111
12.4.6	LC#328. Odd Even Linked List	[Medium]	[A]	[MS]	[G]	111
12.4.7	LC#445. Add Two Numbers II	[Medium]	[A]	[MS]		111
12.4.8	LC#876. Middle of the Linked List	[Medium]	[A]	[MS]		112
□ 12.5	Trees & Binary Search Trees					112
12.5.1	LC#116. Populating Next Right Pointers	[Medium]	[A]	[MS]		112
12.5.2	LC#129. Sum Root to Leaf Numbers	[Medium]	[A]	[MS]		112
12.5.3	LC#173. Binary Search Tree Iterator	[Medium]	[A]	[MS]	[G]	113
12.5.4	LC#222. Count Complete Tree Nodes	[Medium]	[A]	[MS]		113
12.5.5	LC#236. Lowest Common Ancestor of a Binary Tree	[Medium]	[A]	[MS]	[G] [Meta]	113
12.5.6	LC#450. Delete Node in a BST	[Medium]	[A]	[MS]		114
12.5.7	LC#538. Convert BST to Greater Tree	[Medium]	[A]	[MS]		114
12.5.8	LC#669. Trim a Binary Search Tree	[Medium]	[A]	[MS]		114
12.5.9	LC#701. Insert into a Binary Search Tree	[Medium]	[A]	[MS]		115
12.5.10	LC#863. All Nodes Distance K in Binary Tree	[Medium]	[A]	[MS]	[G]	115
12.5.11	LC#958. Check Completeness of Binary Tree	[Medium]	[A]			115
12.5.12	LC#1161. Maximum Level Sum of a Binary Tree	[Medium]	[A]			116
12.5.13	LC#1325. Delete Leaves With a Given Value	[Medium]	[A]			116
12.5.14	LC#889. Construct Binary Tree from Preorder and Postorder	[Medium]		[G]		116
□ 12.6	Graphs					117
12.6.1	LC#130. Surrounded Regions	[Medium]	[A]	[MS]	[G]	117
12.6.2	LC#261. Graph Valid Tree	[Medium]		[G]	[A]	117
12.6.3	LC#286. Walls and Gates	[Medium]	[A]	[G]	[MS]	117
12.6.4	LC#547. Number of Provinces	[Medium]	[A]	[MS]	[G]	118
12.6.5	LC#785. Is Graph Bipartite?	[Medium]	[A]	[MS]	[G]	118
12.6.6	LC#797. All Paths From Source to Target	[Medium]	[A]	[G]		118
12.6.7	LC#841. Keys and Rooms	[Medium]	[A]	[G]		119
12.6.8	LC#886. Possible Bipartition	[Medium]	[A]	[G]		119
12.6.9	LC#1020. Number of Enclaves	[Medium]	[A]	[G]		119
12.6.10	LC#1319. Number of Operations to Make Network Connected	[Medium]	[A]	[G]		120
12.6.11	LC#1376. Time Needed to Inform All Employees	[Medium]	[A]	[MS]		120
12.6.12	LC#1466. Reorder Routes to Make All Paths Lead to City Zero	[Medium]	[A]	[G]		120
□ 12.7	Dynamic Programming					121
12.7.1	LC#96. Unique Binary Search Trees	[Medium]	[A]	[MS]	[G]	121
12.7.2	LC#279. Perfect Squares	[Medium]	[A]	[MS]	[G]	121
12.7.3	LC#309. Best Time to Buy and Sell Stock with Cooldown	[Medium]	[A]	[G]		121
12.7.4	LC#343. Integer Break	[Medium]	[A]	[G]		122
12.7.5	LC#376. Wiggle Subsequence	[Medium]	[A]	[G]		122
12.7.6	LC#377. Combination Sum IV	[Medium]	[A]	[MS]	[G]	122
12.7.7	LC#413. Arithmetic Slices	[Medium]	[A]			123
12.7.8	LC#486. Predict the Winner	[Medium]	[A]	[MS]	[G]	123
12.7.9	LC#494. Target Sum	[Medium]	[A]	[MS]	[G]	123
12.7.10	LC#516. Longest Palindromic Subsequence	[Medium]	[A]	[MS]	[G]	123
12.7.11	LC#576. Out of Boundary Paths	[Medium]	[A]	[G]		124
12.7.12	LC#583. Delete Operation for Two Strings	[Medium]	[A]	[G]		124
12.7.13	LC#650. 2 Keys Keyboard	[Medium]	[A]	[G]		125
12.7.14	LC#673. Number of Longest Increasing Subsequences	[Medium]	[A]	[MS]		125
12.7.15	LC#740. Delete and Earn	[Medium]	[A]	[MS]		125
12.7.16	LC#877. Stone Game	[Medium]	[A]	[MS]		126
12.7.17	LC#931. Minimum Falling Path Sum	[Medium]	[A]	[G]		126
12.7.18	LC#983. Minimum Cost for Tickets	[Medium]	[A]	[G]		126
12.7.19	LC#688. Knight Probability in Chessboard	[Medium]	[A]	[G]		126
12.7.20	LC#97. Interleaving String	[Medium]	[A]	[MS]	[G]	127

□ 12.8	Jump Game Variants	127
12.8.1	LC#55. Jump Game [Medium] [A] [MS] [G]	127
12.8.2	LC#45. Jump Game II [Medium] [A] [MS] [G]	128
12.8.3	LC#1306. Jump Game III [Medium] [A] [G]	128
12.8.4	LC#1696. Jump Game VI [Medium] [A] [G]	128
12.8.5	LC#1871. Jump Game VII [Medium] [A] [G]	129
12.8.6	LC#1654. Minimum Jumps to Reach Home [Medium] [A]	129
□ 12.9	Backtracking	129
12.9.1	LC#22. Generate Parentheses [Medium] [A] [MS] [G]	129
12.9.2	LC#47. Permutations II [Medium] [A] [MS]	130
12.9.3	LC#93. Restore IP Addresses [Medium] [A] [MS]	130
12.9.4	LC#216. Combination Sum III [Medium] [A]	130
12.9.5	LC#526. Beautiful Arrangement [Medium] [A]	131
12.9.6	LC#784. Letter Case Permutation [Medium] [A] [MS]	131
12.9.7	LC#967. Numbers With Same Consecutive Differences [Medium] [A]	131
12.9.8	LC#1079. Letter Tile Possibilities [Medium] [A]	132
□ 12.10	Stack & Monotonic Stack	132
12.10.1	LC#71. Simplify Path [Medium] [A] [MS] [G]	132
12.10.2	LC#316. Remove Duplicate Letters [Medium] [A] [MS] [G]	133
12.10.3	LC#402. Remove K Digits [Medium] [A] [MS] [G]	133
12.10.4	LC#456. 132 Pattern [Medium] [A] [MS] [G]	133
12.10.5	LC#503. Next Greater Element II [Medium] [A] [G]	134
12.10.6	LC#735. Asteroid Collision [Medium] [A] [MS] [G]	134
12.10.7	LC#856. Score of Parentheses [Medium] [A] [G]	134
12.10.8	LC#1019. Next Greater Node in Linked List [Medium] [A]	135
12.10.9	LC#2390. Removing Stars From a String [Medium] [A] [MS]	135
12.10.10	LC#946. Validate Stack Sequences [Medium] [A] [MS]	135
□ 12.11	Heaps & Priority Queues	135
12.11.1	LC#373. Find K Pairs with Smallest Sums [Medium] [A] [MS] [G]	135
12.11.2	LC#451. Sort Characters By Frequency [Medium] [A] [MS] [G]	136
12.11.3	LC#692. Top K Frequent Words [Medium] [A] [MS] [G]	136
12.11.4	LC#703. Kth Largest Element in a Stream [Medium] [A] [MS]	136
12.11.5	LC#767. Reorganize String [Medium] [A] [MS] [G]	137
12.11.6	LC#1438. Longest Continuous Subarray With Absolute Diff $\leq$ Limit [Medium] [A] [G]	137
12.11.7	LC#1642. Furthest Building You Can Reach [Medium] [A] [G]	137
□ 12.12	Greedy & Intervals	138
12.12.1	LC#122. Best Time to Buy and Sell Stock II [Medium] [A] [MS] [G]	138
12.12.2	LC#605. Can Place Flowers [Medium] [A] [MS]	138
12.12.3	LC#649. Dota2 Senate [Medium] [A] [MS]	138
12.12.4	LC#714. Best Time to Buy and Sell Stock with Transaction Fee [Medium] [A] [G]	139
12.12.5	LC#826. Most Profit Assigning Work [Medium] [A] [G]	139
12.12.6	LC#881. Boats to Save People [Medium] [A] [G]	139
12.12.7	LC#948. Bag of Tokens [Medium] [A]	140
12.12.8	LC#1029. Two City Scheduling [Medium] [A] [G]	140
12.12.9	LC#870. Advantage Shuffle [Medium] [A] [G]	140
12.12.10	LC#1053. Previous Permutation With One Swap [Medium] [A]	141
□ 12.13	Math & Bit Manipulation	141
12.13.1	LC#50. Pow(x, n) [Medium] [A] [MS] [G]	141
12.13.2	LC#137. Single Number II [Medium] [A] [MS] [G]	141
12.13.3	LC#201. Bitwise AND of Numbers Range [Medium] [A] [MS]	141
12.13.4	LC#260. Single Number III [Medium] [A] [G]	142
12.13.5	LC#371. Sum of Two Integers [Medium] [A] [MS]	142
12.13.6	LC#338. Counting Bits [Medium] [A] [MS] [G]	142
12.13.7	LC#397. Integer Replacement [Medium] [A]	142
12.13.8	LC#477. Total Hamming Distance [Medium] [A] [MS]	143
12.13.9	LC#779. K-th Symbol in Grammar [Medium] [A]	143
12.13.10	LC#29. Divide Two Integers [Medium] [A] [MS]	143

Chapter 1

Arrays & Two Pointers

1.1 LC#15. 3Sum [\[Medium\]](#) [\[G\]](#) [\[MS\]](#) [\[A\]](#)

Problem

Given array `nums`, return all unique triplets that sum to zero.

```
1 class Solution {
2 public:
3     vector<vector<int>>> threeSum(vector<int>& nums) {
4         sort(nums.begin(), nums.end());
5         vector<vector<int>>> res;
6         int n = nums.size();
7         for (int i = 0; i < n - 2; i++) {
8             if (i > 0 && nums[i] == nums[i-1]) continue;
9             int l = i + 1, r = n - 1;
10            while (l < r) {
11                int s = nums[i] + nums[l] + nums[r];
12                if (s == 0) {
13                    res.push_back({nums[i], nums[l], nums[r]});
14                    while (l < r && nums[l] == nums[l+1]) l++;
15                    while (l < r && nums[r] == nums[r-1]) r--;
16                    l++;
17                    r--;
18                } else if (s < 0) {
19                    l++;
20                } else {
21                    r--;
22                }
23            }
24        }
25        return res;
26    }
27};
```

1.2 LC#11. Container With Most Water [\[Medium\]](#) [\[G\]](#) [\[A\]](#) [\[Meta\]](#)

Problem

Given heights, find two lines that together with the x-axis form a container that holds the most water.

```
1 class Solution {
2 public:
3     int maxArea(vector<int>& h) {
4         int l = 0, r = h.size() - 1, res = 0;
5         while (l < r) {
6             res = max(res, min(h[l], h[r]) * (r - l));
7             if (h[l] < h[r]) l++;
8             else r--;
9         }
10        return res;
11    }
12};
```

1.3 LC#238. Product of Array Except Self [Medium] [G] [A] [MS] [Meta]

Problem

Return array output where $\text{output}[i] = \text{product of all elements except } \text{nums}[i]$, without division, in $O(n)$.

```

1 class Solution {
2 public:
3     vector<int> productExceptSelf(vector<int>& nums) {
4         int n = nums.size();
5         vector<int> res(n, 1);
6         int pre = 1, suf = 1;
7         for (int i = 0; i < n; i++) {
8             res[i] = pre;
9             pre *= nums[i];
10        }
11        for (int i = n - 1; i >= 0; i--) {
12            res[i] *= suf;
13            suf *= nums[i];
14        }
15        return res;
16    }
17 };

```

1.4 LC#53. Maximum Subarray [Medium] [A] [MS]

Problem

Find the contiguous subarray with the largest sum (Kadane's algorithm).

```

1 class Solution {
2 public:
3     int maxSubArray(vector<int>& nums) {
4         int cur = nums[0], res = nums[0];
5         for (int i = 1; i < (int)nums.size(); i++) {
6             cur = max(nums[i], cur + nums[i]);
7             res = max(res, cur);
8         }
9         return res;
10    }
11 };

```

1.5 LC#189. Rotate Array [Medium] [MS] [A]

Problem

Rotate an array to the right by k steps in-place with $O(1)$ extra space.

```

1 class Solution {
2 public:
3     void rotate(vector<int>& nums, int k) {
4         int n = nums.size();
5         k %= n;
6         reverse(nums.begin(), nums.end());
7         reverse(nums.begin(), nums.begin() + k);
8         reverse(nums.begin() + k, nums.end());
9     }
10 };

```

1.6 LC#54. Spiral Matrix [Medium] [G] [MS] [A] [Ap]

Problem

Return all elements of an $m \times n$ matrix in spiral order.

```

1 class Solution {
2 public:

```

```

3     vector<int> spiralOrder(vector<vector<int>>& mat) {
4         vector<int> res;
5         int top = 0, bot = mat.size() - 1;
6         int left = 0, right = mat[0].size() - 1;
7         while (top <= bot && left <= right) {
8             for (int i = left; i <= right; i++) res.push_back(mat[top][i]);
9             top++;
10            for (int i = top; i <= bot; i++) res.push_back(mat[i][right]);
11            right--;
12            if (top <= bot) {
13                for (int i = right; i >= left; i--) res.push_back(mat[bot][i]);
14                bot--;
15            }
16            if (left <= right) {
17                for (int i = bot; i >= top; i--) res.push_back(mat[i][left]);
18                left++;
19            }
20        }
21        return res;
22    }
23 };

```

1.7 LC#73. Set Matrix Zeroes [Medium] [A] [MS] [Ap]

Problem

If an element is 0, set its entire row and column to 0. Do it in-place with $O(1)$ extra space.

```

1     class Solution {
2     public:
3         void setZeroes(vector<vector<int>>& mat) {
4             int m = mat.size(), n = mat[0].size();
5             bool firstRow = false, firstCol = false;
6
7             for (int j = 0; j < n; j++) if (mat[0][j] == 0) firstRow = true;
8             for (int i = 0; i < m; i++) if (mat[i][0] == 0) firstCol = true;
9
10            for (int i = 1; i < m; i++)
11                for (int j = 1; j < n; j++)
12                    if (mat[i][j] == 0) mat[i][0] = mat[0][j] = 0;
13
14            for (int i = 1; i < m; i++)
15                for (int j = 1; j < n; j++)
16                    if (mat[i][0] == 0 || mat[0][j] == 0) mat[i][j] = 0;
17
18            if (firstRow) for (int j = 0; j < n; j++) mat[0][j] = 0;
19            if (firstCol) for (int i = 0; i < m; i++) mat[i][0] = 0;
20        }
21    };

```

1.8 LC#560. Subarray Sum Equals K [Medium] [G] [A] [Meta]

Problem

Given `nums` and integer `k`, return the total number of subarrays whose sum equals `k`.

```

1     class Solution {
2     public:
3         int subarraySum(vector<int>& nums, int k) {
4             unordered_map<int, int> mp{{0, 1}};
5             int pre = 0, res = 0;
6             for (int x : nums) {
7                 pre += x;
8                 res += mp[pre - k];
9                 mp[pre]++;
10            }
11            return res;
12        }
13    };

```

1.9 LC#128. Longest Consecutive Sequence [Medium] [G] [A] [Meta]

Problem

Given unsorted array, return length of longest consecutive elements sequence in $O(n)$.

```

1 class Solution {
2 public:
3     int longestConsecutive(vector<int>& nums) {
4         unordered_set<int> s(nums.begin(), nums.end());
5         int res = 0;
6         for (int x : s) {
7             if (s.count(x - 1)) continue;
8             int len = 1;
9             while (s.count(x + len)) len++;
10            res = max(res, len);
11        }
12        return res;
13    }
14 };

```

1.10 LC#18. 4Sum [Medium] [A]

Problem

Find all unique quadruplets in nums that sum to target.

```

1 class Solution {
2 public:
3     vector<vector<int>>> fourSum(vector<int>& nums, int target) {
4         sort(nums.begin(), nums.end());
5         int n = nums.size();
6         vector<vector<int>>> res;
7         for (int i = 0; i < n - 3; i++) {
8             if (i > 0 && nums[i] == nums[i-1]) continue;
9             for (int j = i + 1; j < n - 2; j++) {
10                if (j > i + 1 && nums[j] == nums[j-1]) continue;
11                int l = j + 1, r = n - 1;
12                while (l < r) {
13                    long long s = (long long)nums[i] + nums[j] + nums[l] + nums[r];
14                    if (s == target) {
15                        res.push_back({nums[i], nums[j], nums[l], nums[r]});
16                        while (l < r && nums[l] == nums[l+1]) l++;
17                        while (l < r && nums[r] == nums[r-1]) r--;
18                        l++;
19                        r--;
20                    } else if (s < target) {
21                        l++;
22                    } else {
23                        r--;
24                    }
25                }
26            }
27        }
28        return res;
29    }
30 };

```

1.11 LC#45. Jump Game II [Medium] [A] [G]

Problem

Return the minimum number of jumps to reach the last index of nums.

```

1 class Solution {
2 public:
3     int jump(vector<int>& nums) {
4         int jumps = 0, cur = 0, far = 0;
5         for (int i = 0; i < (int)nums.size() - 1; i++) {
6             far = max(far, i + nums[i]);
7             if (i == cur) {
8                 jumps++;
9                 cur = far;
10            }
11        }
12        return jumps;
13    }
14 };

```


1.12 LC#287. Find the Duplicate Number [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Given `nums` of $n + 1$ integers in $[1, n]$, find the duplicate without modifying array, $O(1)$ space.

```

1 class Solution {
2 public:
3     int findDuplicate(vector<int>& nums) {
4         int slow = nums[0], fast = nums[0];
5         do {
6             slow = nums[slow];
7             fast = nums[nums[fast]];
8         } while (slow != fast);
9
10        fast = nums[0];
11        while (slow != fast) {
12            slow = nums[slow];
13            fast = nums[fast];
14        }
15        return slow;
16    }
17 };

```

1.13 LC#31. Next Permutation [\[Medium\]](#) [\[G\]](#) [\[MS\]](#)

Problem

Rearrange `nums` into the lexicographically next greater permutation in-place.

```

1 class Solution {
2 public:
3     void nextPermutation(vector<int>& nums) {
4         int n = nums.size(), i = n - 2;
5         while (i >= 0 && nums[i] >= nums[i+1]) i--;
6         if (i >= 0) {
7             int j = n - 1;
8             while (nums[j] <= nums[i]) j--;
9             swap(nums[i], nums[j]);
10        }
11        reverse(nums.begin() + i + 1, nums.end());
12    }
13 };

```

1.14 LC#75. Sort Colors [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Sort an array of 0s, 1s, and 2s in-place (Dutch National Flag). One pass, $O(1)$ extra space.

```

1 class Solution {
2 public:
3     void sortColors(vector<int>& nums) {
4         int lo = 0, mid = 0, hi = nums.size() - 1;
5         while (mid <= hi) {
6             if (nums[mid] == 0) {
7                 swap(nums[lo], nums[mid]);
8                 lo++;
9                 mid++;
10            } else if (nums[mid] == 1) {
11                mid++;
12            } else {
13                swap(nums[mid], nums[hi]);
14                hi--;
15            }
16        }
17    }
18 };

```

1.15 LC#169. Majority Element [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#) [\[Meta\]](#)

Problem

Find the element that appears more than $\lfloor n/2 \rfloor$ times. Solve in $O(n)$ time, $O(1)$ space (Boyer-Moore voting).

```
1 class Solution {
2 public:
3     int majorityElement(vector<int>& nums) {
4         int candidate = nums[0], count = 1;
5         for (int i = 1; i < (int)nums.size(); i++) {
6             if (count == 0) {
7                 candidate = nums[i];
8                 count = 1;
9             } else if (nums[i] == candidate) {
10                count++;
11            } else {
12                count--;
13            }
14        }
15        return candidate;
16    }
17};
```

Chapter 2

Binary Search

2.1 LC#33. Search in Rotated Sorted Array [Medium] [G] [A] [MS]

Problem

Search target in a rotated sorted array in $O(\log n)$.

```
1 class Solution {
2 public:
3     int search(vector<int>& nums, int target) {
4         int l = 0, r = nums.size() - 1;
5         while (l <= r) {
6             int m = (l + r) / 2;
7             if (nums[m] == target) return m;
8             if (nums[l] <= nums[m]) {
9                 if (nums[l] <= target && target < nums[m]) r = m - 1;
10                else l = m + 1;
11            } else {
12                if (nums[m] < target && target <= nums[r]) l = m + 1;
13                else r = m - 1;
14            }
15        }
16        return -1;
17    }
18};
```

2.2 LC#153. Find Minimum in Rotated Sorted Array [Medium] [G] [A]

Problem

Find minimum element in a rotated sorted array (no duplicates) in $O(\log n)$.

```
1 class Solution {
2 public:
3     int findMin(vector<int>& nums) {
4         int l = 0, r = nums.size() - 1;
5         while (l < r) {
6             int m = (l + r) / 2;
7             if (nums[m] > nums[r]) l = m + 1;
8             else r = m;
9         }
10        return nums[l];
11    }
12};
```

2.3 LC#162. Find Peak Element [Medium] [G] [MS] [A]

Problem

A peak element is strictly greater than its neighbours. Return any peak index in $O(\log n)$.

```

1 class Solution {
2 public:
3     int findPeakElement(vector<int>& nums) {
4         int l = 0, r = nums.size() - 1;
5         while (l < r) {
6             int m = (l + r) / 2;
7             if (nums[m] > nums[m+1]) r = m;
8             else l = m + 1;
9         }
10        return l;
11    }
12};

```

2.4 LC#74. Search a 2D Matrix [Medium] [G] [MS]

Problem

Search in a sorted $m \times n$ matrix (each row sorted, first of row > last of previous) in $O(\log(mn))$.

```

1 class Solution {
2 public:
3     bool searchMatrix(vector<vector<int>>& mat, int target) {
4         int m = mat.size(), n = mat[0].size();
5         int l = 0, r = m * n - 1;
6         while (l <= r) {
7             int mid = (l + r) / 2;
8             int val = mat[mid / n][mid % n];
9             if (val == target) return true;
10            else if (val < target) l = mid + 1;
11            else r = mid - 1;
12        }
13        return false;
14    }
15};

```

2.5 LC#875. Koko Eating Bananas [Medium] [A] [G]

Problem

Find minimum eating speed k such that Koko can eat all piles in h hours.

```

1 class Solution {
2 public:
3     int minEatingSpeed(vector<int>& piles, int h) {
4         int l = 1, r = *max_element(piles.begin(), piles.end());
5         while (l < r) {
6             int m = (l + r) / 2;
7             long long hrs = 0;
8             for (int p : piles) hrs += (p + m - 1) / m;
9             if (hrs <= h) r = m;
10            else l = m + 1;
11        }
12        return l;
13    }
14};

```

2.6 LC#4. Median of Two Sorted Arrays [Hard] [G] [A] [MS] [Ap]

Problem

Find the median of two sorted arrays of sizes m and n in $O(\log(\min(m,n)))$.

```

1 class Solution {
2 public:
3     double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
4         if (A.size() > B.size()) swap(A, B);
5         int m = A.size(), n = B.size();
6         int lo = 0, hi = m;
7         while (lo <= hi) {
8             int i = (lo + hi) / 2;
9             int j = (m + n + 1) / 2 - i;
10            int lA = (i ? A[i-1] : INT_MIN), rA = (i < m ? A[i] : INT_MAX);

```

```

11         int lB = (j ? B[j-1] : INT_MIN), rB = (j < n ? B[j] : INT_MAX);
12         if (lA <= rB && lB <= rA) {
13             if ((m + n) % 2) return max(lA, lB);
14             return (max(lA, lB) + min(rA, rB)) / 2.0;
15         } else if (lA > rB) {
16             hi = i - 1;
17         } else {
18             lo = i + 1;
19         }
20     }
21     return 0;
22 }
23 };

```

2.7 LC#1011. Capacity To Ship Packages [Medium] [A] [G]

Problem

Find minimum weight capacity of a ship to ship all packages within `days` days.

```

1 class Solution {
2 public:
3     int shipWithinDays(vector<int>& weights, int days) {
4         int l = *max_element(weights.begin(), weights.end());
5         int r = accumulate(weights.begin(), weights.end(), 0);
6         while (l < r) {
7             int m = (l + r) / 2, cur = 0, d = 1;
8             for (int w : weights) {
9                 if (cur + w > m) {
10                     d++;
11                     cur = 0;
12                 }
13                 cur += w;
14             }
15             if (d <= days) r = m;
16             else l = m + 1;
17         }
18         return l;
19     }
20 };

```

2.8 LC#81. Search in Rotated Sorted Array II [Medium] [A] [MS]

Problem

Same as #33 but array may have duplicates. Return true/false if target exists.

```

1 class Solution {
2 public:
3     bool search(vector<int>& nums, int target) {
4         int l = 0, r = nums.size() - 1;
5         while (l <= r) {
6             int m = (l + r) / 2;
7             if (nums[m] == target) return true;
8             if (nums[l] == nums[m] && nums[m] == nums[r]) {
9                 l++;
10                r--;
11                continue;
12            }
13            if (nums[l] <= nums[m]) {
14                if (nums[l] <= target && target < nums[m]) r = m - 1;
15                else l = m + 1;
16            } else {
17                if (nums[m] < target && target <= nums[r]) l = m + 1;
18                else r = m - 1;
19            }
20        }
21        return false;
22    }
23 };

```

2.9 LC#378. Kth Smallest in Sorted Matrix [Medium] [G]

Problem

Given $n \times m$ matrix with each row and column sorted, find k th smallest element.

```

1  class Solution {
2  public:
3      int kthSmallest(vector<vector<int>>& mat, int k) {
4          int n = mat.size();
5          int l = mat[0][0], r = mat[n-1][n-1];
6          while (l < r) {
7              int m = (l + r) / 2, cnt = 0, j = n - 1;
8              for (int i = 0; i < n; i++) {
9                  while (j >= 0 && mat[i][j] > m) j--;
10                 cnt += j + 1;
11             }
12             if (cnt < k) l = m + 1;
13             else r = m;
14         }
15         return l;
16     }
17 };

```

2.10 LC#540. Single Element in a Sorted Array [Medium] [A] [G] [MS]

Problem

Every element appears twice except one. Find that element in $O(\log n)$ without extra space.

```

1  class Solution {
2  public:
3      int singleNonDuplicate(vector<int>& nums) {
4          int l = 0, r = nums.size() - 1;
5          while (l < r) {
6              int m = (l + r) / 2;
7              // make m even so we compare pairs starting at m
8              if (m % 2 == 1) m--;
9              if (nums[m] == nums[m+1]) l = m + 2;
10             else r = m;
11         }
12         return nums[l];
13     }
14 };

```

Chapter 3

Sliding Window & Strings

3.1 LC#3. Longest Substring Without Repeating Characters [Medium] [G] [A] [MS] [Ap]

Problem

Find the length of the longest substring without repeating characters.

```
1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         unordered_map<char, int> mp;
5         int l = 0, res = 0;
6         for (int r = 0; r < (int)s.size(); r++) {
7             if (mp.count(s[r])) l = max(l, mp[s[r]] + 1);
8             mp[s[r]] = r;
9             res = max(res, r - l + 1);
10        }
11        return res;
12    }
13};
```

3.2 LC#424. Longest Repeating Character Replacement [Medium] [G] [MS]

Problem

Given string *s* and integer *k*, return the length of the longest substring containing the same letter after at most *k* replacements.

```
1 class Solution {
2 public:
3     int characterReplacement(string s, int k) {
4         int cnt[26] = {}, maxCnt = 0, l = 0, res = 0;
5         for (int r = 0; r < (int)s.size(); r++) {
6             maxCnt = max(maxCnt, ++cnt[s[r] - 'A']);
7             while (r - l + 1 - maxCnt > k) cnt[s[l++] - 'A']--;
8             res = max(res, r - l + 1);
9         }
10        return res;
11    }
12};
```

3.3 LC#567. Permutation in String [Medium] [G] [MS] [A]

Problem

Return true if *s1*'s permutation is a substring of *s2*.


```

1 class Solution {
2 public:
3     bool checkInclusion(string s1, string s2) {
4         if (s1.size() > s2.size()) return false;
5         int a[26] = {}, b[26] = {};
6         for (char c : s1) a[c - 'a']++;
7         for (int i = 0; i < (int)s1.size(); i++) b[s2[i] - 'a']++;
8         if (equal(a, a + 26, b)) return true;
9         for (int i = s1.size(); i < (int)s2.size(); i++) {
10             b[s2[i] - 'a']++;
11             b[s2[i - s1.size()] - 'a']--;
12             if (equal(a, a + 26, b)) return true;
13         }
14         return false;
15     }
16 };

```

3.4 LC#76. Minimum Window Substring [Hard] [G] [A] [MS] [Meta]

Problem

Find the minimum window in string *s* that contains all characters of string *t*.

```

1 class Solution {
2 public:
3     string minWindow(string s, string t) {
4         unordered_map<char, int> need, have;
5         for (char c : t) need[c]++;
6         int formed = 0, total = need.size();
7         int l = 0, minLen = INT_MAX, start = 0;
8         for (int r = 0; r < (int)s.size(); r++) {
9             char c = s[r];
10            have[c]++;
11            if (need.count(c) && have[c] == need[c]) formed++;
12            while (formed == total) {
13                if (r - l + 1 < minLen) {
14                    minLen = r - l + 1;
15                    start = l;
16                }
17                have[s[l]]--;
18                if (need.count(s[l]) && have[s[l]] < need[s[l]]) formed--;
19                l++;
20            }
21        }
22        return minLen == INT_MAX ? "" : s.substr(start, minLen);
23    }
24 };

```

3.5 LC#5. Longest Palindromic Substring [Medium] [A] [MS] [G]

Problem

Return the longest palindromic substring of *s*.

```

1 class Solution {
2     string expand(const string& s, int l, int r) {
3         while (l >= 0 && r < (int)s.size() && s[l] == s[r]) {
4             l--;
5             r++;
6         }
7         return s.substr(l + 1, r - l - 1);
8     }
9 public:
10    string longestPalindrome(string s) {
11        string res;
12        for (int i = 0; i < (int)s.size(); i++) {
13            string odd = expand(s, i, i);
14            string even = expand(s, i, i + 1);
15            if (odd.size() > res.size()) res = odd;
16            if (even.size() > res.size()) res = even;
17        }
18        return res;
19    }
20 };

```

3.6 LC#49. Group Anagrams [\[Medium\]](#) [\[G\]](#) [\[A\]](#) [\[MS\]](#) [\[Meta\]](#)

Problem

Group an array of strings into anagram groups.

```

1 class Solution {
2 public:
3     vector<vector<string>> groupAnagrams(vector<string>& strs) {
4         unordered_map<string, vector<string>> mp;
5         for (auto& s : strs) {
6             string key = s;
7             sort(key.begin(), key.end());
8             mp[key].push_back(s);
9         }
10        vector<vector<string>> res;
11        for (auto& [k, v] : mp) res.push_back(v);
12        return res;
13    }
14 };

```

3.7 LC#242. Valid Anagram [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Given two strings *s* and *t*, return true if *t* is an anagram of *s*.

```

1 class Solution {
2 public:
3     bool isAnagram(string s, string t) {
4         if (s.size() != t.size()) return false;
5         int cnt[26] = {};
6         for (char c : s) cnt[c - 'a']++;
7         for (char c : t) if (--cnt[c - 'a'] < 0) return false;
8         return true;
9     }
10 };

```

3.8 LC#271. Encode and Decode Strings [\[Medium\]](#) [\[G\]](#)

Problem

Design an algorithm to encode a list of strings to a single string, and decode it back.

```

1 class Codec {
2 public:
3     string encode(vector<string>& strs) {
4         string res;
5         for (auto& s : strs) res += to_string(s.size()) + "#" + s;
6         return res;
7     }
8
9     vector<string> decode(string s) {
10        vector<string> res;
11        int i = 0;
12        while (i < (int)s.size()) {
13            int j = s.find('#', i);
14            int len = stoi(s.substr(i, j - i));
15            res.push_back(s.substr(j + 1, len));
16            i = j + 1 + len;
17        }
18        return res;
19    }
20 };

```

3.9 LC#239. Sliding Window Maximum [\[Hard\]](#) [\[G\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Return the max values in every sliding window of size *k*.

```

1  class Solution {
2  public:
3      vector<int> maxSlidingWindow(vector<int>& nums, int k) {
4          deque<int> dq;
5          vector<int> res;
6          for (int i = 0; i < (int)nums.size(); i++) {
7              while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
8              while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
9              dq.push_back(i);
10             if (i >= k - 1) res.push_back(nums[dq.front()]);
11         }
12         return res;
13     }
14 };

```

3.10 LC#438. Find All Anagrams in a String [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Given strings *s* and *p*, return all starting indices of *p*'s anagrams in *s*.

```

1  class Solution {
2  public:
3      vector<int> findAnagrams(string s, string p) {
4          vector<int> res;
5          if (s.size() < p.size()) return res;
6          int a[26] = {}, b[26] = {};
7          for (char c : p) a[c - 'a']++;
8          for (int i = 0; i < (int)p.size(); i++) b[s[i] - 'a']++;
9          if (equal(a, a + 26, b)) res.push_back(0);
10         for (int i = p.size(); i < (int)s.size(); i++) {
11             b[s[i] - 'a']++;
12             b[s[i - p.size()] - 'a']--;
13             if (equal(a, a + 26, b)) res.push_back(i - p.size() + 1);
14         }
15         return res;
16     }
17 };

```

Chapter 4

Linked Lists

4.1 LC#206. Reverse Linked List [Medium] [A] [MS] [G]

Problem

Reverse a singly linked list iteratively.

```
1 class Solution {
2 public:
3     ListNode* reverseList(ListNode* head) {
4         ListNode* prev = nullptr;
5         ListNode* cur = head;
6         while (cur != nullptr) {
7             ListNode* nxt = cur->next;
8             cur->next = prev;
9             prev = cur;
10            cur = nxt;
11        }
12        return prev;
13    }
14};
```

4.2 LC#19. Remove Nth Node From End of List [Medium] [A] [MS] [G]

Problem

Remove the n -th node from the end of a linked list in one pass.

```
1 class Solution {
2 public:
3     ListNode* removeNthFromEnd(ListNode* head, int n) {
4         ListNode dummy(0, head);
5         ListNode* fast = &dummy;
6         ListNode* slow = &dummy;
7
8         // advance fast by n+1 steps
9         for (int i = 0; i <= n; i++) fast = fast->next;
10
11        while (fast != nullptr) {
12            slow = slow->next;
13            fast = fast->next;
14        }
15
16        slow->next = slow->next->next;
17        return dummy.next;
18    }
19};
```

4.3 LC#92. Reverse Linked List II [Medium] [A] [MS] [G]

Problem

Reverse the nodes from position `left` to `right` in a linked list in one pass.

```

1  class Solution {
2  public:
3      ListNode* reverseBetween(ListNode* head, int left, int right) {
4          ListNode dummy(0, head);
5          ListNode* pre = &dummy;
6          for (int i = 1; i < left; i++) pre = pre->next;
7
8          ListNode* cur = pre->next;
9          for (int i = 0; i < right - left; i++) {
10             ListNode* nxt = cur->next;
11             cur->next = nxt->next;
12             nxt->next = pre->next;
13             pre->next = nxt;
14         }
15         return dummy.next;
16     }
17 };

```

4.4 LC#142. Linked List Cycle II [Medium] [A] [MS]

Problem

Given a linked list, return the node where the cycle begins, or null if no cycle.

```

1  class Solution {
2  public:
3      ListNode* detectCycle(ListNode* head) {
4          ListNode* slow = head;
5          ListNode* fast = head;
6          while (fast && fast->next) {
7              slow = slow->next;
8              fast = fast->next->next;
9              if (slow == fast) {
10                 fast = head;
11                 while (slow != fast) {
12                     slow = slow->next;
13                     fast = fast->next;
14                 }
15                 return slow;
16             }
17         }
18         return nullptr;
19     }
20 };

```

4.5 LC#143. Reorder List [Medium] [A] [MS] [G]

Problem

Reorder $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ in-place.

```

1  class Solution {
2  public:
3      void reorderList(ListNode* head) {
4          // find middle
5          ListNode* slow = head;
6          ListNode* fast = head->next;
7          while (fast != nullptr && fast->next != nullptr) {
8              slow = slow->next;
9              fast = fast->next->next;
10         }
11
12         // reverse second half
13         ListNode* prev = nullptr;
14         ListNode* cur = slow->next;
15         slow->next = nullptr;
16         while (cur != nullptr) {
17             ListNode* tmp = cur->next;
18             cur->next = prev;
19             prev = cur;
20             cur = tmp;
21         }

```

```

22
23 // merge two halves
24 ListNode* first = head;
25 ListNode* second = prev;
26 while (second != nullptr) {
27     ListNode* tmp1 = first->next;
28     ListNode* tmp2 = second->next;
29     first->next = second;
30     second->next = tmp1;
31     first = tmp1;
32     second = tmp2;
33 }
34 }
35 };

```

4.6 LC#138. Copy List with Random Pointer [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[Meta\]](#)

Problem

Deep copy a linked list where each node has a `next` and a `random` pointer.

```

1 class Solution {
2 public:
3     Node* copyRandomList(Node* head) {
4         unordered_map<Node*, Node*> mp;
5         Node* cur = head;
6         while (cur) {
7             mp[cur] = new Node(cur->val);
8             cur = cur->next;
9         }
10        cur = head;
11        while (cur) {
12            mp[cur]->next = mp[cur->next];
13            mp[cur]->random = mp[cur->random];
14            cur = cur->next;
15        }
16        return mp[head];
17    }
18 };

```

4.7 LC#146. LRU Cache [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#) [\[Meta\]](#)

Problem

Design a data structure that follows Least Recently Used (LRU) cache eviction, with $O(1)$ get and put.

```

1 class LRUCache {
2     int cap;
3     list<pair<int, int>> lst;
4     unordered_map<int, list<pair<int, int>>::iterator> mp;
5
6     void touch(int key, int val) {
7         if (mp.count(key)) lst.erase(mp[key]);
8         lst.push_front({key, val});
9         mp[key] = lst.begin();
10    }
11 public:
12     LRUCache(int c) : cap(c) {}
13
14     int get(int k) {
15         if (!mp.count(k)) return -1;
16         int v = mp[k]->second;
17         touch(k, v);
18         return v;
19     }
20
21     void put(int k, int v) {
22         touch(k, v);
23         if ((int)lst.size() > cap) {
24             mp.erase(lst.back().first);
25             lst.pop_back();
26         }
27     }
28 };

```

4.8 LC#23. Merge K Sorted Lists [\[Hard\]](#) [\[G\]](#) [\[A\]](#) [\[MS\]](#) [\[Ap\]](#)

Problem

Merge k sorted linked lists into one sorted list.

```

1  class Solution {
2  public:
3      ListNode* mergeKLists(vector<ListNode*>& lists) {
4          using P = pair<int, ListNode*>;
5          priority_queue<P, vector<P>, greater<P>> pq;
6          for (auto l : lists) if (l) pq.push({l->val, l});
7
8          ListNode dummy;
9          ListNode* cur = &dummy;
10         while (!pq.empty()) {
11             auto [v, node] = pq.top();
12             pq.pop();
13             cur->next = node;
14             cur = cur->next;
15             if (node->next) pq.push({node->next->val, node->next});
16         }
17         return dummy.next;
18     }
19 };

```

4.9 LC#430. Flatten a Multilevel Doubly Linked List [\[Medium\]](#) [\[MS\]](#)

Problem

Flatten a doubly linked list that may have child lists hanging from some nodes.

```

1  class Solution {
2  public:
3      Node* flatten(Node* head) {
4          Node* cur = head;
5          while (cur) {
6              if (cur->child) {
7                  Node* child = flatten(cur->child);
8                  Node* nxt = cur->next;
9                  cur->next = child;
10                 child->prev = cur;
11                 cur->child = nullptr;
12                 while (cur->next) cur = cur->next;
13                 cur->next = nxt;
14                 if (nxt) nxt->prev = cur;
15             }
16             cur = cur->next;
17         }
18         return head;
19     }
20 };

```

Chapter 5

Trees & Binary Search Trees

5.1 LC#226. Invert Binary Tree [\[Medium\]](#) [\[A\]](#) [\[G\]](#) [\[MS\]](#)

Problem

Invert (mirror) a binary tree and return its root.

```
1 class Solution {
2 public:
3     TreeNode* invertTree(TreeNode* root) {
4         if (root == nullptr) return nullptr;
5         swap(root->left, root->right);
6         invertTree(root->left);
7         invertTree(root->right);
8         return root;
9     }
10 };
```

5.2 LC#102. Binary Tree Level Order Traversal [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return the level-order (BFS) traversal of a binary tree as a list of lists.

```
1 class Solution {
2 public:
3     vector<vector<int>> levelOrder(TreeNode* root) {
4         if (!root) return {};
5         queue<TreeNode*> q;
6         q.push(root);
7         vector<vector<int>> res;
8         while (!q.empty()) {
9             int sz = q.size();
10            vector<int> level;
11            while (sz-->0) {
12                auto node = q.front();
13                q.pop();
14                level.push_back(node->val);
15                if (node->left) q.push(node->left);
16                if (node->right) q.push(node->right);
17            }
18            res.push_back(level);
19        }
20        return res;
21    }
22 };
```

5.3 LC#103. Binary Tree Zigzag Level Order [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[Meta\]](#)

Problem

Return zigzag (alternating direction) level order traversal of a binary tree.


```

1  class Solution {
2  public:
3      vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
4          if (!root) return {};
5          queue<TreeNode*> q;
6          q.push(root);
7          vector<vector<int>> res;
8          bool leftToRight = true;
9          while (!q.empty()) {
10             int sz = q.size();
11             deque<int> level;
12             while (sz-->0) {
13                 auto node = q.front();
14                 q.pop();
15                 if (leftToRight) level.push_back(node->val);
16                 else level.push_front(node->val);
17                 if (node->left) q.push(node->left);
18                 if (node->right) q.push(node->right);
19             }
20             res.push_back({level.begin(), level.end()});
21             leftToRight = !leftToRight;
22         }
23         return res;
24     }
25 };

```

5.4 LC#105. Construct Binary Tree from Preorder and Inorder [Medium] [A] [MS] [G]

Problem

Build a binary tree from preorder and inorder traversal arrays.

```

1  class Solution {
2      unordered_map<int, int> idx;
3
4      TreeNode* build(vector<int>& pre, int pl, int pr, int il, int ir) {
5          if (pl > pr) return nullptr;
6          int rootVal = pre[pl];
7          int i = idx[rootVal];
8          int leftLen = i - il;
9          return new TreeNode(rootVal,
10                             build(pre, pl + 1, pl + leftLen, il, i - 1),
11                             build(pre, pl + leftLen + 1, pr, i + 1, ir));
12     }
13 public:
14     TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
15         for (int i = 0; i < (int)inorder.size(); i++) idx[inorder[i]] = i;
16         return build(preorder, 0, preorder.size() - 1, 0, inorder.size() - 1);
17     }
18 };

```

5.5 LC#235. Lowest Common Ancestor of BST [Medium] [A] [MS] [G] [Meta]

Problem

Find the LCA of two nodes in a Binary Search Tree.

```

1  class Solution {
2  public:
3      TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
4          while (root) {
5              if (p->val < root->val && q->val < root->val) root = root->left;
6              else if (p->val > root->val && q->val > root->val) root = root->right;
7              else return root;
8          }
9          return nullptr;
10     }
11 };

```

5.6 LC#572. Subtree of Another Tree [Medium] [A] [MS] [G]

Problem

Given roots of two trees `root` and `subRoot`, return true if `subRoot` is a subtree of `root`.

```

1  class Solution {
2      bool isSame(TreeNode* s, TreeNode* t) {
3          if (!s && !t) return true;
4          if (!s || !t) return false;
5          return s->val == t->val
6              && isSame(s->left, t->left)
7              && isSame(s->right, t->right);
8      }
9  public:
10     bool isSubtree(TreeNode* root, TreeNode* subRoot) {
11         if (!root) return false;
12         if (isSame(root, subRoot)) return true;
13         return isSubtree(root->left, subRoot)
14             || isSubtree(root->right, subRoot);
15     }
16 };

```

5.7 LC#98. Validate Binary Search Tree [Medium] [A] [MS] [G]

Problem

Determine if a binary tree is a valid BST.

```

1  class Solution {
2      bool check(TreeNode* node, long lo, long hi) {
3          if (!node) return true;
4          if (node->val <= lo || node->val >= hi) return false;
5          return check(node->left, lo, node->val)
6              && check(node->right, node->val, hi);
7      }
8  public:
9      bool isValidBST(TreeNode* root) {
10         return check(root, LONG_MIN, LONG_MAX);
11     }
12 };

```

5.8 LC#230. Kth Smallest Element in a BST [Medium] [A] [MS]

Problem

Return the kth smallest element in a BST.

```

1  class Solution {
2      int k, res;
3      void inorder(TreeNode* node) {
4          if (!node) return;
5          inorder(node->left);
6          if (--k == 0) {
7              res = node->val;
8              return;
9          }
10         inorder(node->right);
11     }
12  public:
13     int kthSmallest(TreeNode* root, int k_) {
14         k = k_;
15         inorder(root);
16         return res;
17     }
18 };

```

5.9 LC#297. Serialize and Deserialize Binary Tree [Hard] [A] [G] [MS] [Meta]

Problem

Design a codec to serialize a binary tree to string and deserialize back.

```

1  class Codec {
2      TreeNode* build(queue<string>& q) {
3          string s = q.front();
4          q.pop();
5          if (s == "#") return nullptr;
6          TreeNode* node = new TreeNode(stoi(s));
7          node->left = build(q);
8          node->right = build(q);
9          return node;
10     }
11 public:
12     string serialize(TreeNode* root) {
13         if (!root) return "#";
14         return to_string(root->val) + ","
15             + serialize(root->left) + ","
16             + serialize(root->right);
17     }
18
19     TreeNode* deserialize(string data) {
20         queue<string> q;
21         stringstream ss(data);
22         string token;
23         while (getline(ss, token, ',')) q.push(token);
24         return build(q);
25     }
26 };

```

5.10 LC#437. Path Sum III [Medium] [G] [A]

Problem

Count the number of paths in a binary tree that sum to `targetSum` (path goes downward, any node to any node).

```

1  class Solution {
2      unordered_map<long, int> pre;
3      int res = 0;
4      long target;
5
6      void dfs(TreeNode* node, long cur) {
7          if (!node) return;
8          cur += node->val;
9          res += pre[cur - target];
10         pre[cur]++;
11         dfs(node->left, cur);
12         dfs(node->right, cur);
13         pre[cur]--;
14     }
15 public:
16     int pathSum(TreeNode* root, int t) {
17         target = t;
18         pre[0] = 1;
19         dfs(root, 0);
20         return res;
21     }
22 };

```

5.11 LC#114. Flatten Binary Tree to Linked List [Medium] [A] [MS]

Problem

Flatten a binary tree to a linked list in-place (preorder order), using the right pointers.

```

1  class Solution {
2 public:
3      void flatten(TreeNode* root) {
4          TreeNode* cur = root;
5          while (cur) {
6              if (cur->left) {
7                  TreeNode* pre = cur->left;
8                  while (pre->right) pre = pre->right;
9                  pre->right = cur->right;
10                 cur->right = cur->left;
11                 cur->left = nullptr;
12             }
13             cur = cur->right;
14         }
15     }

```

```
16 };
```

5.12 LC#199. Binary Tree Right Side View [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return the values visible from the right side of a binary tree, top to bottom.

```
1 class Solution {
2 public:
3     vector<int> rightSideView(TreeNode* root) {
4         if (!root) return {};
5         queue<TreeNode*> q;
6         q.push(root);
7         vector<int> res;
8         while (!q.empty()) {
9             int sz = q.size();
10            TreeNode* last = nullptr;
11            while (sz-->0) {
12                last = q.front();
13                q.pop();
14                if (last->left) q.push(last->left);
15                if (last->right) q.push(last->right);
16            }
17            res.push_back(last->val);
18        }
19        return res;
20    }
21 };
```

5.13 LC#543. Diameter of Binary Tree [\[Medium\]](#) [\[A\]](#) [\[G\]](#) [\[Ap\]](#)

Problem

Find the length of the longest path between any two nodes in a binary tree.

```
1 class Solution {
2     int res = 0;
3
4     int depth(TreeNode* node) {
5         if (!node) return 0;
6         int l = depth(node->left);
7         int r = depth(node->right);
8         res = max(res, l + r);
9         return max(l, r) + 1;
10    }
11 public:
12     int diameterOfBinaryTree(TreeNode* root) {
13         depth(root);
14         return res;
15    }
16 };
```


Chapter 6

Graphs

6.1 LC#200. Number of Islands [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#) [\[Meta\]](#)

Problem

Given a 2D binary grid, return the number of islands (surrounded by water, connected land).

```
1 class Solution {
2     void dfs(vector<vector<char>>& g, int i, int j) {
3         if (i < 0 || i >= (int)g.size()) return;
4         if (j < 0 || j >= (int)g[0].size()) return;
5         if (g[i][j] != '1') return;
6         g[i][j] = '0';
7         dfs(g, i+1, j);
8         dfs(g, i-1, j);
9         dfs(g, i, j+1);
10        dfs(g, i, j-1);
11    }
12 public:
13     int numIslands(vector<vector<char>>& g) {
14         int cnt = 0;
15         for (int i = 0; i < (int)g.size(); i++) {
16             for (int j = 0; j < (int)g[0].size(); j++) {
17                 if (g[i][j] == '1') {
18                     dfs(g, i, j);
19                     cnt++;
20                 }
21             }
22         }
23         return cnt;
24     }
25 };
```

6.2 LC#695. Max Area of Island [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return the maximum area of an island in a binary grid. An island is a connected group of 1s.

```
1 class Solution {
2     int dfs(vector<vector<int>>& g, int i, int j) {
3         if (i < 0 || i >= (int)g.size()) return 0;
4         if (j < 0 || j >= (int)g[0].size()) return 0;
5         if (g[i][j] != 1) return 0;
6         g[i][j] = 0;
7         return 1
8             + dfs(g, i+1, j)
9             + dfs(g, i-1, j)
10            + dfs(g, i, j+1)
11            + dfs(g, i, j-1);
12    }
13 public:
14     int maxAreaOfIsland(vector<vector<int>>& g) {
15         int res = 0;
16         for (int i = 0; i < (int)g.size(); i++) {
17             for (int j = 0; j < (int)g[0].size(); j++) {
18                 if (g[i][j] == 1) res = max(res, dfs(g, i, j));
19             }
20         }
21     }
```

```

21     return res;
22 }
23 };

```

6.3 LC#133. Clone Graph [Medium] [A] [MS] [G]

Problem

Clone (deep copy) an undirected graph.

```

1  class Solution {
2      unordered_map<Node*, Node*> mp;
3  public:
4      Node* cloneGraph(Node* node) {
5          if (!node) return nullptr;
6          if (mp.count(node)) return mp[node];
7          mp[node] = new Node(node->val);
8          for (auto nb : node->neighbors) {
9              mp[node]->neighbors.push_back(cloneGraph(nb));
10         }
11         return mp[node];
12     }
13 };

```

6.4 LC#207. Course Schedule (Cycle Detection) [Medium] [A] [MS] [G]

Problem

Given numCourses and prerequisite pairs, determine if you can finish all courses (no cycle in DAG).

```

1  class Solution {
2  public:
3      bool canFinish(int n, vector<vector<int>>& pre) {
4          vector<vector<int>> adj(n);
5          vector<int> indegree(n, 0);
6          for (auto& p : pre) {
7              adj[p[1]].push_back(p[0]);
8              indegree[p[0]]++;
9          }
10         queue<int> q;
11         for (int i = 0; i < n; i++) if (!indegree[i]) q.push(i);
12         int cnt = 0;
13         while (!q.empty()) {
14             int u = q.front();
15             q.pop();
16             cnt++;
17             for (int v : adj[u]) {
18                 if (--indegree[v] == 0) q.push(v);
19             }
20         }
21         return cnt == n;
22     }
23 };

```

6.5 LC#210. Course Schedule II [Medium] [A] [MS] [G]

Problem

Return a valid course ordering (topological sort), or empty if impossible.

```

1  class Solution {
2  public:
3      vector<int> findOrder(int n, vector<vector<int>>& pre) {
4          vector<vector<int>> adj(n);
5          vector<int> indegree(n, 0);
6          for (auto& p : pre) {
7              adj[p[1]].push_back(p[0]);
8              indegree[p[0]]++;
9          }
10         queue<int> q;
11         for (int i = 0; i < n; i++) if (!indegree[i]) q.push(i);

```

```

12     vector<int> res;
13     while (!q.empty()) {
14         int u = q.front();
15         q.pop();
16         res.push_back(u);
17         for (int v : adj[u]) {
18             if (--indegree[v] == 0) q.push(v);
19         }
20     }
21     return res.size() == n ? res : vector<int>{};
22 }
23 };

```

6.6 LC#417. Pacific Atlantic Water Flow [Medium] [A] [G]

Problem

Find all cells in an $m \times n$ grid where water can flow to both Pacific and Atlantic oceans.

```

1  class Solution {
2      int m, n;
3      int dx[4] = {0, 0, 1, -1};
4      int dy[4] = {1, -1, 0, 0};
5
6      void dfs(vector<vector<int>>& h, vector<vector<bool>>& vis, int i, int j) {
7          vis[i][j] = true;
8          for (int d = 0; d < 4; d++) {
9              int ni = i + dx[d], nj = j + dy[d];
10             if (ni >= 0 && ni < m && nj >= 0 && nj < n
11                 && !vis[ni][nj] && h[ni][nj] >= h[i][j]) {
12                 dfs(h, vis, ni, nj);
13             }
14         }
15     }
16 public:
17     vector<vector<int>> pacificAtlantic(vector<vector<int>>& h) {
18         m = h.size();
19         n = h[0].size();
20         vector<vector<bool>> pac(m, vector<bool>(n));
21         vector<vector<bool>> atl(m, vector<bool>(n));
22         for (int i = 0; i < m; i++) {
23             dfs(h, pac, i, 0);
24             dfs(h, atl, i, n-1);
25         }
26         for (int j = 0; j < n; j++) {
27             dfs(h, pac, 0, j);
28             dfs(h, atl, m-1, j);
29         }
30         vector<vector<int>> res;
31         for (int i = 0; i < m; i++) {
32             for (int j = 0; j < n; j++) {
33                 if (pac[i][j] && atl[i][j]) res.push_back({i, j});
34             }
35         }
36         return res;
37     }
38 };

```

6.7 LC#994. Rotting Oranges [Medium] [A] [MS] [G]

Problem

Multi-source BFS: every minute, rotten oranges infect adjacent fresh ones. Return minutes until all rotten, or -1.

```

1  class Solution {
2  public:
3      int orangesRotting(vector<vector<int>>& g) {
4          int m = g.size(), n = g[0].size(), fresh = 0, mins = 0;
5          queue<pair<int,int>> q;
6          for (int i = 0; i < m; i++) {
7              for (int j = 0; j < n; j++) {
8                  if (g[i][j] == 2) q.push({i, j});
9                  if (g[i][j] == 1) fresh++;
10             }
11         }
12         int dx[] = {0, 0, 1, -1};
13         int dy[] = {1, -1, 0, 0};
14         while (!q.empty() && fresh) {

```



```

15         int sz = q.size();
16         mins++;
17         while (sz-->0) {
18             auto [i, j] = q.front();
19             q.pop();
20             for (int d = 0; d < 4; d++) {
21                 int ni = i + dx[d], nj = j + dy[d];
22                 if (ni >= 0 && ni < m && nj >= 0 && nj < n && g[ni][nj] == 1) {
23                     g[ni][nj] = 2;
24                     fresh--;
25                     q.push({ni, nj});
26                 }
27             }
28         }
29     }
30     return fresh ? -1 : mins;
31 }
32 };

```

6.8 LC#1091. Shortest Path in Binary Matrix [Medium] [A] [G] [MS]

Problem

Find the shortest clear path from top-left to bottom-right in an $n \times n$ binary matrix (8-directional BFS).

```

1 class Solution {
2 public:
3     int shortestPathBinaryMatrix(vector<vector<int>>& g) {
4         int n = g.size();
5         if (g[0][0] == 1 || g[n-1][n-1] == 1) return -1;
6         queue<pair<int,int>> q;
7         q.push({0, 0});
8         g[0][0] = 1;
9         int dist = 1;
10        int dx[] = {-1,-1,-1, 0, 0, 1, 1, 1};
11        int dy[] = {-1, 0, 1,-1, 1,-1, 0, 1};
12        while (!q.empty()) {
13            int sz = q.size();
14            while (sz-->0) {
15                auto [i, j] = q.front();
16                q.pop();
17                if (i == n-1 && j == n-1) return dist;
18                for (int d = 0; d < 8; d++) {
19                    int ni = i + dx[d], nj = j + dy[d];
20                    if (ni >= 0 && ni < n && nj >= 0 && nj < n && g[ni][nj] == 0) {
21                        g[ni][nj] = 1;
22                        q.push({ni, nj});
23                    }
24                }
25            }
26            dist++;
27        }
28        return -1;
29    }
30 };

```

6.9 LC#269. Alien Dictionary [Hard] [G] [Meta]

Problem

Given a sorted alien dictionary's word list, derive the order of characters. Return empty string if invalid.

```

1 class Solution {
2 public:
3     string alienOrder(vector<string>& words) {
4         unordered_map<char, unordered_set<char>> adj;
5         unordered_map<char, int> indegree;
6         for (auto& w : words)
7             for (char c : w)
8                 if (!indegree.count(c)) indegree[c] = 0;
9
10        for (int i = 0; i < (int)words.size() - 1; i++) {
11            string& a = words[i], &b = words[i+1];
12            if (a.size() > b.size() && a.substr(0, b.size()) == b) return "";
13            for (int j = 0; j < (int)min(a.size(), b.size()); j++) {
14                if (a[j] != b[j]) {
15                    if (!adj[a[j]].count(b[j])) {
16                        adj[a[j]].insert(b[j]);
17                        indegree[b[j]]++;
18                    }
19                }
20            }
21        }
22
23        // Topological sort
24        string res;
25        while (indegree.count(res.back()) > 0) res.pop_back();
26        while (res.size() < indegree.size()) {
27            for (char c : indegree) {
28                if (indegree[c] == 0) {
29                    res.push_back(c);
30                    indegree[c] = 0;
31                }
32            }
33        }
34        return res;
35    }
36 };

```

```

18         }
19         break;
20     }
21 }
22 }
23 queue<char> q;
24 string res;
25 for (auto& [c, i] : indegree) if (i == 0) q.push(c);
26 while (!q.empty()) {
27     char u = q.front();
28     q.pop();
29     res += u;
30     for (char v : adj[u]) if (--indegree[v] == 0) q.push(v);
31 }
32 return res.size() == indegree.size() ? res : "";
33 }
34 };

```

6.10 LC#743. Network Delay Time [Medium] [A] [G]

Problem

Given a directed, weighted graph, find the time for all nodes to receive a signal from node k (Dijkstra).

```

1 class Solution {
2 public:
3     int networkDelayTime(vector<vector<int>>& times, int n, int k) {
4         vector<vector<pair<int,int>>> adj(n + 1);
5         for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});
6
7         vector<int> dist(n + 1, INT_MAX);
8         dist[k] = 0;
9         priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
10        pq.push({0, k});
11
12        while (!pq.empty()) {
13            auto [d, u] = pq.top();
14            pq.pop();
15            if (d > dist[u]) continue;
16            for (auto [v, w] : adj[u]) {
17                if (dist[u] + w < dist[v]) {
18                    dist[v] = dist[u] + w;
19                    pq.push({dist[v], v});
20                }
21            }
22        }
23        int ans = *max_element(dist.begin() + 1, dist.end());
24        return ans == INT_MAX ? -1 : ans;
25    }
26 };

```

6.11 LC#787. Cheapest Flights Within K Stops [Medium] [A] [G]

Problem

Find cheapest price from `src` to `dst` with at most k stops (Bellman-Ford $k+1$ rounds).

```

1 class Solution {
2 public:
3     int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
4         vector<int> price(n, INT_MAX);
5         price[src] = 0;
6         for (int i = 0; i <= k; i++) {
7             vector<int> tmp = price;
8             for (auto& f : flights) {
9                 if (price[f[0]] != INT_MAX && price[f[0]] + f[2] < tmp[f[1]]) {
10                     tmp[f[1]] = price[f[0]] + f[2];
11                 }
12             }
13             price = tmp;
14         }
15         return price[dst] == INT_MAX ? -1 : price[dst];
16    }
17 };

```

6.12 LC#323. Number of Connected Components [Medium] [G] [MS] [A]

Problem

Count connected components in an undirected graph with n nodes (Union-Find).

```

1  class Solution {
2      vector<int> par, rnk;
3
4      int find(int x) {
5          return par[x] == x ? x : par[x] = find(par[x]);
6      }
7
8      bool unite(int a, int b) {
9          a = find(a);
10         b = find(b);
11         if (a == b) return false;
12         if (rnk[a] < rnk[b]) swap(a, b);
13         par[b] = a;
14         if (rnk[a] == rnk[b]) rnk[a]++;
15         return true;
16     }
17 public:
18     int countComponents(int n, vector<vector<int>>& edges) {
19         par.resize(n);
20         rnk.resize(n, 0);
21         iota(par.begin(), par.end(), 0);
22         int cnt = n;
23         for (auto& e : edges) if (unite(e[0], e[1])) cnt--;
24         return cnt;
25     }
26 };

```

6.13 LC#684. Redundant Connection [Medium] [A] [MS]

Problem

Find the edge that, if removed, makes the tree of n nodes have no cycle (Union-Find).

```

1  class Solution {
2      vector<int> par;
3
4      int find(int x) {
5          return par[x] == x ? x : par[x] = find(par[x]);
6      }
7  public:
8      vector<int> findRedundantConnection(vector<vector<int>>& edges) {
9          int n = edges.size();
10         par.resize(n + 1);
11         iota(par.begin(), par.end(), 0);
12         for (auto& e : edges) {
13             int a = find(e[0]), b = find(e[1]);
14             if (a == b) return e;
15             par[a] = b;
16         }
17         return {};
18     }
19 };

```

Chapter 7

Dynamic Programming

7.1 LC#198. House Robber [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Rob houses along a street — cannot rob two adjacent houses. Return the maximum amount you can rob.

```
1 class Solution {
2 public:
3     int rob(vector<int>& nums) {
4         int prev2 = 0, prev1 = 0;
5         for (int x : nums) {
6             int cur = max(prev1, prev2 + x);
7             prev2 = prev1;
8             prev1 = cur;
9         }
10        return prev1;
11    }
12};
```

7.2 LC#213. House Robber II [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Houses are arranged in a circle. Cannot rob adjacent houses. Return maximum amount you can rob.

```
1 class Solution {
2     int robRange(vector<int>& nums, int lo, int hi) {
3         int prev2 = 0, prev1 = 0;
4         for (int i = lo; i <= hi; i++) {
5             int cur = max(prev1, prev2 + nums[i]);
6             prev2 = prev1;
7             prev1 = cur;
8         }
9         return prev1;
10    }
11 public:
12     int rob(vector<int>& nums) {
13         int n = nums.size();
14         if (n == 1) return nums[0];
15         return max(robRange(nums, 0, n-2), robRange(nums, 1, n-1));
16    }
17};
```

7.3 LC#300. Longest Increasing Subsequence [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Find the length of the longest strictly increasing subsequence ($O(n \log n)$).

```
1 class Solution {
2 public:
3     int lengthOfLIS(vector<int>& nums) {
```

```

4     vector<int> tails;
5     for (int x : nums) {
6         auto it = lower_bound(tails.begin(), tails.end(), x);
7         if (it == tails.end()) tails.push_back(x);
8         else *it = x;
9     }
10    return tails.size();
11 }
12 };

```

7.4 LC#72. Edit Distance [Hard] [A] [G] [MS]

Problem

Find minimum number of operations (insert, delete, replace) to convert word1 to word2.

```

1 class Solution {
2 public:
3     int minDistance(string a, string b) {
4         int m = a.size(), n = b.size();
5         vector<vector<int>> dp(m + 1, vector<int>(n + 1));
6         for (int i = 0; i <= m; i++) dp[i][0] = i;
7         for (int j = 0; j <= n; j++) dp[0][j] = j;
8         for (int i = 1; i <= m; i++) {
9             for (int j = 1; j <= n; j++) {
10                if (a[i-1] == b[j-1]) {
11                    dp[i][j] = dp[i-1][j-1];
12                } else {
13                    dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
14                }
15            }
16        }
17        return dp[m][n];
18    }
19 };

```

7.5 LC#322. Coin Change [Medium] [A] [G] [MS]

Problem

Find the minimum number of coins to make up the given amount, or -1 if impossible.

```

1 class Solution {
2 public:
3     int coinChange(vector<int>& coins, int amount) {
4         vector<int> dp(amount + 1, INT_MAX);
5         dp[0] = 0;
6         for (int i = 1; i <= amount; i++) {
7             for (int c : coins) {
8                 if (c <= i && dp[i-c] != INT_MAX) {
9                     dp[i] = min(dp[i], dp[i-c] + 1);
10                }
11            }
12        }
13        return dp[amount] == INT_MAX ? -1 : dp[amount];
14    }
15 };

```

7.6 LC#518. Coin Change II [Medium] [A] [G]

Problem

Find the number of combinations of coins to make the given amount.

```

1 class Solution {
2 public:
3     int change(int amount, vector<int>& coins) {
4         vector<long long> dp(amount + 1, 0);
5         dp[0] = 1;
6         for (int c : coins) {
7             for (int i = c; i <= amount; i++) {
8                 dp[i] += dp[i - c];
9             }
10        }

```

```

10     }
11     return dp[amount];
12 }
13 };

```

7.7 LC#139. Word Break [Medium] [A] [G] [MS] [Meta]

Problem

Determine if string `s` can be segmented using words from `wordDict`.

```

1 class Solution {
2 public:
3     bool wordBreak(string s, vector<string>& wordDict) {
4         unordered_set<string> ws(wordDict.begin(), wordDict.end());
5         int n = s.size();
6         vector<bool> dp(n + 1, false);
7         dp[0] = true;
8         for (int i = 1; i <= n; i++) {
9             for (int j = 0; j < i; j++) {
10                 if (dp[j] && ws.count(s.substr(j, i - j))) {
11                     dp[i] = true;
12                     break;
13                 }
14             }
15         }
16         return dp[n];
17     }
18 };

```

7.8 LC#62. Unique Paths [Medium] [A] [G] [MS]

Problem

Count unique paths from top-left to bottom-right of an `m`En grid (only right/down moves).

```

1 class Solution {
2 public:
3     int uniquePaths(int m, int n) {
4         vector<int> dp(n, 1);
5         for (int i = 1; i < m; i++) {
6             for (int j = 1; j < n; j++) {
7                 dp[j] += dp[j-1];
8             }
9         }
10        return dp[n-1];
11    }
12 };

```

7.9 LC#1143. Longest Common Subsequence [Medium] [A] [G] [MS]

Problem

Find the length of the longest common subsequence of two strings.

```

1 class Solution {
2 public:
3     int longestCommonSubsequence(string a, string b) {
4         int m = a.size(), n = b.size();
5         vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
6         for (int i = 1; i <= m; i++) {
7             for (int j = 1; j <= n; j++) {
8                 if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
9                 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
10            }
11        }
12        return dp[m][n];
13    }
14 };

```

7.10 LC#337. House Robber III [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Rob houses on a binary tree. Cannot rob two directly-linked houses. Return max you can rob.

```

1 class Solution {
2     pair<int,int> dfs(TreeNode* node) { // {rob_this, skip_this}
3         if (!node) return {0, 0};
4         auto [l1, l2] = dfs(node->left);
5         auto [r1, r2] = dfs(node->right);
6         int rob = node->val + l2 + r2;
7         int skip = max(l1, l2) + max(r1, r2);
8         return {rob, skip};
9     }
10 public:
11     int rob(TreeNode* root) {
12         auto [r, s] = dfs(root);
13         return max(r, s);
14     }
15 };

```

7.11 LC#416. Partition Equal Subset Sum [\[Medium\]](#) [\[A\]](#) [\[G\]](#) [\[MS\]](#)

Problem

Determine if array can be partitioned into two subsets with equal sum (0/1 knapsack).

```

1 class Solution {
2 public:
3     bool canPartition(vector<int>& nums) {
4         int sum = accumulate(nums.begin(), nums.end(), 0);
5         if (sum % 2) return false;
6         int target = sum / 2;
7         vector<bool> dp(target + 1, false);
8         dp[0] = true;
9         for (int x : nums) {
10             for (int j = target; j >= x; j--) {
11                 dp[j] = dp[j] || dp[j - x];
12             }
13         }
14         return dp[target];
15     }
16 };

```

7.12 LC#91. Decode Ways [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Count the number of ways to decode a string of digits (1-26 mapped to A-Z).

```

1 class Solution {
2 public:
3     int numDecodings(string s) {
4         int n = s.size();
5         if (!n || s[0] == '0') return 0;
6         vector<int> dp(n + 1, 0);
7         dp[0] = 1;
8         dp[1] = 1;
9         for (int i = 2; i <= n; i++) {
10             if (s[i-1] != '0') dp[i] += dp[i-1];
11             int two = stoi(s.substr(i-2, 2));
12             if (two >= 10 && two <= 26) dp[i] += dp[i-2];
13         }
14         return dp[n];
15     }
16 };

```

7.13 LC#221. Maximal Square [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Find the largest square containing only 1s in a binary matrix; return its area.

```

1 class Solution {
2 public:
3     int maximalSquare(vector<vector<char>>& mat) {
4         int m = mat.size(), n = mat[0].size(), res = 0;
5         vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
6         for (int i = 1; i <= m; i++) {
7             for (int j = 1; j <= n; j++) {
8                 if (mat[i-1][j-1] == '1') {
9                     dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;
10                    res = max(res, dp[i][j]);
11                }
12            }
13        }
14        return res * res;
15    }
16 };

```

7.14 LC#55. Jump Game [Medium] [A] [G]

Problem

Given jump lengths, determine if you can reach the last index.

```

1 class Solution {
2 public:
3     bool canJump(vector<int>& nums) {
4         int far = 0;
5         for (int i = 0; i < (int)nums.size(); i++) {
6             if (i > far) return false;
7             far = max(far, i + nums[i]);
8         }
9         return true;
10    }
11 };

```

7.15 LC#188. Best Time to Buy and Sell Stock IV [Hard] [A] [G]

Problem

Find max profit with at most k transactions.

```

1 class Solution {
2 public:
3     int maxProfit(int k, vector<int>& prices) {
4         int n = prices.size();
5         if (k >= n / 2) {
6             int res = 0;
7             for (int i = 1; i < n; i++) {
8                 if (prices[i] > prices[i-1]) res += prices[i] - prices[i-1];
9             }
10            return res;
11        }
12        vector<int> buy(k, INT_MIN), sell(k, 0);
13        for (int p : prices) {
14            for (int i = 0; i < k; i++) {
15                buy[i] = max(buy[i], (i ? sell[i-1] : 0) - p);
16                sell[i] = max(sell[i], buy[i] + p);
17            }
18        }
19        return sell[k-1];
20    }
21 };

```


Chapter 8

Backtracking

8.1 LC#78. Subsets [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return all possible subsets (power set) of a distinct integer array.

```
1 class Solution {
2     void bt(vector<int>& nums, int i, vector<int>& cur, vector<vector<int>>& res) {
3         res.push_back(cur);
4         for (int j = i; j < (int)nums.size(); j++) {
5             cur.push_back(nums[j]);
6             bt(nums, j+1, cur, res);
7             cur.pop_back();
8         }
9     }
10 public:
11     vector<vector<int>> subsets(vector<int>& nums) {
12         vector<vector<int>> res;
13         vector<int> cur;
14         bt(nums, 0, cur, res);
15         return res;
16     }
17 };
```

8.2 LC#90. Subsets II [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Return all possible subsets of an array that may contain duplicates. No duplicate subsets.

```
1 class Solution {
2     void bt(vector<int>& nums, int i, vector<int>& cur, vector<vector<int>>& res) {
3         res.push_back(cur);
4         for (int j = i; j < (int)nums.size(); j++) {
5             if (j > i && nums[j] == nums[j-1]) continue;
6             cur.push_back(nums[j]);
7             bt(nums, j+1, cur, res);
8             cur.pop_back();
9         }
10    }
11 public:
12     vector<vector<int>> subsetsWithDup(vector<int>& nums) {
13         sort(nums.begin(), nums.end());
14         vector<vector<int>> res;
15         vector<int> cur;
16         bt(nums, 0, cur, res);
17         return res;
18    }
19 };
```

8.3 LC#46. Permutations [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return all permutations of a distinct integer array.

```

1  class Solution {
2  public:
3      vector<vector<int>> permute(vector<int>& nums) {
4          vector<vector<int>> res;
5          sort(nums.begin(), nums.end());
6          do {
7              res.push_back(nums);
8          } while (next_permutation(nums.begin(), nums.end()));
9          return res;
10     }
11 };

```

8.4 LC#77. Combinations [Medium] [G] [A] [MS]

Problem

Return all combinations of k numbers chosen from the range $[1, n]$.

```

1  class Solution {
2      void bt(int n, int k, int start, vector<int>& cur, vector<vector<int>>& res) {
3          if ((int)cur.size() == k) {
4              res.push_back(cur);
5              return;
6          }
7          for (int i = start; i <= n; i++) {
8              cur.push_back(i);
9              bt(n, k, i + 1, cur, res);
10             cur.pop_back();
11         }
12     }
13 public:
14     vector<vector<int>> combine(int n, int k) {
15         vector<vector<int>> res;
16         vector<int> cur;
17         bt(n, k, 1, cur, res);
18         return res;
19     }
20 };

```

8.5 LC#39. Combination Sum [Medium] [A] [G] [MS]

Problem

Find all combinations that sum to **target** (elements can be reused).

```

1  class Solution {
2      void bt(vector<int>& c, int i, int rem, vector<int>& cur, vector<vector<int>>& res) {
3          if (rem == 0) {
4              res.push_back(cur);
5              return;
6          }
7          for (int j = i; j < (int)c.size() && c[j] <= rem; j++) {
8              cur.push_back(c[j]);
9              bt(c, j, rem - c[j], cur, res);
10             cur.pop_back();
11         }
12     }
13 public:
14     vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
15         sort(candidates.begin(), candidates.end());
16         vector<vector<int>> res;
17         vector<int> cur;
18         bt(candidates, 0, target, cur, res);
19         return res;
20     }
21 };

```

8.6 LC#40. Combination Sum II [Medium] [A] [MS]

Problem

Combinations summing to target from candidates that may have duplicates. Each number used once.

```

1 class Solution {
2     void bt(vector<int>& c, int i, int rem, vector<int>& cur, vector<vector<int>>& res) {
3         if (rem == 0) {
4             res.push_back(cur);
5             return;
6         }
7         for (int j = i; j < (int)c.size() && c[j] <= rem; j++) {
8             if (j > i && c[j] == c[j-1]) continue;
9             cur.push_back(c[j]);
10            bt(c, j+1, rem - c[j], cur, res);
11            cur.pop_back();
12        }
13    }
14 public:
15     vector<vector<int>> combinationSum2(vector<int>& c, int target) {
16         sort(c.begin(), c.end());
17         vector<vector<int>> res;
18         vector<int> cur;
19         bt(c, 0, target, cur, res);
20         return res;
21     }
22 };

```

8.7 LC#51. N-Queens [Hard] [A] [G] [MS]

Problem

Place n queens on an $n \times n$ chessboard so that no two queens attack each other. Return all solutions.

```

1 class Solution {
2     vector<vector<string>> res;
3
4     void bt(int n, int row, vector<string>& board,
5             unordered_set<int>& cols,
6             unordered_set<int>& diag1,
7             unordered_set<int>& diag2) {
8         if (row == n) {
9             res.push_back(board);
10            return;
11        }
12        for (int c = 0; c < n; c++) {
13            if (cols.count(c) || diag1.count(row-c) || diag2.count(row+c)) continue;
14            board[row][c] = 'Q';
15            cols.insert(c);
16            diag1.insert(row-c);
17            diag2.insert(row+c);
18            bt(n, row+1, board, cols, diag1, diag2);
19            board[row][c] = '.';
20            cols.erase(c);
21            diag1.erase(row-c);
22            diag2.erase(row+c);
23        }
24    }
25 public:
26     vector<vector<string>> solveNQueens(int n) {
27         vector<string> board(n, string(n, '.'));
28         unordered_set<int> cols, d1, d2;
29         bt(n, 0, board, cols, d1, d2);
30         return res;
31     }
32 };

```

8.8 LC#79. Word Search [Medium] [A] [MS] [G]

Problem

Given an $m \times n$ grid of characters and a word, return true if the word exists in the grid (connected adjacent cells).

```

1 class Solution {
2     bool dfs(vector<vector<char>>& b, string& w, int i, int j, int k) {
3         if (k == (int)w.size()) return true;

```

```

4     if (i < 0 || i >= (int)b.size()) return false;
5     if (j < 0 || j >= (int)b[0].size()) return false;
6     if (b[i][j] != w[k]) return false;
7     char tmp = b[i][j];
8     b[i][j] = '#';
9     bool ok = dfs(b, w, i+1, j, k+1)
10             || dfs(b, w, i-1, j, k+1)
11             || dfs(b, w, i, j+1, k+1)
12             || dfs(b, w, i, j-1, k+1);
13     b[i][j] = tmp;
14     return ok;
15 }
16 public:
17     bool exist(vector<vector<char>>& board, string word) {
18         for (int i = 0; i < (int)board.size(); i++) {
19             for (int j = 0; j < (int)board[0].size(); j++) {
20                 if (dfs(board, word, i, j, 0)) return true;
21             }
22         }
23         return false;
24     }
25 };

```

8.9 LC#131. Palindrome Partitioning [\[Medium\]](#) [\[A\]](#) [\[G\]](#) [\[MS\]](#)

Problem

Return all possible palindrome partitioning of string *s*.

```

1 class Solution {
2     bool isPal(string& s, int l, int r) {
3         while (l < r) if (s[l++] != s[r--]) return false;
4         return true;
5     }
6
7     void bt(string& s, int i, vector<string>& cur, vector<vector<string>>& res) {
8         if (i == (int)s.size()) {
9             res.push_back(cur);
10            return;
11        }
12        for (int j = i; j < (int)s.size(); j++) {
13            if (isPal(s, i, j)) {
14                cur.push_back(s.substr(i, j - i + 1));
15                bt(s, j+1, cur, res);
16                cur.pop_back();
17            }
18        }
19    }
20 public:
21     vector<vector<string>> partition(string s) {
22         vector<vector<string>> res;
23         vector<string> cur;
24         bt(s, 0, cur, res);
25         return res;
26     }
27 };

```

8.10 LC#17. Letter Combinations of a Phone Number [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return all possible letter combinations a phone number's digits could represent.

```

1 class Solution {
2     vector<string> mp = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
3 public:
4     vector<string> letterCombinations(string digits) {
5         if (digits.empty()) return {};
6         vector<string> res;
7         function<void(int, string)> bt = [&](int i, string cur) {
8             if (i == (int)digits.size()) {
9                 res.push_back(cur);
10                return;
11            }
12            for (char c : mp[digits[i] - '0']) bt(i + 1, cur + c);
13        };
14        bt(0, "");

```

```
15     return res;  
16 }  
17 };
```


Chapter 9

Stacks, Queues & Heaps

9.1 LC#20. Valid Parentheses [Medium] [A] [G] [MS]

Problem

Determine if a string of brackets is valid (correctly opened and closed in order).

```
1 class Solution {
2 public:
3     bool isValid(string s) {
4         stack<char> st;
5         for (char c : s) {
6             if (c == '(' || c == '[' || c == '{') {
7                 st.push(c);
8             } else {
9                 if (st.empty()) return false;
10                if (c == ')' && st.top() != '(') return false;
11                if (c == ']' && st.top() != '[') return false;
12                if (c == '}' && st.top() != '{') return false;
13                st.pop();
14            }
15        }
16        return st.empty();
17    }
18};
```

9.2 LC#155. Min Stack [Medium] [A] [MS] [G]

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in O(1).

```
1 class MinStack {
2     stack<pair<int,int>> st; // {val, currentMin}
3 public:
4     void push(int val) {
5         int mn = st.empty() ? val : min(val, st.top().second);
6         st.push({val, mn});
7     }
8     void pop() { st.pop(); }
9     int top() { return st.top().first; }
10    int getMin() { return st.top().second; }
11};
```

9.3 LC#496. Next Greater Element I [Medium] [A] [MS] [G]

Problem

For each element of `nums1` find the next greater element in `nums2`. Use a monotonic stack.

```
1 class Solution {
2 public:
3     vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
```



```

4 unordered_map<int, int> nge;
5 stack<int> st;
6 for (int x : nums2) {
7     while (!st.empty() && st.top() < x) {
8         nge[st.top()] = x;
9         st.pop();
10    }
11    st.push(x);
12 }
13 vector<int> res;
14 for (int x : nums1) {
15     res.push_back(nge.count(x) ? nge[x] : -1);
16 }
17 return res;
18 }
19 };

```

9.4 LC#84. Largest Rectangle in Histogram [Hard] [A] [G] [MS]

Problem

Find the area of the largest rectangle in a histogram.

```

1 class Solution {
2 public:
3     int largestRectangleArea(vector<int>& h) {
4         stack<int> st;
5         int res = 0, n = h.size();
6         for (int i = 0; i <= n; i++) {
7             int cur = (i == n ? 0 : h[i]);
8             while (!st.empty() && h[st.top()] > cur) {
9                 int height = h[st.top()];
10                st.pop();
11                int width = st.empty() ? i : i - st.top() - 1;
12                res = max(res, height * width);
13            }
14            st.push(i);
15        }
16        return res;
17    }
18 };

```

9.5 LC#739. Daily Temperatures [Medium] [A] [MS] [G]

Problem

Given daily temperatures, return how many days until a warmer temperature for each day.

```

1 class Solution {
2 public:
3     vector<int> dailyTemperatures(vector<int>& t) {
4         int n = t.size();
5         vector<int> res(n, 0);
6         stack<int> st;
7         for (int i = 0; i < n; i++) {
8             while (!st.empty() && t[i] > t[st.top()]) {
9                 res[st.top()] = i - st.top();
10                st.pop();
11            }
12            st.push(i);
13        }
14        return res;
15    }
16 };

```

9.6 LC#853. Car Fleet [Medium] [A] [G]

Problem

Find the number of car fleets that arrive at the target. Cars at the same position form a fleet.

```

1 class Solution {
2 public:

```

```

3   int carFleet(int target, vector<int>& pos, vector<int>& speed) {
4       int n = pos.size();
5       vector<pair<int, double>> cars(n);
6       for (int i = 0; i < n; i++) {
7           cars[i] = {pos[i], (double)(target - pos[i]) / speed[i]};
8       }
9       sort(cars.rbegin(), cars.rend());
10      stack<double> st;
11      for (auto& [p, t] : cars) {
12          if (st.empty() || t > st.top()) st.push(t);
13      }
14      return st.size();
15  }
16 };

```

9.7 LC#215. Kth Largest Element in an Array [Medium] [A] [MS] [G] [Meta]

Problem

Find the kth largest element in an unsorted array using a min-heap of size k.

```

1   class Solution {
2   public:
3       int findKthLargest(vector<int>& nums, int k) {
4           priority_queue<int, vector<int>, greater<int>> pq;
5           for (int x : nums) {
6               pq.push(x);
7               if ((int)pq.size() > k) pq.pop();
8           }
9           return pq.top();
10      }
11 };

```

9.8 LC#295. Find Median from Data Stream [Hard] [A] [MS] [G]

Problem

Design a data structure that supports addNum and findMedian operations efficiently using two heaps.

```

1   class MedianFinder {
2       priority_queue<int> lo; // max-heap (lower half)
3       priority_queue<int, vector<int>, greater<int>> hi; // min-heap (upper half)
4   public:
5       void addNum(int num) {
6           lo.push(num);
7           hi.push(lo.top());
8           lo.pop();
9           if (hi.size() > lo.size()) {
10              lo.push(hi.top());
11              hi.pop();
12          }
13      }
14      double findMedian() {
15          return lo.size() > hi.size()
16              ? lo.top()
17              : (lo.top() + hi.top()) / 2.0;
18      }
19 };

```

9.9 LC#347. Top K Frequent Elements [Medium] [A] [MS] [G] [Meta]

Problem

Return the k most frequent elements from an integer array.

```

1   class Solution {
2   public:
3       vector<int> topKFrequent(vector<int>& nums, int k) {
4           unordered_map<int, int> freq;
5           for (int x : nums) freq[x]++;
6       }

```

```

7     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
8     for (auto& [x, f] : freq) {
9         pq.push({f, x});
10        if ((int)pq.size() > k) pq.pop();
11    }
12
13    vector<int> res;
14    while (!pq.empty()) {
15        res.push_back(pq.top().second);
16        pq.pop();
17    }
18    return res;
19 }
20 };

```

9.10 LC#355. Design Twitter [Medium] [A] [MS] [G]

Problem

Design a simplified Twitter with postTweet, getNewsFeed (10 recent), follow/unfollow operations.

```

1 class Twitter {
2     int ts = 0;
3     unordered_map<int, vector<pair<int,int>>> tweets; // user -> {time, tweetId}
4     unordered_map<int, unordered_set<int>> following;
5 public:
6     void postTweet(int u, int t) {
7         tweets[u].push_back({ts++, t});
8     }
9
10    vector<int> getNewsFeed(int u) {
11        priority_queue<tuple<int,int,int,int>> pq; // {time, tweetId, userId, idx}
12        auto add = [&](int uid) {
13            if (!tweets[uid].empty()) {
14                int i = tweets[uid].size() - 1;
15                auto& [t, id] = tweets[uid][i];
16                pq.push({t, id, uid, i});
17            }
18        };
19        add(u);
20        for (int f : following[u]) add(f);
21
22        vector<int> res;
23        while (!pq.empty() && (int)res.size() < 10) {
24            auto [t, id, uid, i] = pq.top();
25            pq.pop();
26            res.push_back(id);
27            if (i > 0) {
28                auto& [tt, iid] = tweets[uid][i-1];
29                pq.push({tt, iid, uid, i-1});
30            }
31        }
32        return res;
33    }
34
35    void follow(int f, int e) { following[f].insert(e); }
36    void unfollow(int f, int e) { following[f].erase(e); }
37 };

```

9.11 LC#621. Task Scheduler [Medium] [A] [MS] [G]

Problem

Given tasks and cooldown period n , return the minimum number of intervals to finish all tasks.

```

1 class Solution {
2 public:
3     int leastInterval(vector<char>& tasks, int n) {
4         int freq[26] = {};
5         for (char c : tasks) freq[c - 'A']++;
6         int maxF = *max_element(freq, freq + 26);
7         int maxCnt = count(freq, freq + 26, maxF);
8         return max((int)tasks.size(), (maxF - 1) * (n + 1) + maxCnt);
9     }
10 };

```

Chapter 10

Greedy, Intervals & Miscellaneous

10.1 LC#253. Meeting Rooms II [\[Medium\]](#) [\[G\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Given meeting intervals, find the minimum number of conference rooms required.

```
1 class Solution {
2 public:
3     int minMeetingRooms(vector<vector<int>>& intervals) {
4         vector<int> starts, ends;
5         for (auto& i : intervals) {
6             starts.push_back(i[0]);
7             ends.push_back(i[1]);
8         }
9         sort(starts.begin(), starts.end());
10        sort(ends.begin(), ends.end());
11        int rooms = 0, j = 0;
12        for (int s : starts) {
13            if (s < ends[j]) rooms++;
14            else j++;
15        }
16        return rooms;
17    }
18};
```

10.2 LC#435. Non-overlapping Intervals [\[Medium\]](#) [\[G\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Find minimum number of intervals to remove to make the rest non-overlapping.

```
1 class Solution {
2 public:
3     int eraseOverlapIntervals(vector<vector<int>>& intervals) {
4         sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
5             return a[1] < b[1];
6         });
7         int cnt = 0, end = INT_MIN;
8         for (auto& i : intervals) {
9             if (i[0] >= end) end = i[1];
10            else cnt++;
11        }
12        return cnt;
13    }
14};
```

10.3 LC#56. Merge Intervals [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Merge all overlapping intervals in a list.

```

1 class Solution {
2 public:
3     vector<vector<int>>> merge(vector<vector<int>>& intervals) {
4         sort(intervals.begin(), intervals.end());
5         vector<vector<int>>> res;
6         for (auto& i : intervals) {
7             if (res.empty() || res.back()[1] < i[0]) res.push_back(i);
8             else res.back()[1] = max(res.back()[1], i[1]);
9         }
10        return res;
11    }
12};

```

10.4 LC#57. Insert Interval [Medium] [A] [G] [MS]

Problem

Insert a new interval into a sorted non-overlapping list and merge if necessary.

```

1 class Solution {
2 public:
3     vector<vector<int>>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {
4         vector<vector<int>>> res;
5         int i = 0, n = intervals.size();
6
7         // add all intervals ending before newInterval starts
8         while (i < n && intervals[i][1] < newInterval[0]) res.push_back(intervals[i++]);
9
10        // merge overlapping intervals
11        while (i < n && intervals[i][0] <= newInterval[1]) {
12            newInterval[0] = min(newInterval[0], intervals[i][0]);
13            newInterval[1] = max(newInterval[1], intervals[i][1]);
14            i++;
15        }
16        res.push_back(newInterval);
17
18        // add remaining
19        while (i < n) res.push_back(intervals[i++]);
20        return res;
21    }
22};

```

10.5 LC#134. Gas Station [Medium] [A] [MS] [G]

Problem

Find the starting gas station index to complete the circuit, or -1 if impossible.

```

1 class Solution {
2 public:
3     int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
4         int total = 0, tank = 0, start = 0;
5         for (int i = 0; i < (int)gas.size(); i++) {
6             int diff = gas[i] - cost[i];
7             total += diff;
8             tank += diff;
9             if (tank < 0) {
10                start = i + 1;
11                tank = 0;
12            }
13        }
14        return total >= 0 ? start : -1;
15    }
16};

```

10.6 LC#406. Queue Reconstruction by Height [Medium] [A] [G] [MS]

Problem

Reconstruct a queue of people [h,k] where each person stands among k others taller or equal in front.

```

1 class Solution {

```

```

2 public:
3     vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
4         // sort: taller first; ties broken by k ascending
5         sort(people.begin(), people.end(), [](auto& a, auto& b) {
6             return a[0] != b[0] ? a[0] > b[0] : a[1] < b[1];
7         });
8         vector<vector<int>> res;
9         for (auto& p : people) {
10             res.insert(res.begin() + p[1], p);
11         }
12         return res;
13     }
14 };

```

10.7 LC#846. Hand of Straights [Medium] [G]

Problem

Determine if hand can be rearranged into groups of `groupSize` consecutive cards.

```

1 class Solution {
2 public:
3     bool isNStraightHand(vector<int>& hand, int gs) {
4         if (hand.size() % gs) return false;
5         map<int, int> cnt;
6         for (int x : hand) cnt[x]++;
7         for (auto& [k, v] : cnt) {
8             if (!v) continue;
9             for (int i = 0; i < gs; i++) {
10                 if (cnt[k + i] < v) return false;
11                 cnt[k + i] -= v;
12             }
13         }
14         return true;
15     }
16 };

```

10.8 LC#452. Minimum Number of Arrows to Burst Balloons [Medium] [A] [G]

Problem

Find minimum number of vertical arrows to burst all balloons (intervals on x-axis).

```

1 class Solution {
2 public:
3     int findMinArrowShots(vector<vector<int>>& points) {
4         sort(points.begin(), points.end(), [](auto& a, auto& b) {
5             return a[1] < b[1];
6         });
7         int arrows = 1, end = points[0][1];
8         for (int i = 1; i < (int)points.size(); i++) {
9             if (points[i][0] > end) {
10                 arrows++;
11                 end = points[i][1];
12             }
13         }
14         return arrows;
15     }
16 };

```

10.9 LC#1027. Longest Arithmetic Subsequence [Medium] [A] [G]

Problem

Find the length of the longest arithmetic subsequence in an integer array.

```

1 class Solution {
2 public:
3     int longestArithSeqLength(vector<int>& nums) {
4         int n = nums.size(), res = 2;
5         vector<unordered_map<int, int>> dp(n);

```

```

6     for (int i = 1; i < n; i++) {
7         for (int j = 0; j < i; j++) {
8             int d = nums[i] - nums[j];
9             int prev = dp[j].count(d) ? dp[j][d] : 1;
10            dp[i][d] = max(dp[i][d], prev + 1);
11            res = max(res, dp[i][d]);
12        }
13    }
14    return res;
15 }
16 };

```

10.10 LC#179. Largest Number [Medium] [A] [MS]

Problem

Arrange non-negative integers to form the largest number.

```

1 class Solution {
2 public:
3     string largestNumber(vector<int>& nums) {
4         vector<string> strs;
5         for (int x : nums) strs.push_back(to_string(x));
6         sort(strs.begin(), strs.end(), [](const string& a, const string& b) {
7             return a + b > b + a;
8         });
9         if (strs[0] == "0") return "0";
10        string res;
11        for (auto& s : strs) res += s;
12        return res;
13    }
14 };

```

10.11 LC#1306. Jump Game III [Medium] [A] [G]

Problem

Starting at **start**, you can jump to $i + \text{arr}[i]$ or $i - \text{arr}[i]$. Return true if you can reach any index with value 0.

```

1 class Solution {
2 public:
3     bool canReach(vector<int>& arr, int start) {
4         int n = arr.size();
5         vector<bool> vis(n, false);
6         queue<int> q;
7         q.push(start);
8         while (!q.empty()) {
9             int i = q.front();
10            q.pop();
11            if (arr[i] == 0) return true;
12            if (vis[i]) continue;
13            vis[i] = true;
14            if (i + arr[i] < n) q.push(i + arr[i]);
15            if (i - arr[i] >= 0) q.push(i - arr[i]);
16        }
17        return false;
18    }
19 };

```

10.12 LC#1229. Meeting Scheduler [Medium] [MS] [G]

Problem

Given two persons' availability and a meeting duration, find the earliest available slot of that duration.

```

1 class Solution {
2 public:
3     vector<int> minAvailableDuration(vector<vector<int>>& s1, vector<vector<int>>& s2, int dur) {
4         sort(s1.begin(), s1.end());
5         sort(s2.begin(), s2.end());
6         int i = 0, j = 0;
7         while (i < (int)s1.size() && j < (int)s2.size()) {

```

```

8         int lo = max(s1[i][0], s2[j][0]);
9         int hi = min(s1[i][1], s2[j][1]);
10        if (hi - lo >= dur) return {lo, lo + dur};
11        if (s1[i][1] < s2[j][1]) i++;
12        else j++;
13    }
14    return {};
15 }
16 };

```

10.13 LC#208. Implement Trie [Medium] [A] [MS] [G]

Problem

Implement a trie with insert, search, and startsWith methods.

```

1 class Trie {
2     struct Node {
3         Node* ch[26] = {};
4         bool end = false;
5     };
6     Node* root = new Node();
7 public:
8     void insert(string w) {
9         Node* n = root;
10        for (char c : w) {
11            if (!n->ch[c - 'a']) n->ch[c - 'a'] = new Node();
12            n = n->ch[c - 'a'];
13        }
14        n->end = true;
15    }
16
17    bool search(string w) {
18        Node* n = root;
19        for (char c : w) {
20            if (!n->ch[c - 'a']) return false;
21            n = n->ch[c - 'a'];
22        }
23        return n->end;
24    }
25
26    bool startsWith(string p) {
27        Node* n = root;
28        for (char c : p) {
29            if (!n->ch[c - 'a']) return false;
30            n = n->ch[c - 'a'];
31        }
32        return true;
33    }
34 };

```

10.14 LC#212. Word Search II [Hard] [A] [G] [MS]

Problem

Find all words from the given word list that exist in the board (DFS + Trie pruning).

```

1 struct TrieNode {
2     TrieNode* ch[26] = {};
3     string word;
4 };
5
6 class Solution {
7     void dfs(vector<vector<char>>& b, TrieNode* node, int i, int j, vector<string>& res) {
8         if (i < 0 || i >= (int)b.size() || j < 0 || j >= (int)b[0].size()) return;
9         if (b[i][j] == '#') return;
10        char c = b[i][j];
11        node = node->ch[c - 'a'];
12        if (!node) return;
13        if (!node->word.empty()) {
14            res.push_back(node->word);
15            node->word = "";
16        }
17        b[i][j] = '#';
18        dfs(b, node, i+1, j, res);
19        dfs(b, node, i-1, j, res);
20        dfs(b, node, i, j+1, res);
21        dfs(b, node, i, j-1, res);
22        b[i][j] = c;

```



```

23     }
24 public:
25     vector<string> findWords(vector<vector<char>>& board, vector<string>& words) {
26         TrieNode* root = new TrieNode();
27         for (auto& w : words) {
28             TrieNode* n = root;
29             for (char c : w) {
30                 if (!n->ch[c - 'a']) n->ch[c - 'a'] = new TrieNode();
31                 n = n->ch[c - 'a'];
32             }
33             n->word = w;
34         }
35         vector<string> res;
36         for (int i = 0; i < (int)board.size(); i++) {
37             for (int j = 0; j < (int)board[0].size(); j++) {
38                 dfs(board, root, i, j, res);
39             }
40         }
41         return res;
42     }
43 };

```

10.15 LC#973. K Closest Points to Origin [Medium] [A] [G] [MS]

Problem

Given points on a plane, return the k closest points to the origin.

```

1 class Solution {
2 public:
3     vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
4         auto cmp = [](vector<int>& a, vector<int>& b) {
5             return a[0]*a[0] + a[1]*a[1] > b[0]*b[0] + b[1]*b[1];
6         };
7         priority_queue<vector<int>, vector<vector<int>>, decltype(cmp)> pq(cmp);
8         for (auto& p : points) {
9             pq.push(p);
10            if ((int)pq.size() > k) pq.pop();
11        }
12        vector<vector<int>> res;
13        while (!pq.empty()) {
14            res.push_back(pq.top());
15            pq.pop();
16        }
17        return res;
18    }
19 };

```

10.16 LC#1167. Minimum Cost to Connect Sticks [Medium] [A]

Problem

Connect all sticks into one; cost = sum of two sticks joined. Minimize total cost (Huffman-style greedy).

```

1 class Solution {
2 public:
3     int connectSticks(vector<int>& sticks) {
4         priority_queue<int, vector<int>, greater<int>> pq(sticks.begin(), sticks.end());
5         int total = 0;
6         while (pq.size() > 1) {
7             int a = pq.top(); pq.pop();
8             int b = pq.top(); pq.pop();
9             total += a + b;
10            pq.push(a + b);
11        }
12        return total;
13    }
14 };

```

10.17 LC#1851. Minimum Interval to Include Each Query [Hard] [G] [A]

Problem

For each query, find the size of the smallest interval $[l, r]$ that contains the query point, or -1.

```

1  class Solution {
2  public:
3      vector<int> minInterval(vector<vector<int>>& intervals, vector<int>& queries) {
4          sort(intervals.begin(), intervals.end());
5          int n = queries.size();
6          vector<int> idx(n);
7          iota(idx.begin(), idx.end(), 0);
8          sort(idx.begin(), idx.end(), [&](int a, int b) {
9              return queries[a] < queries[b];
10         });
11         // minHeap: {size, right}
12         priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
13         vector<int> res(n, -1);
14         int i = 0;
15         for (int qi : idx) {
16             int q = queries[qi];
17             while (i < (int)intervals.size() && intervals[i][0] <= q) {
18                 pq.push({intervals[i][1] - intervals[i][0] + 1, intervals[i][1]});
19                 i++;
20             }
21             while (!pq.empty() && pq.top().second < q) pq.pop();
22             if (!pq.empty()) res[qi] = pq.top().first;
23         }
24         return res;
25     }
26 };

```

10.18 LC#528. Random Pick with Weight [Medium] [A] [G]

Problem

Implement weighted random selection: `pickIndex` returns index i with probability proportional to $w[i]$.

```

1  class Solution {
2      vector<int> prefix;
3  public:
4      Solution(vector<int>& w) {
5          prefix = w;
6          for (int i = 1; i < (int)w.size(); i++) prefix[i] += prefix[i-1];
7      }
8
9      int pickIndex() {
10         int r = rand() % prefix.back() + 1;
11         return lower_bound(prefix.begin(), prefix.end(), r) - prefix.begin();
12     }
13 };

```

10.19 LC#332. Reconstruct Itinerary [Hard] [G] [A]

Problem

Given airline tickets, reconstruct the itinerary in lexical order starting from “JFK” (Euler path, Hierholzer’s).

```

1  class Solution {
2      unordered_map<string, multiset<string>> adj;
3      vector<string> res;
4
5      void dfs(const string& s) {
6          while (!adj[s].empty()) {
7              string nxt = *adj[s].begin();
8              adj[s].erase(adj[s].begin());
9              dfs(nxt);
10         }
11         res.push_back(s);
12     }
13 public:
14     vector<string> findItinerary(vector<vector<string>>& tickets) {
15         for (auto& t : tickets) adj[t[0]].insert(t[1]);
16         dfs("JFK");
17         return {res.rbegin(), res.rend()};
18     }
19 };

```

10.20 LC#907. Sum of Subarray Minimums [Medium] [A] [MS]

Problem

Find sum of minimums of all subarrays. Use monotonic stack, answer modulo $10^9 + 7$.

```

1 class Solution {
2 public:
3     int sumSubarrayMins(vector<int>& arr) {
4         long MOD = 1e9 + 7, res = 0, n = arr.size();
5         stack<int> st;
6         vector<long> dp(n, 0);
7         for (int i = 0; i < n; i++) {
8             while (!st.empty() && arr[st.top()] >= arr[i]) st.pop();
9             int left = st.empty() ? i + 1 : i - st.top();
10            dp[i] = left * arr[i] + (st.empty() ? 0 : dp[st.top()]);
11            res = (res + dp[i]) % MOD;
12            st.push(i);
13        }
14        return res;
15    }
16 };

```

10.21 LC#32. Longest Valid Parentheses [Hard] [A] [G] [MS]

Problem

Return the length of the longest valid (well-formed) parentheses substring.

```

1 class Solution {
2 public:
3     int longestValidParentheses(string s) {
4         stack<int> st;
5         st.push(-1);
6         int res = 0;
7         for (int i = 0; i < (int)s.size(); i++) {
8             if (s[i] == '(') {
9                 st.push(i);
10            } else {
11                st.pop();
12                if (st.empty()) st.push(i);
13                else res = max(res, i - st.top());
14            }
15        }
16        return res;
17    }
18 };

```

10.22 LC#264. Ugly Number II [Medium] [A] [G]

Problem

Find the nth ugly number (whose prime factors are only 2, 3, or 5). Use 3-pointer DP.

```

1 class Solution {
2 public:
3     int nthUglyNumber(int n) {
4         vector<int> dp(n);
5         dp[0] = 1;
6         int i2 = 0, i3 = 0, i5 = 0;
7         for (int i = 1; i < n; i++) {
8             int nxt = min({dp[i2]*2, dp[i3]*3, dp[i5]*5});
9             dp[i] = nxt;
10            if (nxt == dp[i2]*2) i2++;
11            if (nxt == dp[i3]*3) i3++;
12            if (nxt == dp[i5]*5) i5++;
13        }
14        return dp[n-1];
15    }
16 };

```

10.23 LC#151. Reverse Words in a String [Medium] [A] [MS]

Problem

Reverse the order of words in a string, handling multiple spaces.

```

1  class Solution {
2  public:
3      string reverseWords(string s) {
4          istringstream iss(s);
5          vector<string> words;
6          string w;
7          while (iss >> w) words.push_back(w);
8          reverse(words.begin(), words.end());
9          string res;
10         for (int i = 0; i < (int)words.size(); i++) {
11             if (i) res += ' ';
12             res += words[i];
13         }
14         return res;
15     }
16 };

```

10.24 LC#443. String Compression [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Compress an array of characters in-place: consecutive repeating characters encoded as char + count. Return new length.

```

1  class Solution {
2  public:
3      int compress(vector<char>& chars) {
4          int write = 0, i = 0, n = chars.size();
5          while (i < n) {
6              char c = chars[i];
7              int cnt = 0;
8              while (i < n && chars[i] == c) {
9                  i++;
10                 cnt++;
11             }
12             chars[write++] = c;
13             if (cnt > 1) {
14                 for (char ch : to_string(cnt)) chars[write++] = ch;
15             }
16         }
17         return write;
18     }
19 };

```

10.25 LC#48. Rotate Image [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#) [\[Ap\]](#)

Problem

Rotate an $n \times n$ matrix 90 degrees clockwise in-place.

```

1  class Solution {
2  public:
3      void rotate(vector<vector<int>>& mat) {
4          int n = mat.size();
5          // transpose
6          for (int i = 0; i < n; i++)
7              for (int j = i + 1; j < n; j++)
8                  swap(mat[i][j], mat[j][i]);
9          // reverse each row
10         for (auto& row : mat) reverse(row.begin(), row.end());
11     }
12 };

```

10.26 LC#978. Longest Turbulent Subarray [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

A turbulent subarray alternates between greater and lesser. Find the max length turbulent subarray.

```

1 class Solution {
2 public:
3     int maxTurbulenceSize(vector<int>& arr) {
4         int n = arr.size(), res = 1, inc = 1, dec = 1;
5         for (int i = 1; i < n; i++) {
6             if (arr[i] > arr[i-1]) {
7                 inc = dec + 1;
8                 dec = 1;
9             } else if (arr[i] < arr[i-1]) {
10                dec = inc + 1;
11                inc = 1;
12            } else {
13                inc = dec = 1;
14            }
15            res = max(res, max(inc, dec));
16        }
17        return res;
18    }
19 };

```

10.27 LC#2265. Count Nodes Equal to Average of Subtree [Medium] [A]

Problem

Count nodes where node's value equals the average of the values in its subtree.

```

1 class Solution {
2     int res = 0;
3
4     pair<int,int> dfs(TreeNode* node) { // {sum, count}
5         if (!node) return {0, 0};
6         auto [ls, lc] = dfs(node->left);
7         auto [rs, rc] = dfs(node->right);
8         int s = ls + rs + node->val;
9         int c = lc + rc + 1;
10        if (s / c == node->val) res++;
11        return {s, c};
12    }
13 public:
14     int averageOfSubtree(TreeNode* root) {
15         dfs(root);
16         return res;
17     }
18 };

```

10.28 LC#652. Find Duplicate Subtrees [Medium] [A] [G]

Problem

Find all duplicate subtrees in a binary tree. Return the root of each duplicate.

```

1 class Solution {
2     unordered_map<string, int> mp;
3     vector<TreeNode*> res;
4
5     string dfs(TreeNode* n) {
6         if (!n) return "#";
7         string s = to_string(n->val) + "," + dfs(n->left) + "," + dfs(n->right);
8         if (++mp[s] == 2) res.push_back(n);
9         return s;
10    }
11 public:
12     vector<TreeNode*> findDuplicateSubtrees(TreeNode* root) {
13         dfs(root);
14         return res;
15     }
16 };

```

10.29 LC#662. Maximum Width of Binary Tree [Medium] [A] [G]

Problem

Find the maximum width of a binary tree (width = number of nodes in same level, including nulls).

```

1  class Solution {
2  public:
3      int widthOfBinaryTree(TreeNode* root) {
4          if (!root) return 0;
5          queue<pair<TreeNode*, unsigned long long>> q;
6          q.push({root, 1});
7          int res = 0;
8          while (!q.empty()) {
9              int sz = q.size();
10             auto [_, l] = q.front();
11             unsigned long long r = l;
12             for (int i = 0; i < sz; i++) {
13                 auto [node, id] = q.front();
14                 q.pop();
15                 r = id;
16                 if (node->left) q.push({node->left, 2*(id-1)+1});
17                 if (node->right) q.push({node->right, 2*(id-1)+2});
18             }
19             res = max(res, (int)(r - l + 1));
20         }
21         return res;
22     }
23 };

```

10.30 LC#211. Design Add and Search Words Data Structure [Medium] [A] [MS]

Problem

Implement a word dictionary where search supports wildcard . (matches any character).

```

1  struct TrieNode {
2      TrieNode* ch[26] = {};
3      bool end = false;
4  };
5
6  class WordDictionary {
7      TrieNode* root = new TrieNode();
8
9      bool dfs(TrieNode* n, const string& w, int i) {
10         if (i == (int)w.size()) return n->end;
11         if (w[i] != '.' ) {
12             if (!n->ch[w[i] - 'a']) return false;
13             return dfs(n->ch[w[i] - 'a'], w, i+1);
14         }
15         for (int j = 0; j < 26; j++) {
16             if (n->ch[j] && dfs(n->ch[j], w, i+1)) return true;
17         }
18         return false;
19     }
20 public:
21     void addWord(string w) {
22         TrieNode* n = root;
23         for (char c : w) {
24             if (!n->ch[c - 'a']) n->ch[c - 'a'] = new TrieNode();
25             n = n->ch[c - 'a'];
26         }
27         n->end = true;
28     }
29
30     bool search(string w) { return dfs(root, w, 0); }
31 };

```

10.31 LC#1235. Maximum Profit in Job Scheduling [Hard] [A] [G]

Problem

Schedule non-overlapping jobs to maximize profit. Each job has start time, end time, and profit.

```

1  class Solution {
2  public:
3      int jobScheduling(vector<int>& start, vector<int>& end, vector<int>& profit) {
4          int n = start.size();
5          vector<array<int,3>> jobs(n);
6          for (int i = 0; i < n; i++) jobs[i] = {end[i], start[i], profit[i]};
7          sort(jobs.begin(), jobs.end());
8          // dp[endTime] = max profit at that end time
9          map<int,int> dp{{0, 0}};
10         for (auto& j : jobs) {
11             int cur = prev(dp.upper_bound(j[1]))->second + j[2];
12             if (cur > dp.rbegin()->second) dp[j[0]] = cur;
13         }
14         return dp.rbegin()->second;
15     }
16 };

```

10.32 LC#721. Accounts Merge [Medium] [A] [G] [MS]

Problem

Merge accounts sharing the same email. Return accounts with sorted emails.

```

1  class Solution {
2      unordered_map<string, string> parent;
3
4      string find(const string& s) {
5          return parent[s] == s ? s : parent[s] = find(parent[s]);
6      }
7
8      void unite(const string& a, const string& b) {
9          parent[find(a)] = find(b);
10     }
11 public:
12     vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
13         unordered_map<string, string> emailToName;
14         for (auto& acc : accounts) {
15             for (int i = 1; i < (int)acc.size(); i++) {
16                 if (!parent.count(acc[i])) parent[acc[i]] = acc[i];
17                 emailToName[acc[i]] = acc[0];
18                 unite(acc[1], acc[i]);
19             }
20         }
21         unordered_map<string, set<string>> groups;
22         for (auto& [e, _] : parent) groups[find(e)].insert(e);
23         vector<vector<string>> res;
24         for (auto& [root, emails] : groups) {
25             vector<string> acc = {emailToName[root]};
26             acc.insert(acc.end(), emails.begin(), emails.end());
27             res.push_back(acc);
28         }
29         return res;
30     }
31 };

```

10.33 LC#227. Basic Calculator II [Medium] [A] [MS] [G]

Problem

Evaluate a string expression with +, −, *, / and spaces. No parentheses.

```

1  class Solution {
2  public:
3      int calculate(string s) {
4          stack<int> st;
5          int num = 0;
6          char op = '+';
7          for (int i = 0; i < (int)s.size(); i++) {
8              char c = s[i];
9              if (isdigit(c)) num = num * 10 + (c - '0');
10             if ((!isdigit(c) && c != ' ') || i == (int)s.size() - 1) {
11                 if (op == '+') st.push(num);
12                 else if (op == '-') st.push(-num);
13                 else if (op == '*') { int t = st.top(); st.pop(); st.push(t * num); }
14                 else if (op == '/') { int t = st.top(); st.pop(); st.push(t / num); }
15                 op = c;
16                 num = 0;
17             }
18         }
19     }
20 };

```

```
18     }
19     int res = 0;
20     while (!st.empty()) { res += st.top(); st.pop(); }
21     return res;
22 }
23 };
```

10.34 150. Minimum Number of Platforms [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Given arrival and departure times of trains, find the minimum number of platforms to avoid any train waiting.

```
1 class Solution {
2 public:
3     int findPlatform(vector<int>& arr, vector<int>& dep) {
4         sort(arr.begin(), arr.end());
5         sort(dep.begin(), dep.end());
6         int n = arr.size(), platforms = 1, maxP = 1, i = 1, j = 0;
7         while (i < n && j < n) {
8             if (arr[i] <= dep[j]) {
9                 platforms++;
10                i++;
11            } else {
12                platforms--;
13                j++;
14            }
15            maxP = max(maxP, platforms);
16        }
17        return maxP;
18    }
19 };
```


Chapter 11

Additional Hard Problems

11.1 Dynamic Programming

11.1.1 LC#44. Wildcard Matching [Hard] [G] [MS] [A]

Problem

Match string *s* against pattern *p* with wildcards *?* (any single char) and *** (any sequence).

```
1 class Solution {
2 public:
3     bool isMatch(string s, string p) {
4         int m=s.size(),n=p.size();
5         vector<vector<bool>> dp(m+1,vector<bool>(n+1,false));
6         dp[0][0]=true;
7         for(int j=1;j<=n;j++) if(p[j-1]=='*') dp[0][j]=dp[0][j-1];
8         for(int i=1;i<=m;i++) for(int j=1;j<=n;j++)
9             dp[i][j]=p[j-1]=='*'?(dp[i-1][j]||dp[i][j-1])
10                :(p[j-1]=='?'||p[j-1]==s[i-1])&&dp[i-1][j-1];
11         return dp[m][n];
12     }
13 };
```

11.1.2 LC#87. Scramble String [Hard] [A]

Problem

Determine if *s2* is a scrambled string of *s1* (recursive splitting + swapping).

```
1 class Solution {
2     map<tuple<string,string>,bool> memo;
3     bool dp(string s, string t){
4         if(s==t) return true;
5         if(s.size()!=t.size()) return false;
6         auto key=make_tuple(s,t);
7         if(memo.count(key)) return memo[key];
8         int n=s.size(); int sa[26]={},ta[26]={};
9         for(char c:s) sa[c-'a']++; for(char c:t) ta[c-'a']++;
10        if(!equal(sa,sa+26,ta)) return memo[key]=false;
11        for(int i=1;i<n;i++){
12            if((dp(s.substr(0,i),t.substr(0,i))&&dp(s.substr(i),t.substr(i)))||
13               (dp(s.substr(0,i),t.substr(n-i))&&dp(s.substr(i),t.substr(0,n-i))))
14                return memo[key]=true;
15        }
16        return memo[key]=false;
17    }
18 public:
19     bool isScramble(string s1,string s2){return dp(s1,s2);}
20 };
```

11.1.3 LC#123. Best Time to Buy and Sell Stock III [Hard] [A] [MS]

Problem

Find max profit with at most 2 transactions.

```

1 class Solution {
2 public:
3     int maxProfit(vector<int>& prices) {
4         int buy1=INT_MIN,sell1=0,buy2=INT_MIN,sell2=0;
5         for(int p:prices){
6             buy1=max(buy1,-p); sell1=max(sell1,buy1+p);
7             buy2=max(buy2,sell1-p); sell2=max(sell2,buy2+p);
8         }
9         return sell2;
10    }
11 };

```

11.1.4 LC#140. Word Break II [Hard] [A] [G] [Meta]

Problem

Return all sentences where **s** can be segmented using dictionary words.

```

1 class Solution {
2     unordered_set<string> ws;
3     unordered_map<int,vector<string>> memo;
4     vector<string> bt(const string& s,int i){
5         if(memo.count(i)) return memo[i];
6         if(i==(int)s.size()) return {" "};
7         vector<string> res;
8         for(int j=i+1;j<=(int)s.size();j++){
9             string w=s.substr(i,j-i);
10            if(ws.count(w)) for(auto& r:bt(s,j))
11                res.push_back(w+(r.empty()? " ": " "+r));
12        }
13        return memo[i]=res;
14    }
15 public:
16     vector<string> wordBreak(string s,vector<string>& wordDict){
17         ws.insert(wordDict.begin(),wordDict.end()); return bt(s,0);
18     }
19 };

```

11.1.5 LC#174. Dungeon Game [Hard] [A] [G]

Problem

Find the minimum initial health needed to rescue the princess, moving right/down through a dungeon grid.

```

1 class Solution {
2 public:
3     int calculateMinimumHP(vector<vector<int>>& dungeon) {
4         int m=dungeon.size(),n=dungeon[0].size();
5         vector<vector<int>> dp(m+1,vector<int>(n+1,INT_MAX));
6         dp[m][n-1]=dp[m-1][n]=1;
7         for(int i=m-1;i>=0;i--) for(int j=n-1;j>=0;j--)
8             dp[i][j]=max(1,min(dp[i+1][j],dp[i][j+1])-dungeon[i][j]);
9         return dp[0][0];
10    }
11 };

```

11.1.6 LC#329. Longest Increasing Path in a Matrix [Hard] [G] [A]

Problem

Find the length of the longest increasing path in a matrix (DFS + memoization).

```

1 class Solution {
2     int m,n,dx[4]={0,0,1,-1},dy[4]={1,-1,0,0};
3     int dfs(vector<vector<int>>& g,vector<vector<int>>& memo,int i,int j){
4         if(memo[i][j]) return memo[i][j];
5         memo[i][j]=1;
6         for(int d=0;d<4;d++){int ni=i+dx[d],nj=j+dy[d];
7             if(ni>=0&&ni<m&&nj>=0&&nj<n&&g[ni][nj]>g[i][j])
8                 memo[i][j]=max(memo[i][j],1+dfs(g,memo,ni,nj));
9         }
10    }
11 public:
12     int longestIncreasingPath(vector<vector<int>>& g){
13         m=g.size();n=g[0].size();
14         vector<vector<int>> memo(m,vector<int>(n,0)); int res=0;

```

```

15     for(int i=0;i<m;i++) for(int j=0;j<n;j++) res=max(res,dfs(g,memo,i,j));
16     return res;
17 }
18 };

```

11.1.7 LC#354. Russian Doll Envelopes [Hard] [G] [A]

Problem

Find max envelopes you can nest (each must be strictly wider and taller). Sort + LIS trick.

```

1 class Solution {
2 public:
3     int maxEnvelopes(vector<vector<int>>& envelopes) {
4         sort(envelopes.begin(),envelopes.end(),[](auto& a,auto& b){
5             return a[0]<b[0]||(a[0]==b[0]&&a[1]>b[1]);
6         });
7         vector<int> lis;
8         for(auto& e:envelopes){
9             auto it=lower_bound(lis.begin(),lis.end(),e[1]);
10            if(it==lis.end()) lis.push_back(e[1]);
11            else *it=e[1];
12        }
13        return lis.size();
14    }
15 };

```

11.1.8 LC#363. Max Sum of Rectangle No Larger Than K [Hard] [G]

Problem

Find the max sum of a rectangle in a matrix such that the sum is no larger than k .

```

1 class Solution {
2 public:
3     int maxSumSubmatrix(vector<vector<int>>& mat,int k){
4         int m=mat.size(),n=mat[0].size(),res=INT_MIN;
5         for(int l=0;l<n;l++){
6             vector<int> row(m,0);
7             for(int r=l;r<n;r++){
8                 for(int i=0;i<m;i++) row[i]+=mat[i][r];
9                 set<int> ps{0}; int pre=0;
10                for(int x:row){pre+=x;auto it=ps.lower_bound(pre-k);if(it!=ps.end())res=max(res,pre-*it);ps.insert
11                    (pre);}
12            }
13            return res;
14        }
15 };

```

11.1.9 LC#403. Frog Jump [Hard] [A]

Problem

Determine if a frog can cross a river by jumping on stones; each jump can be $k - 1$, k , or $k + 1$ units.

```

1 class Solution {
2 public:
3     bool canCross(vector<int>& stones){
4         unordered_map<int,unordered_set<int>> dp;
5         for(int s:stones) dp[s]={};
6         dp[0].insert(0);
7         for(int s:stones) for(int k:dp[s]) for(int d:{k-1,k,k+1})
8             if(d>0&&dp.count(s+d)) dp[s+d].insert(d);
9         return !dp[stones.back()].empty();
10    }
11 };

```

11.1.10 LC#446. Arithmetic Slices II – Subsequence [Hard] [G]

Problem

Count arithmetic subsequences of length ≥ 3 in an integer array.

```

1 class Solution {
2 public:
3     int numberOfArithmeticSlices(vector<int>& nums){
4         int n=nums.size(); long res=0;
5         vector<unordered_map<long,int>> dp(n);
6         for(int i=1;i<n;i++) for(int j=0;j<i;j++){
7             long d=(long)nums[i]-nums[j];
8             int cnt=(dp[j].count(d)?dp[j][d]:0);
9             dp[i][d]+=cnt+1; res+=cnt;
10        }
11        return res;
12    }
13 };

```

11.1.11 LC#514. Freedom Trail [Hard] [G]

Problem

Spell a key string using a rotating ring; minimize total steps (rotations + button presses). DP on ring.

```

1 class Solution {
2 public:
3     int findRotateSteps(string ring, string key){
4         int m=ring.size(),n=key.size();
5         unordered_map<char,vector<int>> pos;
6         for(int i=0;i<m;i++) pos[ring[i]].push_back(i);
7         vector<vector<int>> dp(n+1,vector<int>(m,0));
8         for(int i=n-1;i>=0;i--){
9             fill(dp[i].begin(),dp[i].end(),INT_MAX);
10            for(int j:pos[key[i]]) for(int p=0;p<m;p++){
11                if(dp[i+1][p]==INT_MAX) continue;
12                int diff=abs(j-p), step=min(diff,m-diff)+1;
13                dp[i][p]=min(dp[i][p],dp[i+1][j]+step);
14            }
15        }
16        return *min_element(dp[0].begin(),dp[0].end());
17    }
18 };

```

11.1.12 LC#546. Remove Boxes [Hard] [G]

Problem

Remove groups of same-colored boxes to maximize score; removing k boxes of same color yields k^2 points.

```

1 class Solution {
2     int dp[100][100][100]={};
3     int dfs(vector<int>& b,int l,int r,int k){
4         if(l>r) return 0;
5         if(dp[l][r][k]) return dp[l][r][k];
6         while(l<r&&b[r]==b[r-1]){r--;k++;}
7         int res=(k+1)*(k+1)+dfs(b,l,r-1,0);
8         for(int i=l;i<r;i++){
9             if(b[i]==b[r]) res=max(res,dfs(b,l,i,k+1)+dfs(b,i+1,r-1,0));
10        }
11        return dp[l][r][k]=res;
12    }
13 public:
14     int removeBoxes(vector<int>& boxes){
15         memset(dp,0,sizeof(dp));
16         return dfs(boxes,0,boxes.size()-1,0);
17     }
18 };

```

11.1.13 LC#552. Student Attendance Record II [Hard] [G]

Problem

Count valid attendance records of length n (at most 1 absent, no 3+ consecutive late). Mod $10^9 + 7$.

```

1 class Solution {
2 public:
3     int checkRecord(int n){
4         long MOD=1e9+7;
5         // dp[a][l]: a=absences(0/1), l=trailing lates(0/1/2)
6         vector<vector<long>> dp(2,vector<long>(3,0));

```

```

7     dp[0][0]=1;
8     for(int i=0;i<n;i++){
9         vector<vector<long>> ndp(2,vector<long>(3,0));
10        for(int a=0;a<2;a++) for(int l=0;l<3;l++){
11            if(!dp[a][l]) continue;
12            ndp[a][0]=(ndp[a][0]+dp[a][l])%MOD; // P
13            if(a==0) ndp[1][0]=(ndp[1][0]+dp[a][l])%MOD; // A
14            if(l<2) ndp[a][l+1]=(ndp[a][l+1]+dp[a][l])%MOD; // L
15        }
16        dp=ndp;
17    }
18    long res=0;
19    for(int a=0;a<2;a++) for(int l=0;l<3;l++) res=(res+dp[a][l])%MOD;
20    return res;
21 }
22 };

```

11.1.14 LC#629. K Inverse Pairs Array [Hard] [G]

Problem

Count arrays of $1 \dots n$ with exactly k inverse pairs, modulo $10^9 + 7$.

```

1 class Solution {
2 public:
3     int kInversePairs(int n, int k){
4         long MOD=1e9+7;
5         vector<long> dp(k+1,0); dp[0]=1;
6         for(int i=2;i<=n;i++){
7             vector<long> ndp(k+1,0); long pre=0;
8             for(int j=0;j<=k;j++){
9                 pre=(pre+dp[j])%MOD;
10                if(j>=i) pre=(pre-dp[j-i]+MOD)%MOD;
11                ndp[j]=pre;
12            }
13            dp=ndp;
14        }
15        return dp[k];
16    }
17 };

```

11.1.15 LC#639. Decode Ways II [Hard] [G] [A]

Problem

Like Decode Ways but $*$ can be any digit 1–9. Count decodings modulo $10^9 + 7$.

```

1 class Solution {
2 public:
3     int numDecodings(string s){
4         long MOD=1e9+7,n=s.size();
5         auto w1=[](char c)->int{return c=='*'?'9':(c!='0')};
6         auto w2=[](char a,char b)->int{
7             if(a=='*'&&b=='*') return 15;
8             if(a=='*') return b<='6'?2:1;
9             if(b=='*') return a=='1'?9:(a=='2'?6:0);
10            int v=(a-'0')*10+(b-'0'); return v>=10&&v<=26;
11        };
12        long p2=1,p1=w1(s[0]);
13        for(int i=1;i<n;i++){long cur=(p1*w1(s[i])+p2*w2(s[i-1],s[i]))%MOD;p2=p1;p1=cur;}
14        return p1;
15    }
16 };

```

11.1.16 LC#664. Strange Printer [Hard] [G]

Problem

Find the minimum turns a printer needs to print a string, where each turn prints any character repeatedly.

```

1 class Solution {
2 public:
3     int strangePrinter(string s){
4         int n=s.size(); vector<vector<int>> dp(n,vector<int>(n,0));
5         for(int len=1;len<=n;len++) for(int i=0;i+len-1<n;i++){
6             int j=i+len-1; dp[i][j]=len;

```

```

7         for(int k=i;k<j;k++) dp[i][j]=min(dp[i][j],dp[i][k]+dp[k+1][j]-(s[k]==s[j]));
8     }
9     return dp[0][n-1];
10 }
11 };

```

11.1.17 LC#689. Maximum Sum of 3 Non-Overlapping Subarrays [Hard] [A]

Problem

Find 3 non-overlapping subarrays of size k with maximum sum; return their starting indices.

```

1 class Solution {
2 public:
3     vector<int> maxSumOfThreeSubarrays(vector<int>& nums, int k){
4         int n=nums.size(); vector<int> w(n-k+1,0);
5         for(int i=0;i<k;i++) w[0]+=nums[i];
6         for(int i=1;i<(int)w.size();i++) w[i]=w[i-1]+nums[i+k-1]-nums[i-1];
7         vector<int> L(w.size()),R(w.size());
8         L[0]=0; for(int i=1;i<(int)w.size();i++) L[i]=(w[i]>w[L[i-1]])?i:L[i-1];
9         R[w.size()-1]=w.size()-1; for(int i=w.size()-2;i>=0;i--) R[i]=(w[i]>w[R[i+1]])?i:R[i+1];
10        vector<int> res; int best=0;
11        for(int m=k;m<=(int)w.size()-k;m++){
12            int l=L[m-k],r=R[m+k],total=w[l]+w[m]+w[r];
13            if(total>best){best=total;res={l,m,r};}
14        }
15        return res;
16    }
17 };

```

11.1.18 LC#730. Count Different Palindromic Subsequences [Hard] [G]

Problem

Count distinct palindromic subsequences of a string over alphabet $\{a,b,c,d\}$, mod $10^9 + 7$.

```

1 class Solution {
2 public:
3     int countPalindromicSubsequences(string s){
4         long MOD=1e9+7,n=s.size();
5         vector<vector<long>> dp(n,vector<long>(n,0));
6         for(int i=0;i<n;i++) dp[i][i]=1;
7         for(int len=2;len<=n;len++) for(int i=0;i+len-1<n;i++){
8             int j=i+len-1;
9             if(s[i]==s[j]){
10                int lo=i+1,hi=j-1;
11                while(lo<=hi&&s[lo]!=s[i]) lo++;
12                while(lo<=hi&&s[hi]!=s[i]) hi--;
13                if(lo>hi) dp[i][j]=(dp[i+1][j-1]*2+2)%MOD;
14                else if(lo==hi) dp[i][j]=(dp[i+1][j-1]*2+1)%MOD;
15                else dp[i][j]=(dp[i+1][j-1]*2-dp[lo+1][hi-1]+MOD)%MOD;
16            } else dp[i][j]=(dp[i+1][j]+dp[i][j-1]-dp[i+1][j-1]+MOD)%MOD;
17        }
18        return dp[0][n-1];
19    }
20 };

```

11.1.19 LC#887. Super Egg Drop [Hard] [A] [G] [MS]

Problem

Find the minimum number of trials to determine the critical floor with k eggs and n floors.

```

1 class Solution {
2 public:
3     int superEggDrop(int k,int n){
4         // dp[m][k] = max floors checkable with m moves, k eggs
5         vector<long> dp(k+1,0); int m=0;
6         while(dp[k]<n){
7             vector<long> ndp(k+1,0);
8             for(int j=1;j<=k;j++) ndp[j]=dp[j-1]+dp[j]+1;
9             dp=ndp; m++;
10        }
11        return m;
12    }
13 };

```

11.1.20 LC#940. Distinct Subsequences II [Hard] [G]

Problem

Count the number of distinct non-empty subsequences of s , mod $10^9 + 7$.

```

1 class Solution {
2 public:
3     int distinctSubseqII(string s){
4         long MOD=1e9+7,dp[26]={};
5         for(char c:s){long sum=0;for(long x:dp) sum=(sum+x)%MOD;dp[c-'a']=(sum+1)%MOD;}
6         long res=0; for(long x:dp) res=(res+x)%MOD;
7         return res;
8     }
9 };

```

11.1.21 LC#956. Tallest Billboard [Hard] [A] [G]

Problem

Split rods into two groups of equal total length; maximize the common length.

```

1 class Solution {
2 public:
3     int tallestBillboard(vector<int>& rods){
4         // dp[diff] = max height of shorter side when |taller-shorter| = diff
5         unordered_map<int,int> dp{{0,0}};
6         for(int r:rods){
7             unordered_map<int,int> ndp=dp;
8             for(auto&[d,h]:dp){
9                 ndp[d+r]=max(ndp[d+r],h); // add r to taller
10                if(r>d) ndp[r-d]=max(ndp[r-d],h+d); // add r to shorter $to$ new diff
11                else ndp[d-r]=max(ndp[d-r],h+r);
12            }
13            dp=ndp;
14        }
15        return dp[0];
16    }
17 };

```

11.1.22 LC#975. Odd Even Jump [Hard] [G]

Problem

Count indices from which you can reach the last index (odd jumps: next $\geq$, even: next $\leq$).

```

1 class Solution {
2     vector<int> makeNext(vector<int>& arr,vector<int>& idx){
3         int n=arr.size(); vector<int> nxt(n,-1); stack<int> st;
4         for(int i:idx){while(!st.empty()){nxt[st.top()]=i;st.pop();}st.push(i);}
5         return nxt;
6     }
7 public:
8     int oddEvenJumps(vector<int>& arr){
9         int n=arr.size(); vector<int> idx(n); iota(idx.begin(),idx.end(),0);
10        sort(idx.begin(),idx.end(),[&](int a,int b){return arr[a]<arr[b]||(arr[a]==arr[b]&&a<b);});
11        auto hi=makeNext(arr,idx);
12        sort(idx.begin(),idx.end(),[&](int a,int b){return arr[a]>arr[b]||(arr[a]==arr[b]&&a<b);});
13        auto lo=makeNext(arr,idx);
14        vector<bool> odd(n,false),even(n,false); odd[n-1]=even[n-1]=true;
15        for(int i=n-2;i>=0;i--){
16            if(hi[i]!=-1) odd[i]=even[hi[i]];
17            if(lo[i]!=-1) even[i]=odd[lo[i]];
18        }
19        return count(odd.begin(),odd.end(),true);
20    }
21 };

```

11.1.23 LC#980. Unique Paths III [Hard] [G]

Problem

Count paths from start to end that walk over every non-obstacle cell exactly once (DFS/backtracking).


```

1 class Solution {
2 public:
3     int uniquePathsIII(vector<vector<int>>& grid){
4         int m=grid.size(),n=grid[0].size(),si=-1,sj=-1,empty=0,res=0;
5         for(int i=0;i<m;i++) for(int j=0;j<n;j++){
6             if(grid[i][j]!=-1) empty++;
7             if(grid[i][j]==1){si=i;sj=j;}
8         }
9         function<void(int,int,int)> dfs=[&](int i,int j,int cnt){
10             if(grid[i][j]==2){if(cnt==empty) res++;return;}
11             int t=grid[i][j]; grid[i][j]=-1;
12             int dx[]={0,0,1,-1},dy[]={1,-1,0,0};
13             for(int d=0;d<4;d++){int ni=i+dx[d],nj=j+dy[d];
14                 if(ni>=0&&ni<m&&nj>=0&&nj<n&&grid[ni][nj]!=-1) dfs(ni,nj,cnt+1);}
15             grid[i][j]=t;
16         };
17         dfs(si,sj,1); return res;
18     }
19 };

```

11.1.24 LC#1000. Minimum Cost to Merge Stones [Hard] [G]

Problem

Merge all piles of stones into one by merging k consecutive piles each time (interval DP).

```

1 class Solution {
2 public:
3     int mergeStones(vector<int>& s,int k){
4         int n=s.size(); if((n-1)%(k-1)!=0) return -1;
5         vector<int> pre(n+1,0); for(int i=0;i<n;i++) pre[i+1]=pre[i]+s[i];
6         vector<vector<int>> dp(n,vector<int>(n,0));
7         for(int len=k;len<=n;len++) for(int i=0;i+len-1<n;i++){
8             int j=i+len-1; dp[i][j]=INT_MAX;
9             for(int m=i;m<j;m+=k-1)
10                 if(dp[i][m]!=INT_MAX&&dp[m+1][j]!=INT_MAX)
11                     dp[i][j]=min(dp[i][j],dp[i][m]+dp[m+1][j]);
12             if((len-1)%(k-1)==0) dp[i][j]+=pre[j+1]-pre[i];
13         }
14         return dp[0][n-1];
15     }
16 };

```

11.1.25 LC#1125. Smallest Sufficient Team [Hard] [A]

Problem

Find the smallest team covering all required skills (bitmask DP over skill subsets).

```

1 class Solution {
2 public:
3     vector<int> smallestSufficientTeam(vector<string>& req,vector<vector<string>>& people){
4         int m=req.size(); unordered_map<string,int> idx;
5         for(int i=0;i<m;i++) idx[req[i]]=i;
6         int full=(1<<m)-1;
7         vector<long long> dp(full+1,LLONG_MAX); vector<int> par(full+1,-1),choice(full+1,-1);
8         dp[0]=0;
9         for(int i=0;i<(int)people.size();i++){
10             int mask=0; for(auto&s:people[i]) if(idx.count(s)) mask|=(1<<idx[s]);
11             for(int j=full;j>=0;j--){
12                 if(dp[j]==LLONG_MAX) continue;
13                 int nj=j|mask;
14                 if(dp[j]+1<dp[nj]){dp[nj]=dp[j]+1;par[nj]=j;choice[nj]=i;}
15             }
16         }
17         vector<int> res; int cur=full;
18         while(cur){res.push_back(choice[cur]);cur=par[cur];}
19         return res;
20     }
21 };

```

11.1.26 LC#1220. Count Vowels Permutation [Hard] [G]

Problem

Count strings of length n where each vowel follows specific vowel rules. Mod $10^9 + 7$.

```

1 class Solution {
2 public:
3     int countVowelPermutation(int n){
4         // a->e, e->a,i, i->a,e,o,u, o->i,u, u->a
5         long MOD=1e9+7,a=1,e=1,i=1,o=1,u=1;
6         for(int k=1;k<n;k++){
7             long na=(e+i+u)%MOD,ne=(a+i)%MOD,ni=(e+o)%MOD,no=i%MOD,nu=(i+o)%MOD;
8             a=na;e=ne;i=ni;o=no;u=nu;
9         }
10        return (a+e+i+o+u)%MOD;
11    }
12 };

```

11.1.27 LC#1312. Minimum Insertion Steps to Make a String Palindrome [Hard] [A]

Problem

Minimum insertions to make s a palindrome = $|s| - \text{longest palindromic subsequence (LCS of } s \text{ and reverse)}$.

```

1 class Solution {
2 public:
3     int minInsertions(string s){
4         string rev=s; reverse(rev.begin(),rev.end()); int n=s.size();
5         vector<vector<int>>> dp(n+1,vector<int>>(n+1,0));
6         for(int i=1;i<=n;i++) for(int j=1;j<=n;j++){
7             dp[i][j]=s[i-1]==rev[j-1]?dp[i-1][j-1]+1:max(dp[i-1][j],dp[i][j-1]);
8         }
9         return n-dp[n][n];
10    };

```

11.1.28 LC#1335. Minimum Difficulty of a Job Schedule [Hard] [A]

Problem

Schedule jobs over d days to minimize total difficulty (max job per day). DP: $O(n^2d)$.

```

1 class Solution {
2 public:
3     int minDifficulty(vector<int>& jd,int d){
4         int n=jd.size(); if(n<d) return -1;
5         vector<vector<int>>> dp(n+1,vector<int>>(d+1,INT_MAX));
6         dp[0][0]=0;
7         for(int day=1;day<=d;day++) for(int i=day;i<=n;i++){
8             int mx=0;
9             for(int j=i;j>=day;j--){
10                mx=max(mx,jd[j-1]);
11                if(dp[j-1][day-1]!=INT_MAX) dp[i][day]=min(dp[i][day],dp[j-1][day-1]+mx);
12            }
13        }
14        return dp[n][d]==INT_MAX?-1:dp[n][d];
15    }
16 };

```

11.1.29 LC#1425. Constrained Subsequence Sum [Hard] [A]

Problem

Find the maximum sum subsequence where consecutive chosen indices are at most k apart (deque DP).

```

1 class Solution {
2 public:
3     int constrainedSubsetSum(vector<int>& nums,int k){
4         int n=nums.size(),res=nums[0]; deque<int> dq; vector<int> dp(n);
5         for(int i=0;i<n;i++){
6             dp[i]=nums[i];
7             if(!dq.empty()) dp[i]=max(dp[i],dp[dq.front()]+nums[i]);
8             res=max(res,dp[i]);
9             while(!dq.empty()&&dp[dq.back()]<=dp[i]) dq.pop_back();
10            if(dp[i]>0) dq.push_back(i);
11            if(!dq.empty()&&dq.front()<i-k+1) dq.pop_front();
12        }
13        return res;
14    }
15 };

```

11.1.30 LC#1444. Number of Ways of Cutting a Pizza [Hard] [G]

Problem

Count ways to cut a pizza into k pieces so each piece has at least one apple (DP + 2D prefix sums). Mod $10^9 + 7$.

```

1  class Solution {
2  public:
3      int ways(vector<string>& pizza, int k){
4          int m=pizza.size(),n=pizza[0].size(); long MOD=1e9+7;
5          vector<vector<int>>> pre(m+1,vector<int>(n+1,0));
6          for(int i=m-1;i>=0;i--) for(int j=n-1;j>=0;j--){
7              pre[i][j]=(pizza[i][j]=='A')+pre[i+1][j]+pre[i][j+1]-pre[i+1][j+1];
8              auto apples=[&](int r1,int c1,int r2,int c2){return pre[r1][c1]-pre[r2][c1]-pre[r1][c2]+pre[r2][c2];};
9              vector<vector<vector<long>>>> dp(k,vector<vector<long>>(m+1,vector<long>(n+1,0)));
10             for(int r=0;r<m;r++) for(int c=0;c<n;c++) dp[0][r][c]=(apples(r,c,m,n)>0);
11             for(int cuts=1;cuts<k;cuts++) for(int r=0;r<m;r++) for(int c=0;c<n;c++){
12                 for(int nr=r+1;nr<m;nr++) if(apples(r,c,nr,n)>0) dp[cuts][r][c]=(dp[cuts][r][c]+dp[cuts-1][nr][c])%MOD;
13                 for(int nc=c+1;nc<n;nc++) if(apples(r,c,m,nc)>0) dp[cuts][r][c]=(dp[cuts][r][c]+dp[cuts-1][r][nc])%MOD;
14             }
15             return dp[k-1][0][0];
16         }
17     };

```

11.1.31 LC#1458. Max Dot Product of Two Subsequences [Hard] [G]

Problem

Find the maximum dot product of two non-empty subsequences from `nums1` and `nums2`.

```

1  class Solution {
2  public:
3      int maxDotProduct(vector<int>& a, vector<int>& b){
4          int m=a.size(),n=b.size();
5          vector<vector<int>>> dp(m,vector<int>(n,INT_MIN));
6          for(int i=0;i<m;i++) for(int j=0;j<n;j++){
7              dp[i][j]=max(dp[i][j],a[i]*b[j]);
8              if(i>0&&j>0&&dp[i-1][j-1]!=INT_MIN) dp[i][j]=max(dp[i][j],dp[i-1][j-1]+a[i]*b[j]);
9              if(i>0) dp[i][j]=max(dp[i][j],dp[i-1][j]);
10             if(j>0) dp[i][j]=max(dp[i][j],dp[i][j-1]);
11         }
12         return dp[m-1][n-1];
13     }
14 };

```

11.1.32 LC#1463. Cherry Pickup II [Hard] [A]

Problem

Two robots collect cherries top-to-bottom in a grid; maximize total (3D DP over row and both columns).

```

1  class Solution {
2  public:
3      int cherryPickup(vector<vector<int>>& grid){
4          int m=grid.size(),n=grid[0].size();
5          vector<vector<vector<int>>>> dp(m,vector<vector<int>>(n,vector<int>(n,-1)));
6          dp[0][0][n-1]=grid[0][0]+grid[0][n-1];
7          for(int r=1;r<m;r++) for(int c1=0;c1<n;c1++) for(int c2=0;c2<n;c2++){
8              int best=-1;
9              for(int d1:{-1,0,1}) for(int d2:{-1,0,1}){int p=c1+d1,q=c2+d2;
10                 if(p>=0&&p<n&&q>=0&&q<n&&dp[r-1][p][q]>=0) best=max(best,dp[r-1][p][q]);
11                 if(best>=0) dp[r][c1][c2]=best+(c1==c2?grid[r][c1]:grid[r][c1]+grid[r][c2]);
12             }
13             int res=0; for(int c1=0;c1<n;c1++) for(int c2=0;c2<n;c2++) res=max(res,dp[m-1][c1][c2]);
14             return res;
15         }
16     };

```

11.1.33 LC#1531. String Compression II [Hard] [G]

Problem

Delete at most k characters to minimize the run-length encoded length (DP on runs).

```

1  class Solution {
2  public:
3      int getLengthOfOptimalCompression(string s, int k) {
4          int n = s.size(); vector<vector<int>>> dp(n+1, vector<int>(k+1, n));
5          dp[0][0] = 0;
6          for(int i=1; i<=n; i++) for(int j=0; j<=k; j++) {
7              if(j>0) dp[i][j] = min(dp[i][j], dp[i-1][j-1]); // delete s[i-1]
8              int same=0, del=0;
9              for(int l=i; l>=1; l--) {
10                 if(s[l-1]==s[i-1]) same++;
11                 else del++;
12                 if(del>j) break;
13                 int cost = (same==1?1:same<10?2:same<100?3:4);
14                 dp[i][j] = min(dp[i][j], dp[l-1][j-del]+cost);
15             }
16         }
17         return dp[n][k];
18     }
19 };

```

11.1.34 LC#1671. Minimum Number of Removals to Make Mountain Array [Hard] [G]

Problem

Minimum removals to make an array a mountain; use LIS from left and right at each peak candidate.

```

1  class Solution {
2      vector<int> lisLen(vector<int>& a) {
3          int n = a.size(); vector<int> res(n, 0); vector<int> tails;
4          for(int i=0; i<n; i++) {
5              auto it = lower_bound(tails.begin(), tails.end(), a[i]);
6              res[i] = it - tails.begin();
7              if(it == tails.end()) tails.push_back(a[i]); else *it = a[i];
8          }
9          return res;
10     }
11 public:
12     int minimumMountainRemovals(vector<int>& nums) {
13         auto L = lisLen(nums);
14         vector<int> rev(nums.rbegin(), nums.rend());
15         auto R = lisLen(rev); reverse(R.begin(), R.end());
16         int n = nums.size(), best = 0;
17         for(int i=1; i<n-1; i++) if(L[i]>0 && R[i]>0) best = max(best, L[i]+R[i]+1);
18         return n - best;
19     }
20 };

```

11.1.35 LC#1691. Maximum Height by Stacking Cuboids [Hard] [A]

Problem

Stack cuboids (rotatable) to maximize height; sort dimensions then apply LIS-like DP.

```

1  class Solution {
2  public:
3      int maxHeight(vector<vector<int>>& cuboids) {
4          for(auto& c: cuboids) sort(c.begin(), c.end());
5          sort(cuboids.begin(), cuboids.end());
6          int n = cuboids.size(); vector<int> dp(n); int res = 0;
7          for(int i=0; i<n; i++) {
8              dp[i] = cuboids[i][2];
9              for(int j=0; j<i; j++)
10                 if(cuboids[j][0] <= cuboids[i][0] && cuboids[j][1] <= cuboids[i][1] && cuboids[j][2] <= cuboids[i][2])
11                     dp[i] = max(dp[i], dp[j]+cuboids[i][2]);
12             res = max(res, dp[i]);
13         }
14         return res;
15     }
16 };

```

11.1.36 LC#1751. Maximum Number of Events That Can Be Attended II [Hard] [A]

Problem

Attend at most k non-overlapping events to maximize total value (DP + binary search).

```

1  class Solution {
2  public:
3      int maxValue(vector<vector<int>>& events, int k){
4          sort(events.begin(), events.end());
5          int n=events.size();
6          vector<vector<int>> dp(k+1, vector<int>(n+1, 0));
7          for(int j=1; j<=k; j++) for(int i=n-1; i>=0; i--){
8              dp[j][i]=dp[j][i+1];
9              int lo=i+1, hi=n;
10             while(lo<hi){int m=(lo+hi)/2; if(events[m][0]>events[i][1]) hi=m; else lo=m+1;}
11             dp[j][i]=max(dp[j][i], events[i][2]+dp[j-1][lo]);
12         }
13         return dp[k][0];
14     }
15 };

```

11.1.37 LC#1359. Count All Valid Pickup and Delivery Options [Hard] [A]

Problem

Count orderings of n pickup/delivery pairs where each delivery comes after its pickup. Mod $10^9 + 7$.

```

1  class Solution {
2  public:
3      int countOrders(int n){
4          long MOD=1e9+7, res=1;
5          for(int i=1; i<=n; i++) res=res*i%MOD*(2*i-1)%MOD;
6          return res;
7      }
8  };

```

11.1.38 LC#1411. Number of Ways to Paint $N \times 3$ Grid [Hard] [A]

Problem

Count ways to paint an $N \times 3$ grid with 3 colors so no adjacent cells share a color. Mod $10^9 + 7$.

```

1  class Solution {
2  public:
3      int numOfWays(int n){
4          // aba-pattern row: 6 initial, abc-pattern row: 6 initial
5          // Transitions: aba->3*aba+2*abc, abc->2*aba+2*abc
6          long MOD=1e9+7, aba=6, abc=6;
7          for(int i=1; i<=n; i++){
8              long na=(3*aba+2*abc)%MOD, nb=(2*aba+2*abc)%MOD;
9              aba=na; abc=nb;
10         }
11         return (aba+abc)%MOD;
12     }
13 };

```

11.2 Binary Search

11.2.1 LC#154. Find Min in Rotated Sorted Array II [Hard] [A]

Problem

Find minimum in a rotated sorted array that may contain duplicates.

```

1  class Solution {
2  public:
3      int findMin(vector<int>& nums) {
4          int l=0, r=nums.size()-1;
5          while(l<r){
6              int m=(l+r)/2;
7              if(nums[m]>nums[r]) l=m+1;

```

```

8         else if (nums[m] < nums[r]) r = m;
9         else r--;
10    }
11    return nums[l];
12 }
13 };

```

11.2.2 LC#164. Maximum Gap [Hard] [A]

Problem

Find the maximum difference between successive elements in the sorted form. Solve in $O(n)$ using bucket sort.

```

1 class Solution {
2 public:
3     int maximumGap(vector<int>& nums) {
4         int n = nums.size(); if (n < 2) return 0;
5         int mn = *min_element(nums.begin(), nums.end());
6         int mx = *max_element(nums.begin(), nums.end());
7         if (mn == mx) return 0;
8         int bsize = max(1, (mx - mn) / (n - 1)), nb = (mx - mn) / bsize + 1;
9         vector<int> bmin(nb, INT_MAX), bmax(nb, INT_MIN);
10        for (int x : nums) { int b = (x - mn) / bsize; bmin[b] = min(bmin[b], x); bmax[b] = max(bmax[b], x); }
11        int res = 0, prev = mn;
12        for (int i = 0; i < nb; i++) { if (bmin[i] == INT_MAX) continue; res = max(res, bmin[i] - prev); prev = bmax[i]; }
13        return res;
14    }
15 };

```

11.2.3 LC#410. Split Array Largest Sum [Hard] [G] [A] [MS]

Problem

Split array into k subarrays to minimize the largest subarray sum (binary search on answer).

```

1 class Solution {
2 public:
3     int splitArray(vector<int>& nums, int k) {
4         int lo = *min_element(nums.begin(), nums.end());
5         long hi = accumulate(nums.begin(), nums.end(), 0LL);
6         while (lo < hi) { long mid = (lo + hi) / 2, cur = 0; int cnt = 1;
7             for (int x : nums) { if (cur + x > mid) { cnt++; cur = 0; } cur += x; }
8             if (cnt <= k) hi = mid; else lo = mid + 1; }
9         return lo;
10    }
11 };

```

11.2.4 LC#668. Kth Smallest Number in Multiplication Table [Hard] [G]

Problem

Find the k -th smallest number in an $m \times n$ multiplication table (binary search).

```

1 class Solution {
2 public:
3     int findKthNumber(int m, int n, int k) {
4         int lo = 1, hi = m * n;
5         while (lo < hi) { int mid = (lo + hi) / 2, cnt = 0;
6             for (int i = 1; i <= m; i++) cnt += min(mid / i, n);
7             if (cnt >= k) hi = mid; else lo = mid + 1; }
8         return lo;
9    }
10 };

```

11.2.5 LC#1095. Find in Mountain Array [Hard] [A] [G]

Problem

Find the minimum index of **target** in a mountain array using at most 100 API calls (binary search $\times 3$).

```

1 // Interface: MountainArray (get, length)
2 class Solution {
3 public:
4     int findInMountainArray(int target, MountainArray& ma){
5         int n=ma.length(), lo=0, hi=n-1;
6         while(lo<hi){int m=(lo+hi)/2; if(ma.get(m)<ma.get(m+1)) lo=m+1; else hi=m;}
7         int peak=lo;
8         lo=0; hi=peak;
9         while(lo<=hi){int m=(lo+hi)/2, v=ma.get(m); if(v==target) return m; if(v<target) lo=m+1; else hi=m-1;}
10        lo=peak; hi=n-1;
11        while(lo<=hi){int m=(lo+hi)/2, v=ma.get(m); if(v==target) return m; if(v>target) lo=m+1; else hi=m-1;}
12        return -1;
13    }
14 };

```

11.3 Strings & Parsing

11.3.1 LC#68. Text Justification [Hard] [G] [MS]

Problem

Format words into lines of exactly `maxWidth`, distributing spaces evenly; last line is left-justified.

```

1 class Solution {
2 public:
3     vector<string> fullJustify(vector<string>& words, int mw) {
4         vector<string> res; int n=words.size(), i=0;
5         while(i<n){
6             int j=i, len=words[i].size();
7             while(j+1<n&&len+1+words[j+1].size()<=mw){j++; len+=1+words[j].size();}
8             int totalChars=0; for(int k=i; k<=j; k++) totalChars+=words[k].size();
9             int gaps=j-i; string line;
10            if(j==n-1||gaps==0){
11                for(int k=i; k<=j; k++){if(k>i) line+=' '; line+=words[k];}
12                line+=string(mw-line.size(), ' ');
13            } else {
14                int sp=(mw-totalChars)/gaps, extra=(mw-totalChars)%gaps;
15                for(int k=i; k<=j; k++){line+=words[k]; if(k<j) line+=string(sp+(k-i<extra?1:0), ' ');}
16            }
17            res.push_back(line); i=j+1;
18        }
19        return res;
20    }
21 };

```

11.3.2 LC#214. Shortest Palindrome [Hard] [G]

Problem

Add the fewest characters in front of `s` to make it a palindrome (KMP failure function).

```

1 class Solution {
2 public:
3     string shortestPalindrome(string s) {
4         string rev=s; reverse(rev.begin(), rev.end());
5         string t=s+'#'+rev; int n=t.size();
6         vector<int> kmp(n, 0);
7         for(int i=1; i<n; i++){
8             int j=kmp[i-1];
9             while(j&&t[i]!=t[j]) j=kmp[j-1];
10            kmp[i]=j+(t[i]==t[j]);
11        }
12        return rev.substr(0, (int)s.size()-kmp[n-1])+s;
13    }
14 };

```

11.3.3 LC#224. Basic Calculator [Hard] [A] [G] [MS]

Problem

Evaluate a string expression with `+`, `-`, and parentheses.

```

1 class Solution {
2 public:

```

```

3   int calculate(string s) {
4       stack<int> st; int res=0,num=0,sign=1;
5       for(char c:s){
6           if(isdigit(c)) num=num*10+(c-'0');
7           else if(c=='+'){res+=sign*num;num=0;sign=1;}
8           else if(c=='-'){res+=sign*num;num=0;sign=-1;}
9           else if(c=='('){st.push(res);st.push(sign);res=0;sign=1;}
10          else if(c==')'){res+=sign*num;num=0;res*=st.top();st.pop();res+=st.top();st.pop();}
11      }
12      return res+sign*num;
13  }
14  };

```

11.3.4 LC#273. Integer to English Words [Hard] [A] [MS] [G]

Problem

Convert a non-negative integer to its English words representation.

```

1   class Solution {
2       vector<string> ones={"","One","Two","Three","Four","Five","Six","Seven","Eight","Nine",
3           "Ten","Eleven","Twelve","Thirteen","Fourteen","Fifteen","Sixteen","Seventeen","Eighteen","Nineteen"};
4       vector<string> tens={"","Twenty","Thirty","Forty","Fifty","Sixty","Seventy","Eighty","Ninety"};
5       string helper(int n){
6           if(n<20) return ones[n];
7           if(n<100) return tens[n/10]+(n%10?" "+ones[n%10]: "");
8           return ones[n/100]+" Hundred"+(n%100?" "+helper(n%100): "");
9       }
10      public:
11          string numberToWords(int n) {
12              if(!n) return "Zero";
13              vector<string> suf={"","Thousand","Million","Billion"};
14              string res; int i=0;
15              while(n){
16                  if(n%1000){string s=helper(n%1000);res=(suf[i].empty()?s:s+" "+suf[i])+(res.empty()?"" : " "+res);}
17                  n/=1000; i++;
18              }
19              return res;
20          }
21      };

```

11.3.5 LC#472. Concatenated Words [Hard] [A]

Problem

Find all words in the dictionary that can be formed by concatenating at least two other dictionary words.

```

1   class Solution {
2       unordered_set<string> ws;
3       bool canForm(const string& w){
4           int n=w.size(); vector<int> dp(n+1,-1); dp[0]=0;
5           for(int i=1;i<=n;i++) for(int j=0;j<i;j++)
6               if(dp[j]>=0&&ws.count(w.substr(j,i-j))) {dp[i]=max(dp[i],dp[j]+1); break;}
7           return dp[n]>=2;
8       }
9      public:
10          vector<string> findAllConcatenatedWordsInADict(vector<string>& words){
11              ws.insert(words.begin(),words.end());
12              vector<string> res;
13              for(auto& w:words) if(canForm(w)) res.push_back(w);
14              return res;
15          }
16      };

```

11.3.6 LC#726. Number of Atoms [Hard] [G]

Problem

Parse a chemical formula string and return the atom counts in sorted order.

```

1   class Solution {
2   public:
3       string countOfAtoms(string f){
4           int i=0,n=f.size();
5           function<map<string,int>>()> parse=[&]()->map<string,int>{
6               map<string,int> res;
7               while(i<n&&f[i]!=' '){

```



```

8         if(f[i]=='('){i++;auto inner=parse();i++;int num=0;
9             while(i<n&&isdigit(f[i])) num=num*10+(f[i++]-'0');
10            if(!num) num=1; for(auto&[el,c]:inner) res[el]+=c*num;
11        } else {
12            string el; el+=f[i++]; while(i<n&&islower(f[i])) el+=f[i++];
13            int num=0; while(i<n&&isdigit(f[i])) num=num*10+(f[i++]-'0');
14            if(!num) num=1; res[el]+=num;
15        }
16    }
17    return res;
18 };
19 auto mp=parse(); string ans;
20 for(auto&[el,c]:mp){ans+=el;if(c>1)ans+=to_string(c);}
21 return ans;
22 }
23 };

```

11.3.7 LC#1032. Stream of Characters [Hard] [A]

Problem

Design a system to check if any word in a list is a suffix of a stream of letters (reversed Trie).

```

1 struct TNode{TNode*ch[26]={};bool end=false;};
2 class StreamChecker {
3     TNode* root=new TNode(); string buf;
4 public:
5     StreamChecker(vector<string>& words){
6         for(auto&w:words){TNode*n=root;for(int i=w.size()-1;i>=0;i--){if(!n->ch[w[i]-'a'])n->ch[w[i]-'a']=new
7             TNode();n=n->ch[w[i]-'a'];}n->end=true;}
8     }
9     bool query(char c){
10         buf+=c; TNode*n=root;
11         for(int i=buf.size()-1;i>=0;i--){
12             if(!n->ch[buf[i]-'a']) return false;
13             n=n->ch[buf[i]-'a']; if(n->end) return true;
14         }
15         return false;
16     }
17 };

```

11.3.8 LC#1044. Longest Duplicate Substring [Hard] [A]

Problem

Find the longest substring that appears at least twice (binary search + Rabin-Karp rolling hash).

```

1 class Solution {
2 public:
3     string longestDupSubstring(string s){
4         int n=s.size(),lo=1,hi=n-1; string res="";
5         auto check=[&](int len)->string{
6             long MOD=1e18+9,BASE=31,h=0,pw=1;
7             for(int i=0;i<len;i++){h=(h*BASE+s[i]-'a')%MOD;if(i<len-1)pw=pw*BASE%MOD;}
8             unordered_map<long,vector<int>> seen{{h,{0}}};
9             for(int i=1;i<=n-len;i++){
10                 h=((h-(s[i-1]-'a')*pw%MOD+MOD)*BASE+s[i+len-1]-'a')%MOD;
11                 for(int j:seen[h]) if(s.substr(j,len)==s.substr(i,len)) return s.substr(i,len);
12                 seen[h].push_back(i);
13             }
14             return "";
15         };
16         while(lo<=hi){int m=(lo+hi)/2;string t=check(m);if(!t.empty()){res=t;lo=m+1;}else hi=m-1;}
17         return res;
18     }
19 };

```

11.4 Divide & Conquer / Merge Sort

11.4.1 LC#315. Count of Smaller Numbers After Self [Hard] [G] [A]

Problem

For each element, count how many numbers to its right are smaller (merge sort on indices).

```

1  class Solution {
2      vector<int> res, idx;
3      void msort(vector<int>& nums, int l, int r){
4          if(r-l<=1) return;
5          int m=(l+r)/2; msort(nums, l, m); msort(nums, m, r);
6          vector<int> tmp; int i=l, j=m, cnt=0;
7          while(i<m&&j<r){
8              if(nums[idx[j]]<nums[idx[i]]){tmp.push_back(idx[j++]); cnt++;}
9              else{res[idx[i]]+=cnt; tmp.push_back(idx[i++]);}
10         }
11         while(i<m){res[idx[i]]+=cnt; tmp.push_back(idx[i++]);}
12         while(j<r) tmp.push_back(idx[j++]);
13         for(int k=l; k<r; k++) idx[k]=tmp[k-l];
14     }
15 public:
16     vector<int> countSmaller(vector<int>& nums){
17         int n=nums.size(); res.assign(n, 0); idx.resize(n);
18         iota(idx.begin(), idx.end(), 0); msort(nums, 0, n); return res;
19     }
20 };

```

11.4.2 LC#493. Reverse Pairs [Hard] [A] [G]

Problem

Count pairs (i, j) with $i < j$ and $\text{nums}[i] > 2 \cdot \text{nums}[j]$ (merge sort).

```

1  class Solution {
2      long cnt=0;
3      void msort(vector<int>& nums, int l, int r){
4          if(r-l<=1) return;
5          int m=(l+r)/2; msort(nums, l, m); msort(nums, m, r);
6          int j=m;
7          for(int i=l; i<m; i++){while(j<r&&(long)nums[i]>2LL*nums[j]) j++; cnt+=j-m;}
8          inplace_merge(nums.begin()+l, nums.begin()+m, nums.begin()+r);
9      }
10 public:
11     int reversePairs(vector<int>& nums){msort(nums, 0, nums.size()); return cnt;}
12 };

```

11.5 Graphs

11.5.1 LC#847. Shortest Path Visiting All Nodes [Hard] [G] [A]

Problem

Find the shortest path that visits every node in an undirected graph (BFS + bitmask state).

```

1  class Solution {
2  public:
3      int shortestPathLength(vector<vector<int>>& graph){
4          int n=graph.size(), full=(1<<n)-1;
5          queue<tuple<int, int, int>> q;
6          vector<vector<bool>> vis(n, vector<bool>(1<<n, false));
7          for(int i=0; i<n; i++){q.push({i, 1<<i, 0}); vis[i][1<<i]=true;}
8          while(!q.empty()){
9              auto [node, mask, steps]=q.front(); q.pop();
10             if(mask==full) return steps;
11             for(int nb:graph[node]){int nm=mask|(1<<nb);
12                 if(!vis[nb][nm]){vis[nb][nm]=true; q.push({nb, nm, steps+1});}}
13         }
14         return -1;
15     }
16 };

```

11.5.2 LC#1203. Sort Items by Groups Respecting Dependencies [Hard] [G]

Problem

Topological sort at two levels: items within groups, and groups themselves.

```

1  class Solution {
2      vector<int> topo(int sz, vector<vector<int>>& adj, vector<int>& in){
3          queue<int> q; vector<int> res;

```

```

4     for(int i=0;i<sz;i++) if(!in[i]) q.push(i);
5     while(!q.empty()){int u=q.front();q.pop();res.push_back(u);for(int v:adj[u]) if(!--in[v]) q.push(v);}
6     return res;
7 }
8 public:
9     vector<int> sortItems(int n,int m,vector<int>& group,vector<vector<int>>& beforeItems){
10         int gid=m;
11         for(int i=0;i<n;i++) if(group[i]==-1) group[i]=gid++;
12         int total=gid;
13         vector<vector<int>> gadj(total),iadj(n); vector<int> gin(total,0),iin(n,0);
14         for(int i=0;i<n;i++) for(int b:beforeItems[i]){
15             iadj[b].push_back(i);iin[i]++;
16             if(group[b]!=group[i]){gadj[group[b]].push_back(group[i]);gin[group[i]]++;}
17         }
18         auto go=topo(total,gadj,gin),io=topo(n,iadj,iin);
19         if((int)go.size()!=total||((int)io.size()!=n) return {};
20         vector<vector<int>> gItems(total);
21         for(int i:io) gItems[group[i]].push_back(i);
22         vector<int> res;
23         for(int g:go) for(int i:gItems[g]) res.push_back(i);
24         return res;
25     }
26 };

```

11.5.3 LC#1857. Largest Color Value in a Directed Graph [Hard] [G]

Problem

Find the maximum frequency of any color on any path in a directed graph (topological sort + DP). Return -1 if cycle.

```

1 class Solution {
2 public:
3     int largestPathValue(string colors,vector<vector<int>>& edges){
4         int n=colors.size(); vector<vector<int>> adj(n); vector<int> in(n,0);
5         for(auto& e:edges){adj[e[0]].push_back(e[1]);in[e[1]]++;}
6         queue<int> q; for(int i=0;i<n;i++) if(!in[i]) q.push(i);
7         vector<vector<int>> dp(n,vector<int>(26,0));
8         int processed=0,res=0;
9         while(!q.empty()){
10             int u=q.front();q.pop();processed++;
11             dp[u][colors[u]-'a']++;
12             res=max(res,*max_element(dp[u].begin(),dp[u].end()));
13             for(int v:adj[u]){for(int c=0;c<26;c++) dp[v][c]=max(dp[v][c],dp[u][c]);if(!--in[v]) q.push(v);}
14         }
15         return processed==n?res:-1;
16     }
17 };

```

11.5.4 LC#2050. Parallel Courses III [Hard] [A]

Problem

Find the minimum months to complete all courses given prerequisites and individual times (topological sort + DP).

```

1 class Solution {
2 public:
3     int minimumTime(int n,vector<vector<int>>& relations,vector<int>& time){
4         vector<vector<int>> adj(n+1); vector<int> in(n+1,0);
5         for(auto& r:relations){adj[r[0]].push_back(r[1]);in[r[1]]++;}
6         queue<int> q; vector<int> dp(n+1,0);
7         for(int i=1;i<=n;i++) if(!in[i]){q.push(i);dp[i]=time[i-1];}
8         int res=0;
9         while(!q.empty()){
10             int u=q.front();q.pop(); res=max(res,dp[u]);
11             for(int v:adj[u]){dp[v]=max(dp[v],dp[u]+time[v-1]);if(!--in[v]) q.push(v);}
12         }
13         return res;
14     }
15 };

```

11.6 Heaps & Priority Queues

11.6.1 LC#480. Sliding Window Median [Hard] [A] [G]

Problem

Return the medians of all sliding windows of size k (two multisets: max-heap lo, min-heap hi).

```

1 class Solution {
2 public:
3     vector<double> medianSlidingWindow(vector<int>& nums, int k){
4         multiset<int> lo, hi; vector<double> res;
5         auto balance=[&]() {
6             while(lo.size()>hi.size()+1){hi.insert(*lo.rbegin());lo.erase(lo.find(*lo.rbegin()));}
7             while(hi.size()>lo.size()){lo.insert(*hi.begin());hi.erase(hi.begin());}
8         };
9         for(int i=0;i<(int)nums.size();i++){
10            if(lo.empty()||nums[i]<=*lo.rbegin()) lo.insert(nums[i]); else hi.insert(nums[i]);
11            balance();
12            if(i>=k){int r=nums[i-k];if(lo.count(r))lo.erase(lo.find(r));else hi.erase(hi.find(r));balance();}
13            if(i>=k-1) res.push_back(k%2?*lo.rbegin():((double)*lo.rbegin()+*hi.begin())/2.0);
14        }
15        return res;
16    }
17 };

```

11.6.2 LC#632. Smallest Range Covering K Lists [Hard] [A] [G]

Problem

Find the smallest range that includes at least one element from each of k sorted lists (min-heap).

```

1 class Solution {
2 public:
3     vector<int> smallestRange(vector<vector<int>>& nums){
4         using T=tuple<int,int,int>;
5         priority_queue<T,vector<T>,greater<T>> pq;
6         int curMax=INT_MIN;
7         for(int i=0;i<(int)nums.size();i++){pq.push({nums[i][0],i,0});curMax=max(curMax,nums[i][0]);}
8         int lo=INT_MIN,hi=INT_MAX;
9         while((int)pq.size()==(int)nums.size()){
10            auto[val,i,j]=pq.top();pq.pop();
11            if(curMax-val<hi-lo){lo=val;hi=curMax;}
12            if(j+1<(int)nums[i].size()){pq.push({nums[i][j+1],i,j+1});curMax=max(curMax,nums[i][j+1]);}
13            else break;
14        }
15        return {lo,hi};
16    }
17 };

```

11.6.3 LC#871. Minimum Number of Refueling Stops [Hard] [A] [G]

Problem

Find minimum stops to reach the target; at each station you can refuel (greedy + max-heap).

```

1 class Solution {
2 public:
3     int minRefuelStops(int target, int fuel, vector<vector<int>>& stations){
4         priority_queue<int> pq; int stops=0,i=0,n=stations.size();
5         while((long)fuel<target){
6             while(i<n&&stations[i][0]<=fuel){pq.push(stations[i][1]);i++;}
7             if(pq.empty()) return -1;
8             fuel+=pq.top();pq.pop();stops++;
9         }
10        return stops;
11    }
12 };

```

11.6.4 LC#895. Maximum Frequency Stack [Hard] [A] [G] [MS]

Problem

Design a stack where pop returns the most frequent element (ties: most recently pushed).

```

1 class FreqStack {
2     unordered_map<int,int> freq;
3     unordered_map<int,vector<int>> group;
4     int maxfreq=0;
5 public:
6     void push(int val){int f=++freq[val];maxfreq=max(maxfreq,f);group[f].push_back(val);}
7     int pop(){int val=group[maxfreq].back();group[maxfreq].pop_back();freq[val]--;if(group[maxfreq].empty())
8         maxfreq--;return val;}
9 };

```

11.7 Greedy

11.7.1 LC#330. Patching Array [Hard] [G]

Problem

Find the minimum number of patches to add so every integer in $[1, n]$ can be formed by subset sums.

```

1 class Solution {
2 public:
3     int minPatches(vector<int>& nums, int n) {
4         long miss=1; int patches=0,i=0;
5         while(miss<=n){
6             if(i<(int)nums.size()&&nums[i]<=miss) miss+=nums[i++];
7             else{miss*=2;patches++;}
8         }
9         return patches;
10    }
11 };

```

11.7.2 LC#517. Super Washing Machines [Hard] [G] [A]

Problem

Find the minimum number of moves to equalize the number of dresses across washing machines.

```

1 class Solution {
2 public:
3     int findMinMoves(vector<int>& machines){
4         int n=machines.size(),total=0;
5         for(int x:machines) total+=x;
6         if(total%n) return -1;
7         int target=total/n,res=0,balance=0;
8         for(int m:machines){
9             balance+=m-target;
10            res=max(res,max(abs(balance),m-target));
11        }
12        return res;
13    }
14 };

```

11.7.3 LC#992. Subarrays with K Different Integers [Hard] [G] [A]

Problem

Count subarrays with exactly k distinct integers (atMost(k) - atMost($k-1$) trick).

```

1 class Solution {
2     int atMostK(vector<int>& nums,int k){
3         unordered_map<int,int> cnt; int res=0,l=0;
4         for(int r=0;r<(int)nums.size();r++){
5             if(!cnt[nums[r]]++) k--;
6             while(k<0) if(!--cnt[nums[l++]]) k++;
7             res+=r-l+1;
8         }
9         return res;
10    }
11 public:
12     int subarraysWithKDistinct(vector<int>& nums,int k){return atMostK(nums,k)-atMostK(nums,k-1);}
13 };

```

11.7.4 LC#995. Minimum Number of K Consecutive Bit Flips [Hard] [G]

Problem

Find the minimum flips of k -length subarrays to make all bits 1, or -1 if impossible.

```

1 class Solution {
2 public:
3     int minKBitFlips(vector<int>& nums, int k){
4         int n=nums.size(), res=0, flipped=0; vector<int> flip(n,0);
5         for(int i=0; i<n; i++){
6             flipped ^= flip[i];
7             if((nums[i]^flipped)==0){
8                 if(i+k>n) return -1;
9                 flipped ^= 1; res++;
10                if(i+k<n) flip[i+k]^=1;
11            }
12        }
13        return res;
14    }
15 };

```

11.8 Math & Geometry

11.8.1 LC#149. Max Points on a Line [Hard] [G] [A]

Problem

Find the maximum number of points that lie on the same straight line.

```

1 class Solution {
2 public:
3     int maxPoints(vector<vector<int>>& pts) {
4         int n=pts.size(), res=1;
5         for(int i=0; i<n; i++){
6             map<pair<int, int>, int> slopes;
7             for(int j=i+1; j<n; j++){
8                 int dx=pts[j][0]-pts[i][0], dy=pts[j][1]-pts[i][1];
9                 int g=__gcd(abs(dx), abs(dy));
10                if(g){dx/=g; dy/=g;}
11                if(dx<0 || (dx==0&&dy<0)) {dx=-dx; dy=-dy;}
12                res=max(res, ++slopes[{dx, dy}]+1);
13            }
14        }
15        return res;
16    }
17 };

```

11.8.2 LC#233. Number of Digit One [Hard] [MS] [A]

Problem

Count total occurrences of digit 1 in all non-negative integers $\leq n$.

```

1 class Solution {
2 public:
3     int countDigitOne(int n) {
4         long res=0;
5         for(long i=1; i<=n; i*=10){
6             long high=n/(i*10), cur=(n/i)%10, low=n%i;
7             if(cur>1) res+=(high+1)*i;
8             else if(cur==1) res+=high*i+low+1;
9             else res+=high*i;
10        }
11        return res;
12    }
13 };

```

11.8.3 LC#321. Create Maximum Number [Hard] [G]

Problem

Create the largest number of length k from digits of two arrays while preserving relative order.

```

1  class Solution {
2      vector<int> maxK(vector<int>& a, int k){
3          vector<int> res; int drop=a.size()-k;
4          for(int x:a){while(drop&&!res.empty()&&res.back()<x){res.pop_back();drop--;}res.push_back(x);}
5          return {res.begin(),res.begin()+k};
6      }
7      vector<int> merge(vector<int>& a, vector<int>& b){
8          vector<int> res; int i=0,j=0;
9          while(i<(int)a.size()||j<(int)b.size()){
10             res.push_back(lexicographical_compare(b.begin()+j,b.end(),a.begin()+i,a.end())?a[i++]:b[j++]);
11             return res;
12         }
13     public:
14         vector<int> maxNumber(vector<int>& n1, vector<int>& n2, int k){
15             int m=n1.size(),n=n2.size(); vector<int> res(k,0);
16             for(int i=max(0,k-n);i<=min(k,m);i++){
17                 auto a=maxK(n1,i),b=maxK(n2,k-i),c=merge(a,b);
18                 if(c>res) res=c;
19             }
20             return res;
21         }
22 };

```

11.8.4 LC#391. Perfect Rectangle [Hard] [G]

Problem

Determine if all given rectangles form exactly a large rectangle without overlap or gaps.

```

1  class Solution {
2  public:
3      bool isRectangleCover(vector<vector<int>>& rects){
4          set<pair<int,int>> corners; int x1=INT_MAX,y1=INT_MAX,x2=INT_MIN,y2=INT_MIN; long area=0;
5          for(auto& r:rects){
6              x1=min(x1,r[0]);y1=min(y1,r[1]);x2=max(x2,r[2]);y2=max(y2,r[3]);
7              area+=(long)(r[2]-r[0])*(r[3]-r[1]);
8              for(auto& p:vector<pair<int,int>>{{r[0],r[1]},{r[0],r[3]},{r[2],r[1]},{r[2],r[3]}})
9                  if(!corners.insert(p).second) corners.erase(p);
10             }
11             return area==(long)(x2-x1)*(y2-y1)&&corners.size()==4&&
12                 corners.count({x1,y1})&&corners.count({x1,y2})&&
13                 corners.count({x2,y1})&&corners.count({x2,y2});
14         }
15 };

```

11.8.5 LC#679. 24 Game [Hard] [G]

Problem

Determine if 4 numbers can be combined with +, -, *, / and parentheses to get 24.

```

1  class Solution {
2      bool solve(vector<double> v){
3          if(v.size()==1) return abs(v[0]-24)<1e-6;
4          int n=v.size();
5          for(int i=0;i<n;i++) for(int j=0;j<n;j++){
6              if(i==j) continue;
7              vector<double> nxt;
8              for(int k=0;k<n;k++) if(k!=i&&k!=j) nxt.push_back(v[k]);
9              double a=v[i],b=v[j];
10             for(double c:{a+b,a-b,a*b,b!=0?a/b:1e9}){nxt.push_back(c);if(solve(nxt)) return true;nxt.pop_back();}
11         }
12         return false;
13     }
14     public:
15         bool judgePoint24(vector<int>& c){vector<double> v(c.begin(),c.end());return solve(v);}
16 };

```

11.9 Backtracking

11.9.1 LC#282. Expression Add Operators [Hard] [A] [G] [Meta]

Problem

Find all expressions by inserting +, -, * between digits of num that equal target.

```

1 class Solution {
2     void bt(string& num, long target, int pos, long eval, long mult, string cur, vector<string>& res){
3         if(pos==(int)num.size()&&eval==target){res.push_back(cur);return;}
4         for(int len=1; pos+len<=(int)num.size(); len++){
5             string s=num.substr(pos, len);
6             if(s.size()>1&&s[0]!='0') break;
7             long val=stoll(s);
8             if(pos==0) bt(num, target, pos+len, val, val, s, res);
9             else{
10                bt(num, target, pos+len, eval+val, val, cur+"+"+s, res);
11                bt(num, target, pos+len, eval-val, -val, cur+"-"+s, res);
12                bt(num, target, pos+len, eval-mult+mult*val, mult*val, cur+"*"+s, res);
13            }
14        }
15    }
16 public:
17     vector<string> addOperators(string num, int target){
18         vector<string> res; bt(num, target, 0, 0, 0, "", res); return res;
19     }
20 };

```

11.9.2 LC#301. Remove Invalid Parentheses [Hard] [A] [Meta]

Problem

Remove the minimum number of invalid parentheses to make the input string valid. Return all results (BFS).

```

1 class Solution {
2     bool valid(const string& s){int c=0;for(char x:s){if(x=='(')c++;else if(x==')')if(--c<0)return false;}return c==0;}
3 public:
4     vector<string> removeInvalidParentheses(string s) {
5         unordered_set<string> vis{s}; queue<string> q; q.push(s);
6         vector<string> res; bool found=false;
7         while(!q.empty()){
8             int sz=q.size();
9             while(sz--){
10                string cur=q.front();q.pop();
11                if(valid(cur)){res.push_back(cur);found=true;}
12                if(found) continue;
13                for(int i=0;i<(int)cur.size();i++){
14                    if(cur[i]!='('&&cur[i]!=')') continue;
15                    string nxt=cur.substr(0,i)+cur.substr(i+1);
16                    if(!vis.count(nxt)){vis.insert(nxt);q.push(nxt);}
17                }
18            }
19            if(found) break;
20        }
21        return res;
22    }
23 };

```

11.9.3 LC#336. Palindrome Pairs [Hard] [A] [G]

Problem

Find all index pairs (i, j) such that $\text{words}[i] + \text{words}[j]$ is a palindrome.

```

1 class Solution {
2     bool isPal(const string& s, int l, int r){while(l<r) if(s[l++]!=s[r--]) return false;return true;}
3 public:
4     vector<vector<int>> palindromePairs(vector<string>& words){
5         unordered_map<string, int> mp;
6         for(int i=0;i<(int)words.size();i++) mp[words[i]]=i;
7         vector<vector<int>> res;
8         for(int i=0;i<(int)words.size();i++){
9             string w=words[i]; int n=w.size();
10            for(int k=0;k<=n;k++){
11                if(isPal(w, 0, k-1)){
12                    string rev=w.substr(k); reverse(rev.begin(), rev.end());
13                    if(mp.count(rev)&&mp[rev]!=i) res.push_back({mp[rev], i});
14                }
15                if(k>0&&isPal(w, k, n-1)){
16                    string rev=w.substr(0, k); reverse(rev.begin(), rev.end());
17                    if(mp.count(rev)&&mp[rev]!=i) res.push_back({i, mp[rev]});
18                }
19            }
20        }
21        return res;
22    }
23 };

```


11.9.4 LC#753. Cracking the Safe [Hard] [G]

Problem

Find the shortest string containing every possible password of length n over k digits (De Bruijn sequence / Hierholzer's).

```

1  class Solution {
2  public:
3      string crackSafe(int n,int k){
4          string start(n,'0'); unordered_set<string> vis; string res;
5          function<void(string)> dfs=[&](string node){
6              for(int c=0;c<k;c++){
7                  string nxt=node+char('0'+c);
8                  if(!vis.count(nxt)){vis.insert(nxt);dfs(nxt.substr(1));res+=char('0'+c);}
9              }
10         };
11         vis.insert(start); dfs(start);
12         return start+res;
13     }
14 };

```

11.10 Design & Data Structures

11.10.1 LC#381. Insert Delete GetRandom O(1) – Duplicates Allowed [Hard] [A]

Problem

Design a data structure with O(1) insert/remove/getRandom supporting duplicates.

```

1  class RandomizedCollection {
2      unordered_map<int,unordered_set<int>> pos;
3      vector<int> nums;
4  public:
5      bool insert(int val){nums.push_back(val);pos[val].insert(nums.size()-1);return pos[val].size()==1;}
6      bool remove(int val){
7          if(!pos.count(val)||pos[val].empty()) return false;
8          int i=*pos[val].begin(),last=nums.back();
9          pos[last].erase(nums.size()-1);
10         if(i!=(int)nums.size()-1) pos[last].insert(i);
11         pos[val].erase(i); nums[i]=last; nums.pop_back();
12         return true;
13     }
14     int getRandom(){return nums[rand()%nums.size()];}
15 };

```

11.10.2 LC#432. All O'one Data Structure [Hard] [G]

Problem

Design a data structure with O(log n) inc/dec and O(1) getMaxKey/getMinKey.

```

1  class AllOne {
2      map<int,set<string>> c2k; unordered_map<string,int> k2c;
3  public:
4      void inc(string k){
5          int c=k2c[k]++; c2k[c].erase(k); if(c2k[c].empty()) c2k.erase(c); c2k[c+1].insert(k);
6      }
7      void dec(string k){
8          int c=k2c[k]; if(c==1) k2c.erase(k); else k2c[k]--;
9          c2k[c].erase(k); if(c2k[c].empty()) c2k.erase(c); if(c>1) c2k[c-1].insert(k);
10     }
11     string getMaxKey(){return c2k.empty()?"":*c2k.rbegin()->second.begin();}
12     string getMinKey(){return c2k.empty()?"":*c2k.begin()->second.begin();}
13 };

```

11.10.3 LC#732. My Calendar III [Hard] [G]

Problem

Return the maximum k -booking after each new event added (difference array approach).

```

1  class MyCalendarThree {
2      map<int,int> diff;
3  public:
4      int book(int s,int e){
5          diff[s]++; diff[e]--;
6          int cur=0,res=0;
7          for(auto&[t,d]:diff){cur+=d;res=max(res,cur);}
8          return res;
9      }
10 };

```

11.11 Intervals & Scheduling

11.11.1 LC#759. Employee Free Time [Hard] [G] [A]

Problem

Find all intervals when every employee is free (merge all intervals, return gaps).

```

1  // Interval: {start, end}
2  class Solution {
3  public:
4      vector<vector<int>>> employeeFreeTime(vector<vector<vector<int>>>& schedule){
5          vector<vector<int>>> all;
6          for(auto& emp:schedule) for(auto& iv:emp) all.push_back(iv);
7          sort(all.begin(),all.end());
8          vector<vector<int>>> res; auto cur=all[0];
9          for(auto& iv:all){
10             if(iv[0]<=cur[1]) cur[1]=max(cur[1],iv[1]);
11             else{res.push_back({cur[1],iv[0]});cur=iv;}
12         }
13         return res;
14     }
15 };

```

11.11.2 LC#765. Couples Holding Hands [Hard] [G]

Problem

Minimum swaps for couples to sit next to each other (Union-Find: count components - 1).

```

1  class Solution {
2  public:
3      int minSwapsCouples(vector<int>& row){
4          int n=row.size()/2; vector<int> par(n); iota(par.begin(),par.end(),0);
5          function<int(int)> find=&[(int x)->int{return par[x]==x?x:par[x]=find(par[x]);}];
6          int swaps=0;
7          for(int i=0;i<2*n;i+=2){
8              int a=row[i]/2,b=row[i+1]/2,pa=find(a),pb=find(b);
9              if(pa!=pb){par[pa]=pb;swaps++;}
10         }
11         return swaps;
12     }
13 };

```

11.11.3 LC#1499. Max Value of Equation [Hard] [A]

Problem

Maximize $y_i + y_j + |x_i - x_j|$ for pairs with $|x_i - x_j| \leq k$ (sliding window + monotonic deque).

```

1  class Solution {
2  public:
3      int findMaxValueOfEquation(vector<vector<int>>& pts,int k){
4          deque<pair<int,int>> dq; int res=INT_MIN;
5          for(auto& p:pts){
6              int x=p[0],y=p[1];
7              while(!dq.empty()&&x-dq.front().first>k) dq.pop_front();
8              if(!dq.empty()) res=max(res,y+x+dq.front().second);
9              while(!dq.empty()&&dq.back().second<=y-x) dq.pop_back();
10             dq.push_back({x,y-x});
11         }
12         return res;
13     }

```

```
14 };
```

Chapter 12

Medium Problems — Extended Set

*155+ curated LC Medium problems organized by topic.
Jump Game variants are consolidated in their own section for pattern recognition.*

12.1 Arrays & Two Pointers

12.1.1 LC#80. Remove Duplicates from Sorted Array II [\[Medium\]](#) [\[G\]](#) [\[MS\]](#)

Problem

Allow at most 2 occurrences of each element. Remove in-place, return new length.

```
1 class Solution {
2 public:
3     int removeDuplicates(vector<int>& nums) {
4         int k = 0;
5         for (int x : nums) {
6             if (k < 2 || nums[k-2] != x) nums[k++] = x;
7         }
8         return k;
9     }
10 };
```

12.1.2 LC#209. Minimum Size Subarray Sum [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Find the minimal length of a contiguous subarray whose sum $\geq$ target. Return 0 if none.

```
1 class Solution {
2 public:
3     int minSubArrayLen(int target, vector<int>& nums) {
4         int l = 0, sum = 0, res = INT_MAX;
5         for (int r = 0; r < (int)nums.size(); r++) {
6             sum += nums[r];
7             while (sum >= target) {
8                 res = min(res, r - l + 1);
9                 sum -= nums[l++];
10            }
11        }
12        return res == INT_MAX ? 0 : res;
13    }
14 };
```

12.1.3 LC#334. Increasing Triplet Subsequence [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Return true if there exist indices $i < j < k$ such that $nums[i] < nums[j] < nums[k]$. O(n) time.

```

1 class Solution {
2 public:
3     bool increasingTriplet(vector<int>& nums) {
4         int first = INT_MAX, second = INT_MAX;
5         for (int x : nums) {
6             if (x <= first) first = x;
7             else if (x <= second) second = x;
8             else return true;
9         }
10        return false;
11    }
12 };

```

12.1.4 LC#523. Continuous Subarray Sum [Medium] [A] [G]

Problem

Return true if there is a subarray of length ≥ 2 whose sum is a multiple of k .

```

1 class Solution {
2 public:
3     bool checkSubarraySum(vector<int>& nums, int k) {
4         unordered_map<int, int> mp{{0, -1}};
5         int pre = 0;
6         for (int i = 0; i < (int)nums.size(); i++) {
7             pre = (pre + nums[i]) % k;
8             if (mp.count(pre)) {
9                 if (i - mp[pre] >= 2) return true;
10            } else {
11                mp[pre] = i;
12            }
13        }
14        return false;
15    }
16 };

```

12.1.5 LC#525. Contiguous Array [Medium] [A] [MS]

Problem

Find the maximum length subarray with equal numbers of 0s and 1s.

```

1 class Solution {
2 public:
3     int findMaxLength(vector<int>& nums) {
4         unordered_map<int, int> mp{{0, -1}};
5         int sum = 0, res = 0;
6         for (int i = 0; i < (int)nums.size(); i++) {
7             sum += nums[i] == 1 ? 1 : -1;
8             if (mp.count(sum)) res = max(res, i - mp[sum]);
9             else mp[sum] = i;
10        }
11        return res;
12    }
13 };

```

12.1.6 LC#904. Fruit Into Baskets [Medium] [A] [G]

Problem

Pick fruits into 2 baskets (each basket holds 1 type). Find max fruits you can pick in a contiguous segment.

```

1 class Solution {
2 public:
3     int totalFruit(vector<int>& fruits) {
4         unordered_map<int, int> cnt;
5         int l = 0, res = 0;
6         for (int r = 0; r < (int)fruits.size(); r++) {
7             cnt[fruits[r]]++;
8             while (cnt.size() > 2) {
9                 cnt[fruits[l]]--;
10                if (cnt[fruits[l]] == 0) cnt.erase(fruits[l]);
11                l++;
12            }
13            res = max(res, r - l + 1);
14        }
15    }
16 };

```

```

15     return res;
16 }
17 };

```

12.1.7 LC#974. Subarray Sums Divisible by K [Medium] [A] [G]

Problem

Return the number of subarrays whose sum is divisible by k.

```

1 class Solution {
2 public:
3     int subarraysDivByK(vector<int>& nums, int k) {
4         vector<int> cnt(k, 0);
5         cnt[0] = 1;
6         int pre = 0, res = 0;
7         for (int x : nums) {
8             pre = ((pre + x) % k + k) % k;
9             res += cnt[pre]++;
10        }
11        return res;
12    }
13 };

```

12.1.8 LC#1004. Max Consecutive Ones III [Medium] [A] [MS]

Problem

With at most k flips from 0 to 1, return the maximum number of consecutive 1s.

```

1 class Solution {
2 public:
3     int longestOnes(vector<int>& nums, int k) {
4         int l = 0, zeros = 0, res = 0;
5         for (int r = 0; r < (int)nums.size(); r++) {
6             if (nums[r] == 0) zeros++;
7             while (zeros > k) {
8                 if (nums[l] == 0) zeros--;
9                 l++;
10            }
11            res = max(res, r - l + 1);
12        }
13        return res;
14    }
15 };

```

12.1.9 LC#1248. Count Number of Nice Subarrays [Medium] [A]

Problem

Count subarrays with exactly k odd numbers. Use prefix sum with at-most-k trick.

```

1 class Solution {
2     int atMost(vector<int>& nums, int k) {
3         int l = 0, odds = 0, res = 0;
4         for (int r = 0; r < (int)nums.size(); r++) {
5             if (nums[r] % 2 == 1) odds++;
6             while (odds > k) {
7                 if (nums[l] % 2 == 1) odds--;
8                 l++;
9             }
10            res += r - l + 1;
11        }
12        return res;
13    }
14 public:
15     int numberOfSubarrays(vector<int>& nums, int k) {
16         return atMost(nums, k) - atMost(nums, k - 1);
17     }
18 };

```

12.1.10 LC#1493. Longest Subarray of 1s After Deleting One Element [\[Medium\]](#) [\[A\]](#)

Problem

Delete exactly one element from a binary array. Return length of longest subarray of 1s remaining.

```

1 class Solution {
2 public:
3     int longestSubarray(vector<int>& nums) {
4         int l = 0, zeros = 0, res = 0;
5         for (int r = 0; r < (int)nums.size(); r++) {
6             if (nums[r] == 0) zeros++;
7             while (zeros > 1) {
8                 if (nums[l] == 0) zeros--;
9                 l++;
10            }
11            res = max(res, r - l); // subtract 1 for the deleted element
12        }
13        return res;
14    }
15 };

```

12.1.11 LC#1658. Minimum Operations to Reduce X to Zero [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Remove elements from either end summing to x. Minimize operations. Equivalent: find longest subarray with sum = total - x.

```

1 class Solution {
2 public:
3     int minOperations(vector<int>& nums, int x) {
4         int target = accumulate(nums.begin(), nums.end(), 0) - x;
5         if (target < 0) return -1;
6         if (target == 0) return nums.size();
7         int l = 0, sum = 0, res = -1;
8         for (int r = 0; r < (int)nums.size(); r++) {
9             sum += nums[r];
10            while (sum > target) sum -= nums[l++];
11            if (sum == target) res = max(res, r - l + 1);
12        }
13        return res == -1 ? -1 : (int)nums.size() - res;
14    }
15 };

```

12.1.12 LC#152. Maximum Product Subarray [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Find the contiguous subarray with the largest product.

```

1 class Solution {
2 public:
3     int maxProduct(vector<int>& nums) {
4         int maxP = nums[0], minP = nums[0], res = nums[0];
5         for (int i = 1; i < (int)nums.size(); i++) {
6             if (nums[i] < 0) swap(maxP, minP);
7             maxP = max(nums[i], maxP * nums[i]);
8             minP = min(nums[i], minP * nums[i]);
9             res = max(res, maxP);
10        }
11        return res;
12    }
13 };

```

12.1.13 LC#845. Longest Mountain in Array [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Find the length of the longest mountain subarray (strictly increases then strictly decreases).

```

1 class Solution {
2 public:
3     int longestMountain(vector<int>& arr) {

```

```

4     int n = arr.size(), res = 0;
5     for (int i = 1; i < n - 1; ) {
6         if (arr[i-1] < arr[i] && arr[i] > arr[i+1]) {
7             int l = i - 1, r = i + 1;
8             while (l > 0 && arr[l-1] < arr[l]) l--;
9             while (r < n - 1 && arr[r] > arr[r+1]) r++;
10            res = max(res, r - l + 1);
11            i = r;
12        } else {
13            i++;
14        }
15    }
16    return res;
17 }
18 };

```

12.1.14 LC#930. Binary Subarrays With Sum [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Count subarrays with sum equal to goal in a binary array.

```

1 class Solution {
2 public:
3     int numSubarraysWithSum(vector<int>& nums, int goal) {
4         unordered_map<int, int> mp{{0, 1}};
5         int pre = 0, res = 0;
6         for (int x : nums) {
7             pre += x;
8             res += mp[pre - goal];
9             mp[pre]++;
10        }
11        return res;
12    }
13 };

```

12.1.15 LC#1574. Shortest Subarray to be Removed to Make Array Sorted [\[Medium\]](#) [\[MS\]](#)

Problem

Remove the shortest subarray to make the remaining elements non-decreasing.

```

1 class Solution {
2 public:
3     int findLengthOfShortestSubarray(vector<int>& arr) {
4         int n = arr.size();
5         int r = n - 1;
6         while (r > 0 && arr[r-1] <= arr[r]) r--;
7         int res = r; // remove prefix [0, r-1]
8         for (int l = 0; l < n && (l == 0 || arr[l-1] <= arr[l]); l++) {
9             while (r < n && arr[r] < arr[l]) r++;
10            res = min(res, r - l - 1);
11        }
12        return res;
13    }
14 };

```

12.1.16 LC#2401. Longest Nice Subarray [\[Medium\]](#) [\[A\]](#)

Problem

A “nice” subarray has no two elements sharing a set bit. Find the longest such subarray.

```

1 class Solution {
2 public:
3     int longestNiceSubarray(vector<int>& nums) {
4         int l = 0, used = 0, res = 0;
5         for (int r = 0; r < (int)nums.size(); r++) {
6             while (used & nums[r]) {
7                 used ^= nums[l];
8                 l++;
9             }
10            used |= nums[r];
11            res = max(res, r - l + 1);
12        }
13    }
14 };

```



```

13     return res;
14 }
15 };

```

12.1.17 LC#498. Diagonal Traverse [Medium] [A] [MS]

Problem

Return all elements of an $m \times n$ matrix in diagonal traversal order.

```

1 class Solution {
2 public:
3     vector<int> findDiagonalOrder(vector<vector<int>>& mat) {
4         int m = mat.size(), n = mat[0].size();
5         vector<int> res;
6         for (int d = 0; d < m + n - 1; d++) {
7             if (d % 2 == 0) {
8                 int r = min(d, m - 1), c = d - r;
9                 while (r >= 0 && c < n) res.push_back(mat[r--][c++]);
10            } else {
11                int c = min(d, n - 1), r = d - c;
12                while (c >= 0 && r < m) res.push_back(mat[r++][c--]);
13            }
14        }
15        return res;
16    }
17 };

```

12.1.18 LC#667. Beautiful Arrangement II [Medium] [A]

Problem

Construct an array of n integers where values are $[1, n]$ and there are exactly k distinct absolute differences between adjacent elements.

```

1 class Solution {
2 public:
3     vector<int> constructArray(int n, int k) {
4         vector<int> res;
5         // first k+1 elements: 1, k+1, 2, k, 3, k-1, ... (gives k distinct diffs)
6         int lo = 1, hi = k + 1;
7         for (int i = 0; i < k + 1; i++) {
8             if (i % 2 == 0) res.push_back(lo++);
9             else res.push_back(hi--);
10        }
11        // remaining elements: k+2, k+3, ... n (all diff = 1)
12        for (int i = k + 2; i <= n; i++) res.push_back(i);
13        return res;
14    }
15 };

```

12.2 Binary Search

12.2.1 LC#34. Find First and Last Position of Element [Medium] [A] [MS] [G]

Problem

Find starting and ending position of a target in a sorted array in $O(\log n)$.

```

1 class Solution {
2     int lowerBound(vector<int>& nums, int target) {
3         int l = 0, r = nums.size();
4         while (l < r) {
5             int m = (l + r) / 2;
6             if (nums[m] < target) l = m + 1;
7             else r = m;
8         }
9         return l;
10    }
11 public:
12     vector<int> searchRange(vector<int>& nums, int target) {
13         int lo = lowerBound(nums, target);
14         if (lo == (int)nums.size() || nums[lo] != target) return {-1, -1};
15         return {lo, lowerBound(nums, target + 1) - 1};
16    }
17 };

```

```

16     }
17 };

```

12.2.2 LC#658. Find K Closest Elements [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Find k closest integers to x in sorted array. Return sorted. Ties: prefer smaller element.

```

1  class Solution {
2  public:
3      vector<int> findClosestElements(vector<int>& arr, int k, int x) {
4          int l = 0, r = (int)arr.size() - k;
5          while (l < r) {
6              int m = (l + r) / 2;
7              // compare distance from left vs right of window
8              if (x - arr[m] > arr[m + k] - x) l = m + 1;
9              else r = m;
10         }
11         return vector<int>(arr.begin() + l, arr.begin() + l + k);
12     }
13 };

```

12.2.3 LC#1283. Find the Smallest Divisor Given a Threshold [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Find the smallest divisor such that the sum of ceiling divisions of all elements $\leq$ threshold.

```

1  class Solution {
2  public:
3      int smallestDivisor(vector<int>& nums, int threshold) {
4          int l = 1, r = *max_element(nums.begin(), nums.end());
5          while (l < r) {
6              int m = (l + r) / 2;
7              long long sum = 0;
8              for (int x : nums) sum += (x + m - 1) / m;
9              if (sum <= threshold) r = m;
10             else l = m + 1;
11         }
12         return l;
13     }
14 };

```

12.2.4 LC#1482. Minimum Number of Days to Make m Bouquets [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Given bloom days, find minimum days to make m bouquets of k adjacent bloomed flowers.

```

1  class Solution {
2  public:
3      int minDays(vector<int>& bloomDay, int m, int k) {
4          if ((long long)m * k > bloomDay.size()) return -1;
5          int l = 1, r = *max_element(bloomDay.begin(), bloomDay.end());
6          auto canMake = [&](int day) {
7              int bouquets = 0, consecutive = 0;
8              for (int d : bloomDay) {
9                  if (d <= day) {
10                     consecutive++;
11                     if (consecutive == k) {
12                         bouquets++;
13                         consecutive = 0;
14                     }
15                 } else {
16                     consecutive = 0;
17                 }
18             }
19             return bouquets >= m;
20         };
21         while (l < r) {
22             int mid = (l + r) / 2;
23             if (canMake(mid)) r = mid;
24             else l = mid + 1;
25         }
26         return l;
27     }

```

```
28 };
```

12.2.5 LC#1552. Magnetic Force Between Two Balls [Medium] [A] [G]

Problem

Place m balls in positions to maximize the minimum magnetic force (distance) between any two balls.

```
1 class Solution {
2 public:
3     int maxDistance(vector<int>& position, int m) {
4         sort(position.begin(), position.end());
5         int l = 1, r = position.back() - position.front();
6         auto canPlace = [&](int minDist) {
7             int cnt = 1, last = position[0];
8             for (int i = 1; i < (int)position.size(); i++) {
9                 if (position[i] - last >= minDist) {
10                     cnt++;
11                     last = position[i];
12                 }
13             }
14             return cnt >= m;
15         };
16         while (l < r) {
17             int mid = (l + r + 1) / 2;
18             if (canPlace(mid)) l = mid;
19             else r = mid - 1;
20         }
21         return l;
22     }
23 };
```

12.2.6 LC#1760. Minimum Limit of Balls in a Bag [Medium] [A]

Problem

Split bags of balls with at most `maxOperations` splits. Minimize the maximum bag size.

```
1 class Solution {
2 public:
3     int minimumSize(vector<int>& nums, int maxOps) {
4         int l = 1, r = *max_element(nums.begin(), nums.end());
5         while (l < r) {
6             int mid = (l + r) / 2;
7             long long ops = 0;
8             for (int x : nums) ops += (x - 1) / mid;
9             if (ops <= maxOps) r = mid;
10            else l = mid + 1;
11        }
12        return l;
13    }
14 };
```

12.2.7 LC#275. H-Index II [Medium] [A] [G]

Problem

Sorted citations array. Find H-Index: largest h such that at least h papers have $\geq h$ citations.

```
1 class Solution {
2 public:
3     int hIndex(vector<int>& citations) {
4         int n = citations.size(), l = 0, r = n;
5         while (l < r) {
6             int m = (l + r + 1) / 2;
7             if (citations[n - m] >= m) l = m;
8             else r = m - 1;
9         }
10        return l;
11    }
12 };
```

12.2.8 LC#2300. Successful Pairs of Spells and Potions [Medium] [A]

Problem

For each spell, count potions such that $\text{spell} * \text{potion} \geq \text{success}$. Return counts.

```

1  class Solution {
2  public:
3      vector<int> successfulPairs(vector<int>& spells, vector<int>& potions, long long success) {
4          sort(potions.begin(), potions.end());
5          int m = potions.size();
6          vector<int> res;
7          for (int s : spells) {
8              long long need = (success + s - 1) / s;
9              int idx = lower_bound(potions.begin(), potions.end(), (int)need) - potions.begin();
10             res.push_back(m - idx);
11         }
12         return res;
13     }
14 };

```

12.2.9 LC#2226. Maximum Candies Allocated to K Children [Medium] [A]

Problem

Split candy piles so each of k children gets the same maximum number of candies.

```

1  class Solution {
2  public:
3      int maximumCandies(vector<int>& candies, long long k) {
4          long long total = accumulate(candies.begin(), candies.end(), 0LL);
5          if (total < k) return 0;
6          int l = 1, r = *max_element(candies.begin(), candies.end());
7          while (l < r) {
8              int mid = (long long)(l + r + 1) / 2;
9              long long cnt = 0;
10             for (int c : candies) cnt += c / mid;
11             if (cnt >= k) l = mid;
12             else r = mid - 1;
13         }
14         return l;
15     }
16 };

```

12.2.10 LC#2064. Minimized Maximum of Products Distributed [Medium] [G]

Problem

Distribute products to shops. Minimize the max number any shop carries (binary search on the answer).

```

1  class Solution {
2  public:
3      int minimizedMaximum(int n, vector<int>& quantities) {
4          int l = 1, r = *max_element(quantities.begin(), quantities.end());
5          while (l < r) {
6              int m = (l + r) / 2;
7              long long shops = 0;
8              for (int q : quantities) shops += (q + m - 1) / m;
9              if (shops <= n) r = m;
10             else l = m + 1;
11         }
12         return l;
13     }
14 };

```

12.3 Strings

12.3.1 LC#6. Zigzag Conversion [Medium] [A] [MS]

Problem

Write the string in zigzag across `numRows` rows, then read row by row.

```

1 class Solution {
2 public:
3     string convert(string s, int numRows) {
4         if (numRows == 1) return s;
5         vector<string> rows(numRows);
6         int row = 0, dir = -1;
7         for (char c : s) {
8             rows[row] += c;
9             if (row == 0 || row == numRows - 1) dir = -dir;
10            row += dir;
11        }
12        string res;
13        for (auto& r : rows) res += r;
14        return res;
15    }
16 };

```

12.3.2 LC#12. Integer to Roman [Medium] [A] [MS]

Problem

Convert an integer (1-3999) to its Roman numeral representation.

```

1 class Solution {
2 public:
3     string intToRoman(int num) {
4         vector<pair<int, string>> table = {
5             {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
6             {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
7             {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"}, {1, "I"}
8         };
9         string res;
10        for (auto& [val, sym] : table) {
11            while (num >= val) {
12                res += sym;
13                num -= val;
14            }
15        }
16        return res;
17    }
18 };

```

12.3.3 LC#38. Count and Say [Medium] [A]

Problem

Generate the nth term of the look-and-say sequence (count groups of consecutive identical digits).

```

1 class Solution {
2 public:
3     string countAndSay(int n) {
4         string cur = "1";
5         for (int i = 1; i < n; i++) {
6             string next;
7             int j = 0;
8             while (j < (int)cur.size()) {
9                 char c = cur[j];
10                int cnt = 0;
11                while (j < (int)cur.size() && cur[j] == c) {
12                    j++;
13                    cnt++;
14                }
15                next += to_string(cnt) + c;
16            }
17            cur = next;
18        }
19        return cur;
20    }
21 };

```

12.3.4 LC#43. Multiply Strings [Medium] [A] [MS]

Problem

Multiply two non-negative integers represented as strings. Result is also a string.

```

1  class Solution {
2  public:
3      string multiply(string num1, string num2) {
4          int m = num1.size(), n = num2.size();
5          vector<int> pos(m + n, 0);
6          for (int i = m - 1; i >= 0; i--) {
7              for (int j = n - 1; j >= 0; j--) {
8                  int mul = (num1[i] - '0') * (num2[j] - '0');
9                  int p1 = i + j, p2 = i + j + 1;
10                 int sum = mul + pos[p2];
11                 pos[p2] = sum % 10;
12                 pos[p1] += sum / 10;
13             }
14         }
15         string res;
16         for (int x : pos) if (!(res.empty() && x == 0)) res += to_string(x);
17         return res.empty() ? "0" : res;
18     }
19 };

```

12.3.5 LC#150. Evaluate Reverse Polish Notation [Medium] [A] [MS]

Problem

Evaluate an expression in Reverse Polish Notation (postfix).

```

1  class Solution {
2  public:
3      int evalRPN(vector<string>& tokens) {
4          stack<long long> st;
5          for (auto& t : tokens) {
6              if (t == "+" || t == "-" || t == "*" || t == "/") {
7                  long long b = st.top(); st.pop();
8                  long long a = st.top(); st.pop();
9                  if (t == "+") st.push(a + b);
10                 else if (t == "-") st.push(a - b);
11                 else if (t == "*") st.push(a * b);
12                 else st.push(a / b);
13             } else {
14                 st.push(stoll(t));
15             }
16         }
17         return st.top();
18     }
19 };

```

12.3.6 LC#165. Compare Version Numbers [Medium] [A] [MS]

Problem

Compare two version strings by their integer revisions split on “.”

```

1  class Solution {
2  public:
3      int compareVersion(string v1, string v2) {
4          int i = 0, j = 0;
5          while (i < (int)v1.size() || j < (int)v2.size()) {
6              int n1 = 0, n2 = 0;
7              while (i < (int)v1.size() && v1[i] != '.') n1 = n1 * 10 + (v1[i++] - '0');
8              while (j < (int)v2.size() && v2[j] != '.') n2 = n2 * 10 + (v2[j++] - '0');
9              if (n1 > n2) return 1;
10             if (n1 < n2) return -1;
11             i++;
12             j++;
13         }
14         return 0;
15     }
16 };

```

12.3.7 LC#394. Decode String [Medium] [A] [MS] [G]

Problem

Decode a string like 3[a2[c]] → accaccacc.

```

1 class Solution {
2 public:
3     string decodeString(string s) {
4         stack<int> counts;
5         stack<string> strs;
6         string cur;
7         int k = 0;
8         for (char c : s) {
9             if (isdigit(c)) {
10                 k = k * 10 + (c - '0');
11             } else if (c == '[') {
12                 counts.push(k);
13                 strs.push(cur);
14                 cur = "";
15                 k = 0;
16             } else if (c == ']') {
17                 int repeat = counts.top(); counts.pop();
18                 string prev = strs.top(); strs.pop();
19                 for (int i = 0; i < repeat; i++) prev += cur;
20                 cur = prev;
21             } else {
22                 cur += c;
23             }
24         }
25         return cur;
26     }
27 };

```

12.3.8 LC#539. Minimum Time Difference [Medium] [A]

Problem

Given a list of 24-hour clock times, find the minimum difference in minutes between any two.

```

1 class Solution {
2 public:
3     int findMinDifference(vector<string>& timePoints) {
4         vector<int> mins;
5         for (auto& t : timePoints) {
6             int h = stoi(t.substr(0, 2));
7             int m = stoi(t.substr(3, 2));
8             mins.push_back(h * 60 + m);
9         }
10        sort(mins.begin(), mins.end());
11        int res = INT_MAX;
12        for (int i = 1; i < (int)mins.size(); i++) {
13            res = min(res, mins[i] - mins[i-1]);
14        }
15        // wrap-around case
16        res = min(res, 1440 - mins.back() + mins.front());
17        return res;
18    }
19 };

```

12.3.9 LC#647. Palindromic Substrings [Medium] [A] [MS] [G]

Problem

Count the number of palindromic substrings in *s*.

```

1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int res = 0;
5         for (int i = 0; i < (int)s.size(); i++) {
6             // odd length
7             for (int l = i, r = i; l >= 0 && r < (int)s.size() && s[l] == s[r]; l--, r++) res++;
8             // even length
9             for (int l = i, r = i + 1; l >= 0 && r < (int)s.size() && s[l] == s[r]; l--, r++) res++;
10        }
11        return res;
12    }
13 };

```

12.3.10 LC#763. Partition Labels [Medium] [A] [MS] [G]

Problem

Partition string into as many parts as possible so each letter appears in at most one part.

```

1 class Solution {
2 public:
3     vector<int> partitionLabels(string s) {
4         int last[26] = {};
5         for (int i = 0; i < (int)s.size(); i++) last[s[i] - 'a'] = i;
6         vector<int> res;
7         int start = 0, end = 0;
8         for (int i = 0; i < (int)s.size(); i++) {
9             end = max(end, last[s[i] - 'a']);
10            if (i == end) {
11                res.push_back(end - start + 1);
12                start = i + 1;
13            }
14        }
15        return res;
16    }
17 };

```

12.3.11 LC#1047. Remove All Adjacent Duplicates in String [Medium] [A]

Problem

Repeatedly remove adjacent duplicate characters. Return the final string.

```

1 class Solution {
2 public:
3     string removeDuplicates(string s) {
4         string stk;
5         for (char c : s) {
6             if (!stk.empty() && stk.back() == c) stk.pop_back();
7             else stk.push_back(c);
8         }
9         return stk;
10    }
11 };

```

12.3.12 LC#1209. Remove All Adjacent Duplicates in String II [Medium] [A] [MS]

Problem

Remove k adjacent identical characters repeatedly until no more such groups exist.

```

1 class Solution {
2 public:
3     string removeDuplicates(string s, int k) {
4         stack<pair<char, int>> st; // {char, count}
5         for (char c : s) {
6             if (!st.empty() && st.top().first == c) {
7                 st.top().second++;
8                 if (st.top().second == k) st.pop();
9             } else {
10                st.push({c, 1});
11            }
12        }
13        string res;
14        while (!st.empty()) {
15            auto [c, cnt] = st.top();
16            st.pop();
17            res += string(cnt, c);
18        }
19        reverse(res.begin(), res.end());
20        return res;
21    }
22 };

```

12.4 Linked List

12.4.1 LC#2. Add Two Numbers [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Add two non-negative integers represented as reversed linked lists. Return result as reversed linked list.

```

1 class Solution {
2 public:
3     ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
4         ListNode dummy;
5         ListNode* cur = &dummy;
6         int carry = 0;
7         while (l1 || l2 || carry) {
8             int sum = carry;
9             if (l1) { sum += l1->val; l1 = l1->next; }
10            if (l2) { sum += l2->val; l2 = l2->next; }
11            carry = sum / 10;
12            cur->next = new ListNode(sum % 10);
13            cur = cur->next;
14        }
15        return dummy.next;
16    }
17 };

```

12.4.2 LC#24. Swap Nodes in Pairs [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Swap every two adjacent nodes in a linked list.

```

1 class Solution {
2 public:
3     ListNode* swapPairs(ListNode* head) {
4         ListNode dummy(0, head);
5         ListNode* pre = &dummy;
6         while (pre->next && pre->next->next) {
7             ListNode* a = pre->next;
8             ListNode* b = pre->next->next;
9             a->next = b->next;
10            b->next = a;
11            pre->next = b;
12            pre = a;
13        }
14        return dummy.next;
15    }
16 };

```

12.4.3 LC#61. Rotate List [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Rotate a linked list to the right by k places.

```

1 class Solution {
2 public:
3     ListNode* rotateRight(ListNode* head, int k) {
4         if (!head || !head->next || k == 0) return head;
5         int len = 1;
6         ListNode* tail = head;
7         while (tail->next) { tail = tail->next; len++; }
8         tail->next = head; // make circular
9         k = k % len;
10        int steps = len - k;
11        ListNode* newTail = head;
12        for (int i = 1; i < steps; i++) newTail = newTail->next;
13        ListNode* newHead = newTail->next;
14        newTail->next = nullptr;
15        return newHead;
16    }
17 };

```

12.4.4 LC#82. Remove Duplicates from Sorted List II [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Delete all nodes that have duplicate values, leaving only distinct values.

```

1 class Solution {
2 public:
3     ListNode* deleteDuplicates(ListNode* head) {
4         ListNode dummy(0, head);
5         ListNode* pre = &dummy;
6         while (pre->next) {
7             ListNode* cur = pre->next;
8             if (cur->next && cur->val == cur->next->val) {
9                 int val = cur->val;
10                while (pre->next && pre->next->val == val) pre->next = pre->next->next;
11            } else {
12                pre = pre->next;
13            }
14        }
15        return dummy.next;
16    }
17 };

```

12.4.5 LC#86. Partition List [Medium] [A] [MS]

Problem

Partition a linked list around value x : nodes $< x$ come before nodes $\geq x$, preserving order.

```

1 class Solution {
2 public:
3     ListNode* partition(ListNode* head, int x) {
4         ListNode lessHead, greaterHead;
5         ListNode* less = &lessHead;
6         ListNode* greater = &greaterHead;
7         while (head) {
8             if (head->val < x) { less->next = head; less = less->next; }
9             else { greater->next = head; greater = greater->next; }
10            head = head->next;
11        }
12        greater->next = nullptr;
13        less->next = greaterHead.next;
14        return lessHead.next;
15    }
16 };

```

12.4.6 LC#328. Odd Even Linked List [Medium] [A] [MS] [G]

Problem

Group all odd-indexed nodes first, then all even-indexed nodes. $O(1)$ space.

```

1 class Solution {
2 public:
3     ListNode* oddEvenList(ListNode* head) {
4         if (!head) return head;
5         ListNode* odd = head;
6         ListNode* even = head->next;
7         ListNode* evenHead = even;
8         while (even && even->next) {
9             odd->next = even->next;
10            odd = odd->next;
11            even->next = odd->next;
12            even = even->next;
13        }
14        odd->next = evenHead;
15        return head;
16    }
17 };

```

12.4.7 LC#445. Add Two Numbers II [Medium] [A] [MS]

Problem

Add two numbers represented as non-reversed linked lists. Return result (non-reversed) without reversing input.

```

1 class Solution {
2 public:
3     ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {

```

```

4     stack<int> s1, s2;
5     while (l1) { s1.push(l1->val); l1 = l1->next; }
6     while (l2) { s2.push(l2->val); l2 = l2->next; }
7     int carry = 0;
8     ListNode* head = nullptr;
9     while (!s1.empty() || !s2.empty() || carry) {
10        int sum = carry;
11        if (!s1.empty()) { sum += s1.top(); s1.pop(); }
12        if (!s2.empty()) { sum += s2.top(); s2.pop(); }
13        carry = sum / 10;
14        ListNode* node = new ListNode(sum % 10);
15        node->next = head;
16        head = node;
17    }
18    return head;
19 }
20 };

```

12.4.8 LC#876. Middle of the Linked List [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Return the middle node of a linked list. If two middles, return the second one.

```

1 class Solution {
2 public:
3     ListNode* middleNode(ListNode* head) {
4         ListNode* slow = head;
5         ListNode* fast = head;
6         while (fast && fast->next) {
7             slow = slow->next;
8             fast = fast->next->next;
9         }
10        return slow;
11    }
12 };

```

12.5 Trees & Binary Search Trees

12.5.1 LC#116. Populating Next Right Pointers [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Connect each node to its next right node in a perfect binary tree.

```

1 class Solution {
2 public:
3     Node* connect(Node* root) {
4         if (!root) return root;
5         Node* leftmost = root;
6         while (leftmost->left) {
7             Node* cur = leftmost;
8             while (cur) {
9                 cur->left->next = cur->right;
10                if (cur->next) cur->right->next = cur->next->left;
11                cur = cur->next;
12            }
13            leftmost = leftmost->left;
14        }
15        return root;
16    }
17 };

```

12.5.2 LC#129. Sum Root to Leaf Numbers [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Sum all root-to-leaf numbers (each path forms a number by concatenating digits).

```

1 class Solution {
2     int dfs(TreeNode* node, int cur) {
3         if (!node) return 0;
4         cur = cur * 10 + node->val;
5         if (!node->left && !node->right) return cur;
6         return dfs(node->left, cur) + dfs(node->right, cur);

```

```

7     }
8 public:
9     int sumNumbers(TreeNode* root) {
10         return dfs(root, 0);
11     }
12 };

```

12.5.3 LC#173. Binary Search Tree Iterator [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Implement an iterator over BST in-order using $O(h)$ space.

```

1 class BSTIterator {
2     stack<TreeNode*> st;
3
4     void pushLeft(TreeNode* node) {
5         while (node) {
6             st.push(node);
7             node = node->left;
8         }
9     }
10 public:
11     BSTIterator(TreeNode* root) { pushLeft(root); }
12
13     int next() {
14         TreeNode* node = st.top();
15         st.pop();
16         pushLeft(node->right);
17         return node->val;
18     }
19
20     bool hasNext() { return !st.empty(); }
21 };

```

12.5.4 LC#222. Count Complete Tree Nodes [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Count nodes in a complete binary tree in $O(\log^2 n)$ time.

```

1 class Solution {
2     int height(TreeNode* node) {
3         int h = 0;
4         while (node) { h++; node = node->left; }
5         return h;
6     }
7 public:
8     int countNodes(TreeNode* root) {
9         if (!root) return 0;
10        int lh = height(root->left);
11        int rh = height(root->right);
12        if (lh == rh) {
13            // left subtree is perfect
14            return (1 << lh) + countNodes(root->right);
15        } else {
16            // right subtree is perfect (one level shorter)
17            return (1 << rh) + countNodes(root->left);
18        }
19    }
20 };

```

12.5.5 LC#236. Lowest Common Ancestor of a Binary Tree [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#) [\[Meta\]](#)

Problem

Find the LCA of two nodes in a binary tree (not necessarily a BST).

```

1 class Solution {
2 public:
3     TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
4         if (!root || root == p || root == q) return root;
5         TreeNode* left = lowestCommonAncestor(root->left, p, q);
6         TreeNode* right = lowestCommonAncestor(root->right, p, q);

```

```

7         if (left && right) return root;
8         return left ? left : right;
9     }
10 };

```

12.5.6 LC#450. Delete Node in a BST [Medium] [A] [MS]

Problem

Delete a node with given key from a BST. Return the root.

```

1 class Solution {
2     TreeNode* successor(TreeNode* node) {
3         node = node->right;
4         while (node->left) node = node->left;
5         return node;
6     }
7 public:
8     TreeNode* deleteNode(TreeNode* root, int key) {
9         if (!root) return nullptr;
10        if (key < root->val) {
11            root->left = deleteNode(root->left, key);
12        } else if (key > root->val) {
13            root->right = deleteNode(root->right, key);
14        } else {
15            if (!root->left) return root->right;
16            if (!root->right) return root->left;
17            TreeNode* succ = successor(root);
18            root->val = succ->val;
19            root->right = deleteNode(root->right, succ->val);
20        }
21        return root;
22    }
23 };

```

12.5.7 LC#538. Convert BST to Greater Tree [Medium] [A] [MS]

Problem

Convert a BST so that each key becomes the sum of all keys greater than or equal to the original key. Reverse in-order traversal.

```

1 class Solution {
2     int suffix = 0;
3     void dfs(TreeNode* node) {
4         if (!node) return;
5         dfs(node->right);
6         suffix += node->val;
7         node->val = suffix;
8         dfs(node->left);
9     }
10 public:
11     TreeNode* convertBST(TreeNode* root) {
12         dfs(root);
13         return root;
14     }
15 };

```

12.5.8 LC#669. Trim a Binary Search Tree [Medium] [A] [MS]

Problem

Trim a BST so all values fall in [low, high].

```

1 class Solution {
2 public:
3     TreeNode* trimBST(TreeNode* root, int low, int high) {
4         if (!root) return nullptr;
5         if (root->val < low) return trimBST(root->right, low, high);
6         if (root->val > high) return trimBST(root->left, low, high);
7         root->left = trimBST(root->left, low, high);
8         root->right = trimBST(root->right, low, high);
9         return root;
10    }
11 };

```

12.5.9 LC#701. Insert into a Binary Search Tree [Medium] [A] [MS]

Problem

Insert a value into a BST and return the root.

```

1 class Solution {
2 public:
3     TreeNode* insertIntoBST(TreeNode* root, int val) {
4         if (!root) return new TreeNode(val);
5         if (val < root->val) root->left = insertIntoBST(root->left, val);
6         else root->right = insertIntoBST(root->right, val);
7         return root;
8     }
9 };

```

12.5.10 LC#863. All Nodes Distance K in Binary Tree [Medium] [A] [MS] [G]

Problem

Find all nodes at distance k from target node. Build parent map + BFS.

```

1 class Solution {
2     unordered_map<TreeNode*, TreeNode*> parent;
3
4     void buildParent(TreeNode* node, TreeNode* par) {
5         if (!node) return;
6         parent[node] = par;
7         buildParent(node->left, node);
8         buildParent(node->right, node);
9     }
10 public:
11     vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
12         buildParent(root, nullptr);
13         queue<TreeNode*> q;
14         unordered_set<TreeNode*> visited;
15         q.push(target);
16         visited.insert(target);
17         int dist = 0;
18         while (!q.empty() && dist < k) {
19             int sz = q.size();
20             while (sz-- > 0) {
21                 TreeNode* node = q.front(); q.pop();
22                 for (TreeNode* nb : {node->left, node->right, parent[node]}) {
23                     if (nb && !visited.count(nb)) {
24                         visited.insert(nb);
25                         q.push(nb);
26                     }
27                 }
28             }
29             dist++;
30         }
31         vector<int> res;
32         while (!q.empty()) { res.push_back(q.front()->val); q.pop(); }
33         return res;
34     }
35 };

```

12.5.11 LC#958. Check Completeness of Binary Tree [Medium] [A]

Problem

Check if a binary tree is a complete binary tree (all levels full except possibly last, filled left to right).

```

1 class Solution {
2 public:
3     bool isCompleteTree(TreeNode* root) {
4         queue<TreeNode*> q;
5         q.push(root);
6         bool end = false;
7         while (!q.empty()) {
8             TreeNode* node = q.front(); q.pop();
9             if (!node) {
10                 end = true;
11             } else {
12                 if (end) return false;
13                 q.push(node->left);
14                 q.push(node->right);
15             }
16         }
17     }
18 };

```

```

16     }
17     return true;
18 }
19 };

```

12.5.12 LC#1161. Maximum Level Sum of a Binary Tree [Medium] [A]

Problem

Return the smallest level number (1-indexed) with the maximum sum of node values.

```

1 class Solution {
2 public:
3     int maxLevelSum(TreeNode* root) {
4         queue<TreeNode*> q;
5         q.push(root);
6         int res = 1, maxSum = INT_MIN, level = 0;
7         while (!q.empty()) {
8             int sz = q.size(), sum = 0;
9             level++;
10            while (sz-->0) {
11                TreeNode* node = q.front(); q.pop();
12                sum += node->val;
13                if (node->left) q.push(node->left);
14                if (node->right) q.push(node->right);
15            }
16            if (sum > maxSum) { maxSum = sum; res = level; }
17        }
18        return res;
19    }
20 };

```

12.5.13 LC#1325. Delete Leaves With a Given Value [Medium] [A]

Problem

Repeatedly delete all leaf nodes equal to `target` until no such leaves remain.

```

1 class Solution {
2 public:
3     TreeNode* removeLeafNodes(TreeNode* root, int target) {
4         if (!root) return nullptr;
5         root->left = removeLeafNodes(root->left, target);
6         root->right = removeLeafNodes(root->right, target);
7         // delete if now a leaf with target value
8         if (!root->left && !root->right && root->val == target) return nullptr;
9         return root;
10    }
11 };

```

12.5.14 LC#889. Construct Binary Tree from Preorder and Postorder [Medium] [G]

Problem

Construct a binary tree from preorder and postorder traversal arrays (not unique, return any valid tree).

```

1 class Solution {
2 public:
3     TreeNode* constructFromPrePost(vector<int>& pre, vector<int>& post) {
4         unordered_map<int, int> postIdx;
5         for (int i = 0; i < (int)post.size(); i++) postIdx[post[i]] = i;
6         function<TreeNode*(int, int, int, int)> build = [&](int pl, int pr, int qol, int qor) -> TreeNode* {
7             if (pl > pr) return nullptr;
8             TreeNode* root = new TreeNode(pre[pl]);
9             if (pl == pr) return root;
10            int leftRoot = pre[pl + 1];
11            int leftSize = postIdx[leftRoot] - qol + 1;
12            root->left = build(pl + 1, pl + leftSize, qol, qol + leftSize - 1);
13            root->right = build(pl + leftSize + 1, pr, qol + leftSize, qor - 1);
14            return root;
15        };
16        return build(0, pre.size() - 1, 0, post.size() - 1);
17    }
18 };

```

12.6 Graphs

12.6.1 LC#130. Surrounded Regions [Medium] [A] [MS] [G]

Problem

Flip all surrounded 0 regions to X. Cells connected to the border are not surrounded.

```

1 class Solution {
2     void dfs(vector<vector<char>>& b, int i, int j) {
3         if (i < 0 || i >= (int)b.size() || j < 0 || j >= (int)b[0].size()) return;
4         if (b[i][j] != '0') return;
5         b[i][j] = 'S';
6         dfs(b, i+1, j); dfs(b, i-1, j);
7         dfs(b, i, j+1); dfs(b, i, j-1);
8     }
9 public:
10    void solve(vector<vector<char>>& board) {
11        int m = board.size(), n = board[0].size();
12        for (int i = 0; i < m; i++) { dfs(board, i, 0); dfs(board, i, n-1); }
13        for (int j = 0; j < n; j++) { dfs(board, 0, j); dfs(board, m-1, j); }
14        for (int i = 0; i < m; i++) {
15            for (int j = 0; j < n; j++) {
16                if (board[i][j] == '0') board[i][j] = 'X';
17                else if (board[i][j] == 'S') board[i][j] = '0';
18            }
19        }
20    }
21};

```

12.6.2 LC#261. Graph Valid Tree [Medium] [G] [A]

Problem

Given n nodes and edges, determine if they form a valid tree (connected, no cycle). Use Union-Find.

```

1 class Solution {
2     vector<int> par;
3     int find(int x) { return par[x] == x ? x : par[x] = find(par[x]); }
4 public:
5     bool validTree(int n, vector<vector<int>>& edges) {
6         if ((int)edges.size() != n - 1) return false;
7         par.resize(n);
8         iota(par.begin(), par.end(), 0);
9         for (auto& e : edges) {
10             int a = find(e[0]), b = find(e[1]);
11             if (a == b) return false;
12             par[a] = b;
13         }
14         return true;
15     }
16 };

```

12.6.3 LC#286. Walls and Gates [Medium] [A] [G] [MS]

Problem

Fill each empty room with distance to nearest gate (multi-source BFS from all gates simultaneously).

```

1 class Solution {
2 public:
3     void wallsAndGates(vector<vector<int>>& rooms) {
4         int m = rooms.size(), n = rooms[0].size();
5         queue<pair<int, int>> q;
6         for (int i = 0; i < m; i++)
7             for (int j = 0; j < n; j++)
8                 if (rooms[i][j] == 0) q.push({i, j});
9         int dx[] = {0, 0, 1, -1};
10        int dy[] = {1, -1, 0, 0};
11        while (!q.empty()) {
12            auto [i, j] = q.front();
13            q.pop();
14            for (int d = 0; d < 4; d++) {
15                int ni = i + dx[d], nj = j + dy[d];
16                if (ni >= 0 && ni < m && nj >= 0 && nj < n && rooms[ni][nj] == INT_MAX) {
17                    rooms[ni][nj] = rooms[i][j] + 1;
18                    q.push({ni, nj});
19                }
20            }
21        }
22    }
23 };

```



```

19         }
20     }
21 }
22 }
23 };

```

12.6.4 LC#547. Number of Provinces [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Count the number of provinces (connected components) in a friendship matrix.

```

1 class Solution {
2     void dfs(vector<vector<int>>& isConnected, vector<bool>& vis, int i) {
3         vis[i] = true;
4         for (int j = 0; j < (int)isConnected.size(); j++) {
5             if (isConnected[i][j] == 1 && !vis[j]) dfs(isConnected, vis, j);
6         }
7     }
8 public:
9     int findCircleNum(vector<vector<int>>& isConnected) {
10        int n = isConnected.size(), cnt = 0;
11        vector<bool> vis(n, false);
12        for (int i = 0; i < n; i++) {
13            if (!vis[i]) { dfs(isConnected, vis, i); cnt++; }
14        }
15        return cnt;
16    }
17 };

```

12.6.5 LC#785. Is Graph Bipartite? [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Determine if a graph is bipartite (2-colorable) using BFS.

```

1 class Solution {
2 public:
3     bool isBipartite(vector<vector<int>>& graph) {
4         int n = graph.size();
5         vector<int> color(n, -1);
6         for (int i = 0; i < n; i++) {
7             if (color[i] != -1) continue;
8             queue<int> q;
9             q.push(i);
10            color[i] = 0;
11            while (!q.empty()) {
12                int u = q.front(); q.pop();
13                for (int v : graph[u]) {
14                    if (color[v] == -1) {
15                        color[v] = 1 - color[u];
16                        q.push(v);
17                    } else if (color[v] == color[u]) {
18                        return false;
19                    }
20                }
21            }
22        }
23        return true;
24    }
25 };

```

12.6.6 LC#797. All Paths From Source to Target [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Find all paths from node 0 to node n-1 in a DAG using DFS backtracking.

```

1 class Solution {
2     void dfs(vector<vector<int>>& g, int node, vector<int>& path, vector<vector<int>>& res) {
3         if (node == (int)g.size() - 1) { res.push_back(path); return; }
4         for (int nb : g[node]) {
5             path.push_back(nb);
6             dfs(g, nb, path, res);
7             path.pop_back();
8         }
9     }
10 };

```

```

9     }
10    public:
11        vector<vector<int>>> allPathsSourceTarget(vector<vector<int>>& graph) {
12            vector<vector<int>>> res;
13            vector<int> path = {0};
14            dfs(graph, 0, path, res);
15            return res;
16        }
17    };

```

12.6.7 LC#841. Keys and Rooms [Medium] [A] [G]

Problem

Start in room 0. Each room has keys to other rooms. Check if you can visit all rooms.

```

1    class Solution {
2    public:
3        bool canVisitAllRooms(vector<vector<int>>& rooms) {
4            int n = rooms.size();
5            vector<bool> visited(n, false);
6            stack<int> st;
7            st.push(0);
8            visited[0] = true;
9            int cnt = 1;
10           while (!st.empty()) {
11               int room = st.top(); st.pop();
12               for (int key : rooms[room]) {
13                   if (!visited[key]) {
14                       visited[key] = true;
15                       cnt++;
16                       st.push(key);
17                   }
18               }
19           }
20           return cnt == n;
21       }
22   };

```

12.6.8 LC#886. Possible Bipartition [Medium] [A] [G]

Problem

Split people into two groups where no two disliked people are in the same group (bipartite check).

```

1    class Solution {
2    public:
3        bool possibleBipartition(int n, vector<vector<int>>& dislikes) {
4            vector<vector<int>>> adj(n + 1);
5            for (auto& d : dislikes) {
6                adj[d[0]].push_back(d[1]);
7                adj[d[1]].push_back(d[0]);
8            }
9            vector<int> color(n + 1, -1);
10           for (int i = 1; i <= n; i++) {
11               if (color[i] != -1) continue;
12               queue<int> q;
13               q.push(i);
14               color[i] = 0;
15               while (!q.empty()) {
16                   int u = q.front(); q.pop();
17                   for (int v : adj[u]) {
18                       if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
19                       else if (color[v] == color[u]) return false;
20                   }
21               }
22           }
23           return true;
24       }
25   };

```

12.6.9 LC#1020. Number of Enclaves [Medium] [A] [G]

Problem

Count land cells from which you cannot walk off the boundary of the grid.

```

1 class Solution {
2     void dfs(vector<vector<int>>& g, int i, int j) {
3         if (i < 0 || i >= (int)g.size() || j < 0 || j >= (int)g[0].size()) return;
4         if (g[i][j] != 1) return;
5         g[i][j] = 0;
6         dfs(g, i+1, j); dfs(g, i-1, j);
7         dfs(g, i, j+1); dfs(g, i, j-1);
8     }
9 public:
10    int numEnclaves(vector<vector<int>>& grid) {
11        int m = grid.size(), n = grid[0].size();
12        for (int i = 0; i < m; i++) { dfs(grid, i, 0); dfs(grid, i, n-1); }
13        for (int j = 0; j < n; j++) { dfs(grid, 0, j); dfs(grid, m-1, j); }
14        int cnt = 0;
15        for (auto& row : grid) for (int x : row) cnt += x;
16        return cnt;
17    }
18 };

```

12.6.10 LC#1319. Number of Operations to Make Network Connected [Medium] [A] [G]

Problem

Given n computers and cables, find minimum cables to reconnect everything, or -1 if impossible.

```

1 class Solution {
2     vector<int> par;
3     int find(int x) { return par[x] == x ? x : par[x] = find(par[x]); }
4 public:
5     int makeConnected(int n, vector<vector<int>>& connections) {
6         if ((int)connections.size() < n - 1) return -1;
7         par.resize(n);
8         iota(par.begin(), par.end(), 0);
9         int components = n;
10        for (auto& c : connections) {
11            int a = find(c[0]), b = find(c[1]);
12            if (a != b) { par[a] = b; components--; }
13        }
14        return components - 1;
15    }
16 };

```

12.6.11 LC#1376. Time Needed to Inform All Employees [Medium] [A] [MS]

Problem

Manager tree where each manager takes `informTime` to inform subordinates. Find total time.

```

1 class Solution {
2 public:
3     int numOfMinutes(int n, int headID, vector<int>& manager, vector<int>& informTime) {
4         vector<vector<int>> sub(n);
5         for (int i = 0; i < n; i++) {
6             if (manager[i] != -1) sub[manager[i]].push_back(i);
7         }
8         int res = 0;
9         queue<pair<int,int>> q; // {employee, time_received}
10        q.push({headID, 0});
11        while (!q.empty()) {
12            auto [emp, t] = q.front(); q.pop();
13            res = max(res, t);
14            for (int s : sub[emp]) q.push({s, t + informTime[emp]});
15        }
16        return res;
17    }
18 };

```

12.6.12 LC#1466. Reorder Routes to Make All Paths Lead to City Zero [Medium] [A] [G]

Problem

Minimum number of edges to reverse in a directed tree so all nodes can reach node 0.

```

1  class Solution {
2  public:
3      int minReorder(int n, vector<vector<int>>& connections) {
4          vector<vector<pair<int,int>>> adj(n);
5          for (auto& c : connections) {
6              adj[c[0]].push_back({c[1], 1}); // 1 = needs reversal
7              adj[c[1]].push_back({c[0], 0}); // 0 = free
8          }
9          vector<bool> visited(n, false);
10         queue<int> q;
11         q.push(0);
12         visited[0] = true;
13         int res = 0;
14         while (!q.empty()) {
15             int u = q.front(); q.pop();
16             for (auto [v, cost] : adj[u]) {
17                 if (!visited[v]) {
18                     visited[v] = true;
19                     res += cost;
20                     q.push(v);
21                 }
22             }
23         }
24         return res;
25     }
26 };

```

12.7 Dynamic Programming

12.7.1 LC#96. Unique Binary Search Trees [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Count structurally unique BSTs with n nodes (Catalan number).

```

1  class Solution {
2  public:
3      int numTrees(int n) {
4          vector<int> dp(n + 1, 0);
5          dp[0] = dp[1] = 1;
6          for (int i = 2; i <= n; i++) {
7              for (int j = 1; j <= i; j++) {
8                  dp[i] += dp[j-1] * dp[i-j];
9              }
10         }
11         return dp[n];
12     }
13 };

```

12.7.2 LC#279. Perfect Squares [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Find the minimum number of perfect squares that sum to n .

```

1  class Solution {
2  public:
3      int numSquares(int n) {
4          vector<int> dp(n + 1, INT_MAX);
5          dp[0] = 0;
6          for (int i = 1; i <= n; i++) {
7              for (int j = 1; j * j <= i; j++) {
8                  dp[i] = min(dp[i], dp[i - j*j] + 1);
9              }
10         }
11         return dp[n];
12     }
13 };

```

12.7.3 LC#309. Best Time to Buy and Sell Stock with Cooldown [\[Medium\]](#) [\[A\]](#) [\[G\]](#)

Problem

Max profit with cooldown (must wait 1 day after selling before buying again).

```

1 class Solution {
2 public:
3     int maxProfit(vector<int>& prices) {
4         int hold = INT_MIN, sold = 0, rest = 0;
5         for (int p : prices) {
6             int prevSold = sold;
7             sold = hold + p;
8             hold = max(hold, rest - p);
9             rest = max(rest, prevSold);
10        }
11        return max(sold, rest);
12    }
13 };

```

12.7.4 LC#343. Integer Break [Medium] [A] [G]

Problem

Break n into at least 2 positive integers summing to n . Maximize their product.

```

1 class Solution {
2 public:
3     int integerBreak(int n) {
4         vector<int> dp(n + 1, 0);
5         dp[1] = 1;
6         for (int i = 2; i <= n; i++) {
7             for (int j = 1; j < i; j++) {
8                 dp[i] = max(dp[i], max(j * (i - j), j * dp[i-j]));
9             }
10        }
11        return dp[n];
12    }
13 };

```

12.7.5 LC#376. Wiggle Subsequence [Medium] [A] [G]

Problem

Find the length of the longest wiggle subsequence (alternating increases and decreases).

```

1 class Solution {
2 public:
3     int wiggleMaxLength(vector<int>& nums) {
4         int up = 1, down = 1;
5         for (int i = 1; i < (int)nums.size(); i++) {
6             if (nums[i] > nums[i-1]) up = down + 1;
7             else if (nums[i] < nums[i-1]) down = up + 1;
8         }
9         return max(up, down);
10    }
11 };

```

12.7.6 LC#377. Combination Sum IV [Medium] [A] [MS] [G]

Problem

Count the number of ordered combinations of **nums** that sum to **target**.

```

1 class Solution {
2 public:
3     int combinationSum4(vector<int>& nums, int target) {
4         vector<unsigned int> dp(target + 1, 0);
5         dp[0] = 1;
6         for (int i = 1; i <= target; i++) {
7             for (int x : nums) {
8                 if (x <= i) dp[i] += dp[i - x];
9             }
10        }
11        return dp[target];
12    }
13 };

```

12.7.7 LC#413. Arithmetic Slices [Medium] [A]

Problem

Count all arithmetic slices (subarrays of length ≥ 3 with constant difference).

```

1  class Solution {
2  public:
3      int numberOfArithmeticSlices(vector<int>& nums) {
4          int cur = 0, res = 0;
5          for (int i = 2; i < (int)nums.size(); i++) {
6              if (nums[i] - nums[i-1] == nums[i-1] - nums[i-2]) {
7                  cur++;
8                  res += cur;
9              } else {
10                 cur = 0;
11             }
12         }
13         return res;
14     }
15 };

```

12.7.8 LC#486. Predict the Winner [Medium] [A] [MS] [G]

Problem

Two players take from either end. Player 1 goes first. Does player 1 win? Interval DP.

```

1  class Solution {
2  public:
3      bool predictTheWinner(vector<int>& nums) {
4          int n = nums.size();
5          // dp[i][j] = max score advantage current player can get over [i,j]
6          vector<vector<int>>> dp(n, vector<int>(n, 0));
7          for (int i = 0; i < n; i++) dp[i][i] = nums[i];
8          for (int len = 2; len <= n; len++) {
9              for (int i = 0; i <= n - len; i++) {
10                 int j = i + len - 1;
11                 dp[i][j] = max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1]);
12             }
13         }
14         return dp[0][n-1] >= 0;
15     }
16 };

```

12.7.9 LC#494. Target Sum [Medium] [A] [MS] [G]

Problem

Assign $+$ / $-$ to each number to get **target**. Count the number of ways.

```

1  class Solution {
2  public:
3      int findTargetSumWays(vector<int>& nums, int target) {
4          int sum = accumulate(nums.begin(), nums.end(), 0);
5          if (abs(target) > sum || (sum + target) % 2) return 0;
6          int pos = (sum + target) / 2;
7          vector<int> dp(pos + 1, 0);
8          dp[0] = 1;
9          for (int x : nums) {
10             for (int j = pos; j >= x; j--) {
11                 dp[j] += dp[j - x];
12             }
13         }
14         return dp[pos];
15     }
16 };

```

12.7.10 LC#516. Longest Palindromic Subsequence [Medium] [A] [MS] [G]

Problem

Find the length of the longest palindromic subsequence in **s**.

```

1 class Solution {
2 public:
3     int longestPalindromeSubseq(string s) {
4         int n = s.size();
5         vector<vector<int>> dp(n, vector<int>(n, 0));
6         for (int i = 0; i < n; i++) dp[i][i] = 1;
7         for (int len = 2; len <= n; len++) {
8             for (int i = 0; i <= n - len; i++) {
9                 int j = i + len - 1;
10                if (s[i] == s[j]) dp[i][j] = dp[i+1][j-1] + 2;
11                else dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
12            }
13        }
14        return dp[0][n-1];
15    }
16 };

```

12.7.11 LC#576. Out of Boundary Paths [Medium] [A] [G]

Problem

Count paths of length exactly `maxMove` that move a ball starting at `(startRow, startCol)` out of an `m` `n` grid.

```

1 class Solution {
2 public:
3     int findPaths(int m, int n, int maxMove, int sr, int sc) {
4         const int MOD = 1e9 + 7;
5         vector<vector<int>> dp(m, vector<int>(n, 0));
6         dp[sr][sc] = 1;
7         int res = 0;
8         int dx[] = {0, 0, 1, -1};
9         int dy[] = {1, -1, 0, 0};
10        for (int step = 0; step < maxMove; step++) {
11            vector<vector<int>> tmp(m, vector<int>(n, 0));
12            for (int i = 0; i < m; i++) {
13                for (int j = 0; j < n; j++) {
14                    if (!dp[i][j]) continue;
15                    for (int d = 0; d < 4; d++) {
16                        int ni = i + dx[d], nj = j + dy[d];
17                        if (ni < 0 || ni >= m || nj < 0 || nj >= n) res = (res + dp[i][j]) % MOD;
18                        else tmp[ni][nj] = (tmp[ni][nj] + dp[i][j]) % MOD;
19                    }
20                }
21            }
22            dp = tmp;
23        }
24        return res;
25    }
26 };

```

12.7.12 LC#583. Delete Operation for Two Strings [Medium] [A] [G]

Problem

Minimum number of deletions to make two strings equal (related to LCS).

```

1 class Solution {
2 public:
3     int minDistance(string word1, string word2) {
4         int m = word1.size(), n = word2.size();
5         // LCS length
6         vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
7         for (int i = 1; i <= m; i++) {
8             for (int j = 1; j <= n; j++) {
9                 if (word1[i-1] == word2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
10                else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
11            }
12        }
13        int lcs = dp[m][n];
14        return (m - lcs) + (n - lcs);
15    }
16 };

```

12.7.13 LC#650. 2 Keys Keyboard [Medium] [A] [G]

Problem

Start with one 'A'. You can copy all and paste. Minimum operations to get exactly n 'A's.

```

1 class Solution {
2 public:
3     int minSteps(int n) {
4         int res = 0;
5         for (int d = 2; d <= n; d++) {
6             while (n % d == 0) {
7                 res += d;
8                 n /= d;
9             }
10        }
11        return res;
12    }
13 };

```

12.7.14 LC#673. Number of Longest Increasing Subsequences [Medium] [A] [MS]

Problem

Count the number of longest increasing subsequences.

```

1 class Solution {
2 public:
3     int findNumberOfLIS(vector<int>& nums) {
4         int n = nums.size(), maxLen = 0, res = 0;
5         vector<int> len(n, 1), cnt(n, 1);
6         for (int i = 1; i < n; i++) {
7             for (int j = 0; j < i; j++) {
8                 if (nums[j] < nums[i]) {
9                     if (len[j] + 1 > len[i]) { len[i] = len[j] + 1; cnt[i] = cnt[j]; }
10                    else if (len[j] + 1 == len[i]) cnt[i] += cnt[j];
11                }
12            }
13            maxLen = max(maxLen, len[i]);
14        }
15        for (int i = 0; i < n; i++) if (len[i] == maxLen) res += cnt[i];
16        return res;
17    }
18 };

```

12.7.15 LC#740. Delete and Earn [Medium] [A] [MS]

Problem

Earn x points by deleting all occurrences of x . But must delete all occurrences of $x - 1$ and $x + 1$ first. Max total points.

```

1 class Solution {
2 public:
3     int deleteAndEarn(vector<int>& nums) {
4         int maxVal = *max_element(nums.begin(), nums.end());
5         vector<int> total(maxVal + 1, 0);
6         for (int x : nums) total[x] += x;
7         // now it's house robber on total[]
8         int prev2 = 0, prev1 = 0;
9         for (int i = 1; i <= maxVal; i++) {
10            int cur = max(prev1, prev2 + total[i]);
11            prev2 = prev1;
12            prev1 = cur;
13        }
14        return prev1;
15    }
16 };

```


12.7.16 LC#877. Stone Game [Medium] [A] [MS]

Problem

Alice and Bob take turns taking piles from either end. Alice goes first. Does Alice win? (Spoiler: she always does with optimal play — but prove it with DP.)

```

1  class Solution {
2  public:
3      bool stoneGame(vector<int>& piles) {
4          // Math: Alice always wins (she picks even or odd piles).
5          // DP approach for generality:
6          int n = piles.size();
7          vector<vector<int>>> dp(n, vector<int>(n, 0));
8          for (int i = 0; i < n; i++) dp[i][i] = piles[i];
9          for (int len = 2; len <= n; len++) {
10             for (int i = 0; i <= n - len; i++) {
11                 int j = i + len - 1;
12                 dp[i][j] = max(piles[i] - dp[i+1][j], piles[j] - dp[i][j-1]);
13             }
14         }
15         return dp[0][n-1] > 0;
16     }
17 };

```

12.7.17 LC#931. Minimum Falling Path Sum [Medium] [A] [G]

Problem

Find the minimum sum of a falling path through an $n \times n$ matrix (each step goes to adjacent column below).

```

1  class Solution {
2  public:
3      int minFallingPathSum(vector<vector<int>>& matrix) {
4          int n = matrix.size();
5          for (int i = 1; i < n; i++) {
6              for (int j = 0; j < n; j++) {
7                  int best = matrix[i-1][j];
8                  if (j > 0) best = min(best, matrix[i-1][j-1]);
9                  if (j < n-1) best = min(best, matrix[i-1][j+1]);
10                 matrix[i][j] += best;
11             }
12         }
13         return *min_element(matrix[n-1].begin(), matrix[n-1].end());
14     }
15 };

```

12.7.18 LC#983. Minimum Cost for Tickets [Medium] [A] [G]

Problem

Buy 1/7/30-day travel passes optimally to cover all travel days.

```

1  class Solution {
2  public:
3      int mincostTickets(vector<int>& days, vector<int>& costs) {
4          set<int> travelDays(days.begin(), days.end());
5          int last = days.back();
6          vector<int> dp(last + 1, 0);
7          for (int i = 1; i <= last; i++) {
8              if (!travelDays.count(i)) { dp[i] = dp[i-1]; continue; }
9              dp[i] = dp[i-1] + costs[0];
10             dp[i] = min(dp[i], dp[max(0, i-7)] + costs[1]);
11             dp[i] = min(dp[i], dp[max(0, i-30)] + costs[2]);
12         }
13         return dp[last];
14     }
15 };

```

12.7.19 LC#688. Knight Probability in Chessboard [Medium] [A] [G]

Problem

A knight makes exactly k moves on an $n \times n$ board. Find the probability it stays on the board.

```

1 class Solution {
2 public:
3     double knightProbability(int n, int k, int row, int column) {
4         vector<vector<double>> dp(n, vector<double>(n, 0));
5         dp[row][column] = 1.0;
6         int dx[] = {-2,-2,-1,-1, 1, 1, 2, 2};
7         int dy[] = {-1, 1,-2, 2,-2, 2,-1, 1};
8         for (int step = 0; step < k; step++) {
9             vector<vector<double>> tmp(n, vector<double>(n, 0));
10            for (int i = 0; i < n; i++) {
11                for (int j = 0; j < n; j++) {
12                    if (!dp[i][j]) continue;
13                    for (int d = 0; d < 8; d++) {
14                        int ni = i + dx[d], nj = j + dy[d];
15                        if (ni >= 0 && ni < n && nj >= 0 && nj < n) {
16                            tmp[ni][nj] += dp[i][j] / 8.0;
17                        }
18                    }
19                }
20            }
21            dp = tmp;
22        }
23        double res = 0;
24        for (auto& row : dp) for (double x : row) res += x;
25        return res;
26    }
27 };

```

12.7.20 LC#97. Interleaving String [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Determine if `s3` is formed by interleaving `s1` and `s2`.

```

1 class Solution {
2 public:
3     bool isInterleave(string s1, string s2, string s3) {
4         int m = s1.size(), n = s2.size();
5         if (m + n != (int)s3.size()) return false;
6         vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));
7         dp[0][0] = true;
8         for (int i = 1; i <= m; i++) dp[i][0] = dp[i-1][0] && s1[i-1] == s3[i-1];
9         for (int j = 1; j <= n; j++) dp[0][j] = dp[0][j-1] && s2[j-1] == s3[j-1];
10        for (int i = 1; i <= m; i++) {
11            for (int j = 1; j <= n; j++) {
12                dp[i][j] = (dp[i-1][j] && s1[i-1] == s3[i+j-1])
13                    || (dp[i][j-1] && s2[j-1] == s3[i+j-1]);
14            }
15        }
16        return dp[m][n];
17    }
18 };

```

12.8 Jump Game Variants

All Jump Game variants in one place for pattern recognition. #55, #45, and #1306 are also covered in their topic chapters; they are included here for the variant progression.

12.8.1 LC#55. Jump Game [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Given jump lengths, can you reach the last index? Greedy: track the furthest reachable index.

```

1 class Solution {
2 public:
3     bool canJump(vector<int>& nums) {
4         int far = 0;
5         for (int i = 0; i < (int)nums.size(); i++) {
6             if (i > far) return false;
7             far = max(far, i + nums[i]);
8         }
9         return true;
10    }
11 };

```

12.8.2 LC#45. Jump Game II [Medium] [A] [MS] [G]

Problem

Minimum jumps to reach the last index. Greedy BFS: expand the current “level” boundary.

```

1 class Solution {
2 public:
3     int jump(vector<int>& nums) {
4         int jumps = 0, curEnd = 0, farthest = 0;
5         for (int i = 0; i < (int)nums.size() - 1; i++) {
6             farthest = max(farthest, i + nums[i]);
7             if (i == curEnd) {
8                 jumps++;
9                 curEnd = farthest;
10            }
11        }
12        return jumps;
13    }
14 };

```

12.8.3 LC#1306. Jump Game III [Medium] [A] [G]

Problem

From index i , jump to $i + arr[i]$ or $i - arr[i]$. Can you reach any index with value 0? BFS.

```

1 class Solution {
2 public:
3     bool canReach(vector<int>& arr, int start) {
4         int n = arr.size();
5         vector<bool> vis(n, false);
6         queue<int> q;
7         q.push(start);
8         while (!q.empty()) {
9             int i = q.front(); q.pop();
10            if (arr[i] == 0) return true;
11            if (vis[i]) continue;
12            vis[i] = true;
13            if (i + arr[i] < n) q.push(i + arr[i]);
14            if (i - arr[i] >= 0) q.push(i - arr[i]);
15        }
16        return false;
17    }
18 };

```

12.8.4 LC#1696. Jump Game VI [Medium] [A] [G]

Problem

From index i , jump to $[i + 1, i + k]$. Collect $nums[index]$ along the way. Maximize score to reach the end. DP + sliding window max (deque).

```

1 class Solution {
2 public:
3     int maxResult(vector<int>& nums, int k) {
4         int n = nums.size();
5         vector<int> dp(n);
6         dp[0] = nums[0];
7         deque<int> dq; // indices, front = max dp value in window
8         dq.push_back(0);
9         for (int i = 1; i < n; i++) {
10            // remove out-of-window indices
11            while (!dq.empty() && dq.front() < i - k) dq.pop_front();
12            dp[i] = nums[i] + dp[dq.front()];
13            // maintain decreasing deque
14            while (!dq.empty() && dp[dq.back()] <= dp[i]) dq.pop_back();
15            dq.push_back(i);
16        }
17        return dp[n - 1];
18    }
19 };

```

12.8.5 LC#1871. Jump Game VII [Medium] [A] [G]

Problem

Binary string. From index i (which is '0'), jump to $[i + \text{minJump}, i + \text{maxJump}]$ if that index is '0'. Can you reach the last index? Prefix sum BFS.

```

1 class Solution {
2 public:
3     bool canReach(string s, int minJump, int maxJump) {
4         int n = s.size();
5         if (s[n-1] == '1') return false;
6         vector<bool> dp(n, false);
7         dp[0] = true;
8         int reachable = 0; // count of reachable positions in sliding window
9         for (int i = 1; i < n; i++) {
10             if (i >= minJump && dp[i - minJump]) reachable++;
11             if (i > maxJump && dp[i - maxJump - 1]) reachable--;
12             if (reachable > 0 && s[i] == '0') dp[i] = true;
13         }
14         return dp[n - 1];
15     }
16 };

```

12.8.6 LC#1654. Minimum Jumps to Reach Home [Medium] [A]

Problem

Jump forward by a or backward by b (no consecutive backward jumps, avoid forbidden positions). Minimum jumps to reach x . BFS with state = (position, jumped_back).

```

1 class Solution {
2 public:
3     int minimumJumps(vector<int>& forbidden, int a, int b, int x) {
4         unordered_set<int> banned(forbidden.begin(), forbidden.end());
5         int limit = x + a + b + *max_element(forbidden.begin(), forbidden.end());
6         // state: {pos, canJumpBack}
7         set<pair<int, bool>> visited;
8         queue<pair<int, bool>> q;
9         q.push({0, true});
10        visited.insert({0, true});
11        int steps = 0;
12        while (!q.empty()) {
13            int sz = q.size();
14            while (sz-- > 0) {
15                auto [pos, canBack] = q.front();
16                q.pop();
17                if (pos == x) return steps;
18                // jump forward
19                int fwd = pos + a;
20                if (fwd <= limit && !banned.count(fwd) && !visited.count({fwd, true})) {
21                    visited.insert({fwd, true});
22                    q.push({fwd, true});
23                }
24                // jump backward (only if didn't just jump back)
25                if (canBack) {
26                    int bwd = pos - b;
27                    if (bwd >= 0 && !banned.count(bwd) && !visited.count({bwd, false})) {
28                        visited.insert({bwd, false});
29                        q.push({bwd, false});
30                    }
31                }
32            }
33            steps++;
34        }
35        return -1;
36    }
37 };

```

12.9 Backtracking

12.9.1 LC#22. Generate Parentheses [Medium] [A] [MS] [G]

Problem

Generate all combinations of n pairs of well-formed parentheses.

```

1  class Solution {
2      void bt(int open, int close, int n, string cur, vector<string>& res) {
3          if (cur.size() == 2 * n) { res.push_back(cur); return; }
4          if (open < n) bt(open + 1, close, n, cur + '(', res);
5          if (close < open) bt(open, close + 1, n, cur + ')', res);
6      }
7  public:
8      vector<string> generateParenthesis(int n) {
9          vector<string> res;
10         bt(0, 0, n, "", res);
11         return res;
12     }
13 };

```

12.9.2 LC#47. Permutations II [Medium] [A] [MS]

Problem

Return all unique permutations of an array that may contain duplicates.

```

1  class Solution {
2      void bt(vector<int>& nums, vector<bool>& used, vector<int>& cur, vector<vector<int>>& res) {
3          if (cur.size() == nums.size()) { res.push_back(cur); return; }
4          for (int i = 0; i < (int)nums.size(); i++) {
5              if (used[i]) continue;
6              if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue;
7              used[i] = true;
8              cur.push_back(nums[i]);
9              bt(nums, used, cur, res);
10             cur.pop_back();
11             used[i] = false;
12         }
13     }
14  public:
15     vector<vector<int>> permuteUnique(vector<int>& nums) {
16         sort(nums.begin(), nums.end());
17         vector<vector<int>> res;
18         vector<int> cur;
19         vector<bool> used(nums.size(), false);
20         bt(nums, used, cur, res);
21         return res;
22     }
23 };

```

12.9.3 LC#93. Restore IP Addresses [Medium] [A] [MS]

Problem

Return all valid IP addresses formed from digits string *s* (no leading zeros, each octet 0–255).

```

1  class Solution {
2      void bt(string& s, int i, int part, string cur, vector<string>& res) {
3          if (part == 4 && i == (int)s.size()) { res.push_back(cur.substr(0, cur.size()-1)); return; }
4          if (part == 4 || i == (int)s.size()) return;
5          for (int len = 1; len <= 3 && i + len <= (int)s.size(); len++) {
6              string seg = s.substr(i, len);
7              if (len > 1 && seg[0] == '0') break;
8              if (stoi(seg) > 255) break;
9              bt(s, i + len, part + 1, cur + seg + ".", res);
10         }
11     }
12  public:
13     vector<string> restoreIpAddresses(string s) {
14         vector<string> res;
15         bt(s, 0, 0, "", res);
16         return res;
17     }
18 };

```

12.9.4 LC#216. Combination Sum III [Medium] [A]

Problem

Find all combinations of *k* distinct digits (1–9) that sum to *n*.

```

1 class Solution {
2     void bt(int k, int n, int start, vector<int>& cur, vector<vector<int>>& res) {
3         if (cur.size() == k && n == 0) { res.push_back(cur); return; }
4         if (cur.size() == k || n <= 0) return;
5         for (int i = start; i <= 9; i++) {
6             cur.push_back(i);
7             bt(k, n - i, i + 1, cur, res);
8             cur.pop_back();
9         }
10    }
11    public:
12        vector<vector<int>> combinationSum3(int k, int n) {
13            vector<vector<int>> res;
14            vector<int> cur;
15            bt(k, n, 1, cur, res);
16            return res;
17        }
18    };

```

12.9.5 LC#526. Beautiful Arrangement [Medium] [A]

Problem

Count permutations of 1..n where position i is divisible by $\text{perm}[i]$ or vice versa.

```

1 class Solution {
2     int res = 0;
3     void bt(int n, int pos, vector<bool>& used) {
4         if (pos > n) { res++; return; }
5         for (int i = 1; i <= n; i++) {
6             if (!used[i] && (pos % i == 0 || i % pos == 0)) {
7                 used[i] = true;
8                 bt(n, pos + 1, used);
9                 used[i] = false;
10            }
11        }
12    }
13    public:
14        int countArrangement(int n) {
15            vector<bool> used(n + 1, false);
16            bt(n, 1, used);
17            return res;
18        }
19    };

```

12.9.6 LC#784. Letter Case Permutation [Medium] [A] [MS]

Problem

For each letter in string s , transform it to upper or lower case. Return all possibilities.

```

1 class Solution {
2     public:
3         vector<string> letterCasePermutation(string s) {
4             vector<string> res;
5             function<void(int)> bt = [&](int i) {
6                 if (i == (int)s.size()) { res.push_back(s); return; }
7                 bt(i + 1);
8                 if (isalpha(s[i])) {
9                     s[i] ^= 32; // toggle case
10                    bt(i + 1);
11                    s[i] ^= 32;
12                }
13            };
14            bt(0);
15            return res;
16        }
17    };

```

12.9.7 LC#967. Numbers With Same Consecutive Differences [Medium] [A]

Problem

Return all n -digit numbers where absolute difference of consecutive digits equals k .

```

1  class Solution {
2  public:
3      vector<int> numsSameConsecDiff(int n, int k) {
4          vector<int> cur = {1, 2, 3, 4, 5, 6, 7, 8, 9};
5          if (n == 1) { cur.push_back(0); return cur; }
6          vector<int> res;
7          queue<int> q(deque<int>(cur.begin(), cur.end()));
8          for (int step = 1; step < n; step++) {
9              int sz = q.size();
10             while (sz-- > 0) {
11                 int num = q.front(); q.pop();
12                 int lastDigit = num % 10;
13                 if (lastDigit + k <= 9) q.push(num * 10 + lastDigit + k);
14                 if (k != 0 && lastDigit - k >= 0) q.push(num * 10 + lastDigit - k);
15             }
16         }
17         while (!q.empty()) { res.push_back(q.front()); q.pop(); }
18         return res;
19     }
20 };

```

12.9.8 LC#1079. Letter Tile Possibilities [Medium] [A]

Problem

Count all non-empty sequences you can make using the letters (tiles) in tiles.

```

1  class Solution {
2  public:
3      int bt(int freq[26]) {
4          int res = 0;
5          for (int i = 0; i < 26; i++) {
6              if (freq[i] > 0) {
7                  freq[i]--;
8                  res += 1 + bt(freq);
9                  freq[i]++;
10             }
11         }
12         return res;
13     }
14     public:
15     int numTilePossibilities(string tiles) {
16         int freq[26] = {};
17         for (char c : tiles) freq[c - 'A']++;
18         return bt(freq);
19     }
20 };

```

12.10 Stack & Monotonic Stack

12.10.1 LC#71. Simplify Path [Medium] [A] [MS] [G]

Problem

Simplify a Unix file path string (handle ., .., and multiple slashes).

```

1  class Solution {
2  public:
3      string simplifyPath(string path) {
4          vector<string> stk;
5          stringstream ss(path);
6          string token;
7          while (getline(ss, token, '/')) {
8              if (token == "" || token == ".") continue;
9              if (token == "..") {
10                 if (!stk.empty()) stk.pop_back();
11             } else {
12                 stk.push_back(token);
13             }
14         }
15         string res;
16         for (auto& s : stk) res += "/" + s;
17         return res.empty() ? "/" : res;
18     }
19 };

```

12.10.2 LC#316. Remove Duplicate Letters [Medium] [A] [MS] [G]

Problem

Remove duplicate letters so result is smallest in lexicographic order with each letter appearing once.

```

1 class Solution {
2 public:
3     string removeDuplicateLetters(string s) {
4         int last[26] = {};
5         bool inStack[26] = {};
6         for (int i = 0; i < (int)s.size(); i++) last[s[i] - 'a'] = i;
7         string stk;
8         for (int i = 0; i < (int)s.size(); i++) {
9             int c = s[i] - 'a';
10            if (inStack[c]) continue;
11            while (!stk.empty() && stk.back() > s[i] && last[stk.back() - 'a'] > i) {
12                inStack[stk.back() - 'a'] = false;
13                stk.pop_back();
14            }
15            stk.push_back(s[i]);
16            inStack[c] = true;
17        }
18        return stk;
19    }
20 };

```

12.10.3 LC#402. Remove K Digits [Medium] [A] [MS] [G]

Problem

Remove k digits to get the smallest possible number (as a string, no leading zeros).

```

1 class Solution {
2 public:
3     string removeKdigits(string num, int k) {
4         string stk;
5         for (char c : num) {
6             while (k > 0 && !stk.empty() && stk.back() > c) {
7                 stk.pop_back();
8                 k--;
9             }
10            stk.push_back(c);
11        }
12        while (k-- > 0) stk.pop_back();
13        // remove leading zeros
14        int start = 0;
15        while (start < (int)stk.size() - 1 && stk[start] == '0') start++;
16        return stk.substr(start);
17    }
18 };

```

12.10.4 LC#456. 132 Pattern [Medium] [A] [MS] [G]

Problem

Return true if there exist $i < j < k$ such that $nums[i] < nums[k] < nums[j]$.

```

1 class Solution {
2 public:
3     bool find132pattern(vector<int>& nums) {
4         stack<int> st;
5         int third = INT_MIN; // the "2" in 132 pattern (nums[k])
6         for (int i = nums.size() - 1; i >= 0; i--) {
7             if (nums[i] < third) return true;
8             while (!st.empty() && nums[i] > st.top()) {
9                 third = st.top();
10                st.pop();
11            }
12            st.push(nums[i]);
13        }
14        return false;
15    }
16 };

```


12.10.5 LC#503. Next Greater Element II [Medium] [A] [G]

Problem

Circular array — find the next greater element for each (wrap around). Use monotonic stack.

```

1 class Solution {
2 public:
3     vector<int> nextGreaterElements(vector<int>& nums) {
4         int n = nums.size();
5         vector<int> res(n, -1);
6         stack<int> st;
7         for (int i = 0; i < 2 * n; i++) {
8             while (!st.empty() && nums[st.top()] < nums[i % n]) {
9                 res[st.top()] = nums[i % n];
10                st.pop();
11            }
12            if (i < n) st.push(i);
13        }
14        return res;
15    }
16 };

```

12.10.6 LC#735. Asteroid Collision [Medium] [A] [MS] [G]

Problem

Positive asteroids move right, negative move left. Simulate collisions (same-size both explode, smaller explodes).

```

1 class Solution {
2 public:
3     vector<int> asteroidCollision(vector<int>& asteroids) {
4         stack<int> st;
5         for (int a : asteroids) {
6             bool survived = true;
7             while (survived && a < 0 && !st.empty() && st.top() > 0) {
8                 if (st.top() < -a) st.pop();
9                 else if (st.top() == -a) { st.pop(); survived = false; }
10                else survived = false;
11            }
12            if (survived) st.push(a);
13        }
14        vector<int> res(st.size());
15        for (int i = st.size() - 1; i >= 0; i--) { res[i] = st.top(); st.pop(); }
16        return res;
17    }
18 };

```

12.10.7 LC#856. Score of Parentheses [Medium] [A] [G]

Problem

Compute score of parentheses: () = 1, AB = A+B, (A) = 2A.

```

1 class Solution {
2 public:
3     int scoreOfParentheses(string s) {
4         stack<int> st;
5         st.push(0);
6         for (char c : s) {
7             if (c == '(') {
8                 st.push(0);
9             } else {
10                int v = st.top(); st.pop();
11                st.top() += max(2 * v, 1);
12            }
13        }
14        return st.top();
15    }
16 };

```

12.10.8 LC#1019. Next Greater Node in Linked List [Medium] [A]

Problem

For each node in a linked list, return the value of the next node with a greater value.

```

1  class Solution {
2  public:
3      vector<int> nextLargerNodes(ListNode* head) {
4          vector<int> vals;
5          while (head) { vals.push_back(head->val); head = head->next; }
6          int n = vals.size();
7          vector<int> res(n, 0);
8          stack<int> st; // indices
9          for (int i = 0; i < n; i++) {
10             while (!st.empty() && vals[st.top()] < vals[i]) {
11                 res[st.top()] = vals[i];
12                 st.pop();
13             }
14             st.push(i);
15         }
16         return res;
17     }
18 };

```

12.10.9 LC#2390. Removing Stars From a String [Medium] [A] [MS]

Problem

A star removes the closest non-star character to its left. Return the resulting string.

```

1  class Solution {
2  public:
3      string removeStars(string s) {
4          string stk;
5          for (char c : s) {
6              if (c == '*') stk.pop_back();
7              else stk.push_back(c);
8          }
9          return stk;
10     }
11 };

```

12.10.10 LC#946. Validate Stack Sequences [Medium] [A] [MS]

Problem

Given pushed and popped sequences, determine if they could be valid stack operations.

```

1  class Solution {
2  public:
3      bool validateStackSequences(vector<int>& pushed, vector<int>& popped) {
4          stack<int> st;
5          int j = 0;
6          for (int x : pushed) {
7              st.push(x);
8              while (!st.empty() && st.top() == popped[j]) {
9                  st.pop();
10                 j++;
11             }
12         }
13         return st.empty();
14     }
15 };

```

12.11 Heaps & Priority Queues

12.11.1 LC#373. Find K Pairs with Smallest Sums [Medium] [A] [MS] [G]

Problem

Given two sorted arrays, find k pairs with the smallest sums. Use a min-heap.

```

1  class Solution {
2  public:
3      vector<vector<int>> kSmallestPairs(vector<int>& nums1, vector<int>& nums2, int k) {
4          using T = tuple<int, int, int>;
5          priority_queue<T, vector<T>, greater<T>> pq;
6          for (int i = 0; i < min(k, (int)nums1.size()); i++) {
7              pq.push({nums1[i] + nums2[0], i, 0});
8          }
9          vector<vector<int>> res;
10         while (!pq.empty() && (int)res.size() < k) {
11             auto [sum, i, j] = pq.top(); pq.pop();
12             res.push_back({nums1[i], nums2[j]});
13             if (j + 1 < (int)nums2.size()) {
14                 pq.push({nums1[i] + nums2[j+1], i, j + 1});
15             }
16         }
17         return res;
18     }
19 };

```

12.11.2 LC#451. Sort Characters By Frequency [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Sort characters by decreasing frequency.

```

1  class Solution {
2  public:
3      string frequencySort(string s) {
4          unordered_map<char, int> freq;
5          for (char c : s) freq[c]++;
6          priority_queue<pair<int, char>> pq;
7          for (auto& [c, f] : freq) pq.push({f, c});
8          string res;
9          while (!pq.empty()) {
10             auto [f, c] = pq.top(); pq.pop();
11             res += string(f, c);
12         }
13         return res;
14     }
15 };

```

12.11.3 LC#692. Top K Frequent Words [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Return the k most frequent words. Ties broken alphabetically.

```

1  class Solution {
2  public:
3      vector<string> topKFrequent(vector<string>& words, int k) {
4          unordered_map<string, int> freq;
5          for (auto& w : words) freq[w]++;
6          auto cmp = [](pair<int, string>& a, pair<int, string>& b) {
7              return a.first > b.first || (a.first == b.first && a.second < b.second);
8          };
9          priority_queue<pair<int, string>, vector<pair<int, string>>, decltype(cmp)> pq(cmp);
10         for (auto& [w, f] : freq) {
11             pq.push({f, w});
12             if ((int)pq.size() > k) pq.pop();
13         }
14         vector<string> res;
15         while (!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
16         reverse(res.begin(), res.end());
17         return res;
18     }
19 };

```

12.11.4 LC#703. Kth Largest Element in a Stream [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Design a class to find the k -th largest element in a stream. Use a min-heap of size k .

```

1  class KthLargest {
2      int k;

```

```

3 priority_queue<int, vector<int>, greater<int>> pq;
4 public:
5     KthLargest(int k, vector<int>& nums) : k(k) {
6         for (int x : nums) add(x);
7     }
8     int add(int val) {
9         pq.push(val);
10        if ((int)pq.size() > k) pq.pop();
11        return pq.top();
12    }
13 };

```

12.11.5 LC#767. Reorganize String [Medium] [A] [MS] [G]

Problem

Rearrange characters so no two adjacent characters are the same, or return "" if impossible.

```

1 class Solution {
2 public:
3     string reorganizeString(string s) {
4         int freq[26] = {};
5         for (char c : s) freq[c - 'a']++;
6         priority_queue<pair<int, char>> pq;
7         for (int i = 0; i < 26; i++) if (freq[i]) pq.push({freq[i], 'a' + i});
8         string res;
9         while (pq.size() >= 2) {
10            auto [f1, c1] = pq.top(); pq.pop();
11            auto [f2, c2] = pq.top(); pq.pop();
12            res += c1;
13            res += c2;
14            if (f1 > 1) pq.push({f1 - 1, c1});
15            if (f2 > 1) pq.push({f2 - 1, c2});
16        }
17        if (!pq.empty()) {
18            if (pq.top().first > 1) return "";
19            res += pq.top().second;
20        }
21        return res;
22    }
23 };

```

12.11.6 LC#1438. Longest Continuous Subarray With Absolute Diff $\leq$ Limit [Medium] [A] [G]

Problem

Sliding window with two deques tracking max and min. Shrink when $\text{max} - \text{min} > \text{limit}$.

```

1 class Solution {
2 public:
3     int longestSubarray(vector<int>& nums, int limit) {
4         deque<int> maxDq, minDq;
5         int l = 0, res = 0;
6         for (int r = 0; r < (int)nums.size(); r++) {
7             while (!maxDq.empty() && nums[maxDq.back()] <= nums[r]) maxDq.pop_back();
8             while (!minDq.empty() && nums[minDq.back()] >= nums[r]) minDq.pop_back();
9             maxDq.push_back(r);
10            minDq.push_back(r);
11            while (nums[maxDq.front()] - nums[minDq.front()] > limit) {
12                l++;
13                if (maxDq.front() < l) maxDq.pop_front();
14                if (minDq.front() < l) minDq.pop_front();
15            }
16            res = max(res, r - l + 1);
17        }
18        return res;
19    }
20 };

```

12.11.7 LC#1642. Furthest Building You Can Reach [Medium] [A] [G]

Problem

With bricks and ladders, reach as far as possible. Use ladders on the biggest jumps (greedy + min-heap).

```

1  class Solution {
2  public:
3      int furthestBuilding(vector<int>& heights, int bricks, int ladders) {
4          priority_queue<int, vector<int>, greater<int>> pq; // min-heap of ladder usages
5          for (int i = 0; i < (int)heights.size() - 1; i++) {
6              int diff = heights[i+1] - heights[i];
7              if (diff <= 0) continue;
8              pq.push(diff);
9              if ((int)pq.size() > ladders) {
10                 // swap smallest ladder use for bricks
11                 bricks -= pq.top();
12                 pq.pop();
13             }
14             if (bricks < 0) return i;
15         }
16         return heights.size() - 1;
17     }
18 };

```

12.12 Greedy & Intervals

12.12.1 LC#122. Best Time to Buy and Sell Stock II [Medium] [A] [MS] [G]

Problem

Unlimited transactions (not simultaneously). Maximize total profit.

```

1  class Solution {
2  public:
3      int maxProfit(vector<int>& prices) {
4          int res = 0;
5          for (int i = 1; i < (int)prices.size(); i++) {
6              if (prices[i] > prices[i-1]) res += prices[i] - prices[i-1];
7          }
8          return res;
9      }
10 };

```

12.12.2 LC#605. Can Place Flowers [Medium] [A] [MS]

Problem

Place n new flowers in a flowerbed where 1s represent flowers (no two adjacent). Is it possible?

```

1  class Solution {
2  public:
3      bool canPlaceFlowers(vector<int>& flowerbed, int n) {
4          int size = flowerbed.size();
5          for (int i = 0; i < size && n > 0; i++) {
6              if (flowerbed[i] == 0) {
7                  bool leftEmpty = (i == 0 || flowerbed[i-1] == 0);
8                  bool rightEmpty = (i == size-1 || flowerbed[i+1] == 0);
9                  if (leftEmpty && rightEmpty) {
10                     flowerbed[i] = 1;
11                     n--;
12                 }
13             }
14         }
15         return n <= 0;
16     }
17 };

```

12.12.3 LC#649. Dota2 Senate [Medium] [A] [MS]

Problem

Two factions vote to ban each other. Senators act in order; each can ban the next opposing senator. Who wins?

```

1  class Solution {
2  public:
3      string predictPartyVictory(string senate) {
4          queue<int> radiant, dire;
5          int n = senate.size();

```

```

6     for (int i = 0; i < n; i++) {
7         if (senate[i] == 'R') radiant.push(i);
8         else dire.push(i);
9     }
10    while (!radiant.empty() && !dire.empty()) {
11        int r = radiant.front(); radiant.pop();
12        int d = dire.front(); dire.pop();
13        if (r < d) radiant.push(r + n);
14        else dire.push(d + n);
15    }
16    return radiant.empty() ? "Dire" : "Radiant";
17 }
18 };

```

12.12.4 LC#714. Best Time to Buy and Sell Stock with Transaction Fee [Medium] [A] [G]

Problem

Max profit with unlimited transactions but a fee per transaction.

```

1  class Solution {
2  public:
3      int maxProfit(vector<int>& prices, int fee) {
4          int hold = -prices[0], cash = 0;
5          for (int i = 1; i < (int)prices.size(); i++) {
6              int prevHold = hold;
7              hold = max(hold, cash - prices[i]);
8              cash = max(cash, prevHold + prices[i] - fee);
9          }
10         return cash;
11     }
12 };

```

12.12.5 LC#826. Most Profit Assigning Work [Medium] [A] [G]

Problem

Each worker can do one job with difficulty $\leq$ their ability. Assign jobs to maximize total profit.

```

1  class Solution {
2  public:
3      int maxProfitAssignment(vector<int>& difficulty, vector<int>& profit, vector<int>& worker) {
4          int n = difficulty.size();
5          vector<pair<int,int>> jobs(n);
6          for (int i = 0; i < n; i++) jobs[i] = {difficulty[i], profit[i]};
7          sort(jobs.begin(), jobs.end());
8          sort(worker.begin(), worker.end());
9          int res = 0, best = 0, j = 0;
10         for (int w : worker) {
11             while (j < n && jobs[j].first <= w) {
12                 best = max(best, jobs[j].second);
13                 j++;
14             }
15             res += best;
16         }
17         return res;
18     }
19 };

```

12.12.6 LC#881. Boats to Save People [Medium] [A] [G]

Problem

Each boat holds at most 2 people with total weight $\leq$ limit. Minimum boats needed.

```

1  class Solution {
2  public:
3      int numRescueBoats(vector<int>& people, int limit) {
4          sort(people.begin(), people.end());
5          int l = 0, r = people.size() - 1, boats = 0;
6          while (l <= r) {
7              if (people[l] + people[r] <= limit) l++;
8              r--;
9              boats++;
10         }

```

```

11     return boats;
12 }
13 };

```

12.12.7 LC#948. Bag of Tokens [Medium] [A]

Problem

Play tokens face-up (spend power, gain score) or face-down (spend score, gain power). Maximize score.

```

1 class Solution {
2 public:
3     int bagOfTokensScore(vector<int>& tokens, int power) {
4         sort(tokens.begin(), tokens.end());
5         int l = 0, r = tokens.size() - 1, score = 0, res = 0;
6         while (l <= r) {
7             if (power >= tokens[l]) {
8                 power -= tokens[l++];
9                 score++;
10                res = max(res, score);
11            } else if (score > 0) {
12                power += tokens[r--];
13                score--;
14            } else {
15                break;
16            }
17        }
18        return res;
19    }
20 };

```

12.12.8 LC#1029. Two City Scheduling [Medium] [A] [G]

Problem

Send n people to city A and n to city B. Minimize total cost. Sort by cost difference.

```

1 class Solution {
2 public:
3     int twoCitySchedCost(vector<vector<int>>& costs) {
4         sort(costs.begin(), costs.end(), [](auto& a, auto& b) {
5             return (a[0] - a[1]) < (b[0] - b[1]);
6         });
7         int n = costs.size() / 2, res = 0;
8         for (int i = 0; i < n; i++) res += costs[i][0];
9         for (int i = n; i < 2*n; i++) res += costs[i][1];
10        return res;
11    }
12 };

```

12.12.9 LC#870. Advantage Shuffle [Medium] [A] [G]

Problem

Rearrange `nums1` to maximize the number of indices where `nums1[i] > nums2[i]` (greedy matching).

```

1 class Solution {
2 public:
3     vector<int> advantageCount(vector<int>& nums1, vector<int>& nums2) {
4         int n = nums1.size();
5         sort(nums1.begin(), nums1.end());
6         // pair each nums2[i] with its index, sort by value desc
7         vector<pair<int, int>> sorted2(n);
8         for (int i = 0; i < n; i++) sorted2[i] = {nums2[i], i};
9         sort(sorted2.rbegin(), sorted2.rend());
10        vector<int> res(n);
11        int lo = 0, hi = n - 1;
12        for (auto& [val, idx] : sorted2) {
13            if (nums1[hi] > val) res[idx] = nums1[hi--];
14            else res[idx] = nums1[lo++];
15        }
16        return res;
17    }
18 };

```

12.12.10 LC#1053. Previous Permutation With One Swap [\[Medium\]](#) [\[A\]](#)

Problem

Find the largest permutation less than `arr` obtainable by one swap (or return `arr` if none).

```

1 class Solution {
2 public:
3     vector<int> prevPermOpt1(vector<int>& arr) {
4         int n = arr.size();
5         // find rightmost descent
6         int i = n - 2;
7         while (i >= 0 && arr[i] <= arr[i+1]) i--;
8         if (i < 0) return arr;
9         // find rightmost element < arr[i] and maximum among those
10        int j = n - 1;
11        while (arr[j] >= arr[i]) j--;
12        // among equal candidates, pick leftmost to avoid ties
13        while (j > 0 && arr[j] == arr[j-1]) j--;
14        swap(arr[i], arr[j]);
15        return arr;
16    }
17 };

```

12.13 Math & Bit Manipulation

12.13.1 LC#50. Pow(x, n) [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Implement x^n (fast exponentiation, handle negative n).

```

1 class Solution {
2     double power(double x, long n) {
3         if (n == 0) return 1;
4         if (n < 0) { x = 1 / x; n = -n; }
5         double half = power(x, n / 2);
6         return n % 2 == 0 ? half * half : half * half * x;
7     }
8 public:
9     double myPow(double x, int n) {
10        return power(x, (long)n);
11    }
12 };

```

12.13.2 LC#137. Single Number II [\[Medium\]](#) [\[A\]](#) [\[MS\]](#) [\[G\]](#)

Problem

Every element appears 3 times except one. Find that one in $O(n)$ time, $O(1)$ space (bit counting).

```

1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         int res = 0;
5         for (int bit = 0; bit < 32; bit++) {
6             int sum = 0;
7             for (int x : nums) sum += (x >> bit) & 1;
8             res |= (sum % 3) << bit;
9         }
10        return res;
11    }
12 };

```

12.13.3 LC#201. Bitwise AND of Numbers Range [\[Medium\]](#) [\[A\]](#) [\[MS\]](#)

Problem

Return the bitwise AND of all numbers in range `[left, right]`. Find common prefix.


```

1 class Solution {
2 public:
3     int rangeBitwiseAnd(int left, int right) {
4         int shift = 0;
5         while (left != right) {
6             left >>= 1;
7             right >>= 1;
8             shift++;
9         }
10        return left << shift;
11    }
12 };

```

12.13.4 LC#260. Single Number III [Medium] [A] [G]

Problem

Every element appears twice except exactly two elements. Find those two in $O(n)$ time, $O(1)$ space.

```

1 class Solution {
2 public:
3     vector<int> singleNumber(vector<int>& nums) {
4         long xorAll = 0;
5         for (int x : nums) xorAll ^= x;
6         // lowest set bit differentiates the two unique numbers
7         long diff = xorAll & (-xorAll);
8         int a = 0;
9         for (int x : nums) if (x & diff) a ^= x;
10        return {a, (int)(xorAll ^ a)};
11    }
12 };

```

12.13.5 LC#371. Sum of Two Integers [Medium] [A] [MS]

Problem

Add two integers without using $+$ or $-$ operators (bit manipulation).

```

1 class Solution {
2 public:
3     int getSum(int a, int b) {
4         while (b != 0) {
5             unsigned int carry = (unsigned int)(a & b) << 1;
6             a = a ^ b;
7             b = carry;
8         }
9         return a;
10    }
11 };

```

12.13.6 LC#338. Counting Bits [Medium] [A] [MS] [G]

Problem

Return an array where $\text{ans}[i] = \text{number of 1 bits in } i$, for $0 \leq i \leq n$.

```

1 class Solution {
2 public:
3     vector<int> countBits(int n) {
4         vector<int> dp(n + 1, 0);
5         for (int i = 1; i <= n; i++) {
6             dp[i] = dp[i >> 1] + (i & 1);
7         }
8         return dp;
9     }
10 };

```

12.13.7 LC#397. Integer Replacement [Medium] [A]

Problem

Reduce n to 1 using minimum operations: if even divide by 2, if odd add or subtract 1. BFS/greedy.

```

1  class Solution {
2  public:
3      int integerReplacement(int n) {
4          int res = 0;
5          long x = n;
6          while (x != 1) {
7              if (x % 2 == 0) {
8                  x /= 2;
9              } else if (x == 3 || ((x >> 1) & 1) == 0) {
10                 x--;
11             } else {
12                 x++;
13             }
14             res++;
15         }
16         return res;
17     }
18 };

```

12.13.8 LC#477. Total Hamming Distance [Medium] [A] [MS]

Problem

Find the total Hamming distance between all pairs of integers in `nums`.

```

1  class Solution {
2  public:
3      int totalHammingDistance(vector<int>& nums) {
4          int res = 0, n = nums.size();
5          for (int bit = 0; bit < 32; bit++) {
6              int ones = 0;
7              for (int x : nums) ones += (x >> bit) & 1;
8              res += ones * (n - ones);
9          }
10         return res;
11     }
12 };

```

12.13.9 LC#779. K-th Symbol in Grammar [Medium] [A]

Problem

Row 1: "0". Each row: $0 \rightarrow 01$, $1 \rightarrow 10$. Find the k -th symbol in row n (1-indexed, recursive).

```

1  class Solution {
2  public:
3      int kthGrammar(int n, int k) {
4          if (n == 1) return 0;
5          int parent = kthGrammar(n - 1, (k + 1) / 2);
6          bool isLeftChild = (k % 2 == 1);
7          if (parent == 0) return isLeftChild ? 0 : 1;
8          else return isLeftChild ? 1 : 0;
9      }
10 };

```

12.13.10 LC#29. Divide Two Integers [Medium] [A] [MS]

Problem

Divide two integers without using multiplication, division, or mod. Handle overflow.

```

1  class Solution {
2  public:
3      int divide(int dividend, int divisor) {
4          if (dividend == INT_MIN && divisor == -1) return INT_MAX;
5          long a = abs((long)dividend), b = abs((long)divisor);
6          int res = 0;
7          while (a >= b) {
8              long tmp = b, multiple = 1;
9              while (a >= (tmp << 1)) { tmp <<= 1; multiple <<= 1; }
10             a -= tmp;
11             res += multiple;
12         }
13         return (dividend > 0) == (divisor > 0) ? res : -res;
14     }
15 };

```